The Complete DSA Interview Guide — From Zero to Advanced

> **One README to rule them all.** This document is designed so that you do **NOT** need to grind 500+ LeetCode problems. Instead, you master roughly **16 patterns** and **~150 anchor problems**, and you will be able to solve 90% of interview questions asked at product-based companies (FAANG, unicorns, startups) and service-based companies.
>
> Format used throughout for every important question:
>
> 1. **Problem** (explained in simple words, like you're explaining to a friend)
> 2. **Brute Force** approach (usually O(n²) or worse) — so you understand *why* it's slow
> 3. **Optimized** approach (usually O(n) or O(n log n)) — the interview-winning solution
> 4. **Code** (Java primarily, with Python/C++ notes where useful)
> 5. **Pattern tag** — so you can connect it to the master pattern list

---

📚 Table of Contents

1. How to Use This Guide (Read This First)
2. Complexity Analysis Fundamentals (Big O, Big Θ, Big Ω)
3. Math & Bit Manipulation Foundations
4. Arrays & Strings — Basics to Advanced
5. Hashing (HashMap / HashSet)
6. Linked Lists — Singly, Doubly, Circular
7. Stacks & Queues (incl. Monotonic Stack/Queue)
8. Recursion & Backtracking
9. Sorting Algorithms — All of Them, Explained
10. Searching & Binary Search Patterns
11. Trees — Binary Trees, BST, Balanced Trees, Tries
12. Heaps / Priority Queues
13. Graphs — Representation to Shortest Paths

14. Dynamic Programming — Every Pattern Explained
15. Greedy Algorithms
16. Advanced Data Structures (Segment Tree, Fenwick Tree, DSU, LRU/LFU)
17. The Master Pattern Cheat-Sheet (16 Patterns)
18. Top Interview Questions by Pattern (Explained Simply)
19. The No-Grind Practice Roadmap
20. Complexity Cheat Sheet & Common Mistakes
21. Interview Day Tips & Communication Templates
22. Final Words & Resources
23. Additional Graph Algorithms (SCC, Bridges, Eulerian Path, Max-Flow)
24. Advanced String Algorithms (KMP, Z-Algorithm, Rabin-Karp, Manacher's)
25. Combinatorics, Probability, and Math for Interviews
26. Data Structure Design Questions
27. Worked Numeric Dry-Runs of Key Algorithms
28. Language Cheat Sheet (Java / Python / C++)
29. Quick-Reference Problem Index (175 Problems)
30. Frequently Asked Questions
31. Closing Summary

---

1. How to Use This Guide (Read This First)

1.1 The Core Philosophy

Most people fail DSA interviews not because they don't know enough problems, but because they don't recognize **patterns**. There are only a handful of "shapes" that interview problems take. Once you can look at a problem and say *"Ah, this is a sliding window problem"* or *"this smells like a two-pointer problem"*, you stop needing to memorize individual solutions and start **deriving** them.

This guide is structured in two layers:

- **Layer 1 — Foundations (Sections 3–16):** Every core data structure and algorithm, explained from absolute basics (what is an array, what is Big O) up to advanced topics (segment trees, Dijkstra, digit DP). Read this once, slowly, understanding *why* each structure exists and what problem it solves.

- **Layer 2 — Pattern Mastery (Sections 17–19):** The condensed, high-leverage part. This is what you revise the night before an interview. 16 patterns, each with a template, a trigger ("when should I use this pattern?"), and 5-10 representative problems.

1.2 The "No 500 Problems" Strategy

Here's the honest truth: solving 500 problems without understanding patterns is like memorizing 500 sentences in a foreign language without learning grammar. You'll freeze the moment you see a new sentence.

Instead:

1. **Learn 1 pattern at a time** (allocate 2-4 days per pattern).
2. **Solve 5-8 problems per pattern** — no more. If you can solve the 5th problem in a pattern without looking at the solution, you've mastered it.
3. **Re-derive, don't memorize.** Before coding, always ask: *"What is the brute force? Why is it slow? What redundant work am I repeating? Can I cache it / use two pointers / use a heap to avoid repeating it?"*
4. **Do timed mock problems weekly** to simulate interview pressure.
5. **Explain out loud** (or to a rubber duck) — verbalizing your approach is 50% of the interview score.

By the end of this guide, you'll have internalized these patterns:

#	Pattern	Typical Time Complexity Achieved
1	Two Pointers	O(n)
2	Sliding Window	O(n)
3	Fast & Slow Pointers (Floyd's Cycle)	O(n)
4	Merge Intervals	O(n log n)
5	Cyclic Sort	O(n)
6	In-place Linked List Reversal	O(n)
7	Tree BFS	O(n)
8	Tree DFS	O(n)
9	Two Heaps	O(n log n)

#	Pattern	Typical Time Complexity Achieved
10	Subsets (Backtracking)	O(2^n) but optimal for the problem
11	Modified Binary Search	O(log n)
12	Top K Elements	O(n log k)
13	K-way Merge	O(n log k)
14	Topological Sort	O(V + E)
15	Dynamic Programming (Knapsack/LCS/etc.)	O(n·m) usually
16	Monotonic Stack/Queue	O(n)

1.3 How Interviews Are Actually Graded

Every major tech company grades on roughly these axes:

- **Problem-solving:** Did you identify the right approach, and could you get there with reasonable hints?
- **Coding:** Is your code correct, clean, and does it compile/run mentally without bugs?
- **Communication:** Did you explain your thought process, trade-offs, and complexity?
- **Testing:** Did you walk through edge cases (empty input, single element, duplicates, negative numbers)?

A "brute force but well communicated with correct complexity analysis, then optimized" answer often scores **higher** than a silent, memorized optimal solution typed without explanation.

---

2. Complexity Analysis Fundamentals

2.1 Why Complexity Matters

When we say an algorithm is "fast," we don't mean fast on your laptop — we mean it **scales well** as input size `n` grows. Complexity analysis lets us predict behavior for `n = 10^9` without running the code.

2.2 Big O, Big Ω, Big Θ — In Plain English

- **Big O (O)** — the **worst-case upper bound**. "This algorithm will never take longer than this."
- **Big Ω (Omega)** — the **best-case lower bound**. "This algorithm will never be faster than this."
- **Big Θ (Theta)** — the **tight bound** — when best and worst case behave the same asymptotically. This is what people usually mean colloquially when they say "Big O" in interviews.

In interviews, always state complexity in terms of **Big O** (worst case) unless asked otherwise.

2.3 Common Complexity Classes (Fastest to Slowest)

Complexity	Name	Example	n=10	n=10,000	n=10^9
O(1)	Constant	Array index access	instant	instant	instant
O(log n)	Logarithmic	Binary search	instant	instant	~30 steps
O(n)	Linear	Single loop	instant	instant	~1 sec
O(n log n)	Linearithmic	Merge sort	instant	instant	~30 sec
O(n²)	Quadratic	Nested loops	instant	~1 sec	never finishes
O(2^n)	Exponential	Subsets, naive recursion	instant	never finishes	never finishes
O(n!)	Factorial	Permutations, brute-force TSP	instant	never finishes	never finishes

Rule of thumb for interviews: If `n ≤ 10^5` or higher is mentioned in constraints, an O(n²) solution will likely **Time Limit Exceed (TLE)**. You need O(n log n) or better. If `n ≤ 20`, exponential/bitmask solutions are expected.

2.4 Space Complexity

Space complexity measures extra memory used (not counting input). Common categories:

- **O(1) auxiliary space** — in-place algorithms (e.g., swapping in-place, two pointers).
- **O(n) space** — using a hashmap, extra array, or recursion stack.
- **O(h) space** for tree recursion, where `h` is tree height (can be O(log n) balanced or O(n) skewed).

Always mention **both time and space complexity** in interviews — many interviewers explicitly ask "can you do this with O(1) extra space?"

2.5 Amortized Analysis

Some operations are expensive occasionally but cheap on average. Classic example: **dynamic array (ArrayList/Vector)** `push_back`. Most insertions are O(1), but when the array is full, resizing costs O(n). Averaged over `n` insertions, it's still O(1) amortized because resizing happens only at doubling intervals (1, 2, 4, 8, ...), so total resizing cost across `n` inserts is O(n), giving O(1) per insert on average.

How to compute Big O quickly by counting loops:

- One loop over n → O(n)
- Nested loop (loop inside loop, both over n) → O(n²)
- Loop that halves the search space each time (like binary search) → O(log n)
- Recursion that branches into 2 calls at every level, depth n → O(2^n)
- Recursion with memoization storing each unique state once → O(number of unique states × work per state)

2.6 Master Theorem (For Divide & Conquer Recursion)

For recurrences of the form `T(n) = a*T(n/b) + O(n^d)`:

- If `d > log_b(a)` → `T(n) = O(n^d)`
- If `d = log_b(a)` → `T(n) = O(n^d log n)`
- If `d < log_b(a)` → `T(n) = O(n^(log_b a))`

Example: Merge Sort → `T(n) = 2T(n/2) + O(n)` → a=2, b=2, d=1 → log_b(a) = 1 = d → `O(n log n)`. This matches what we know!

Example: Binary Search → `T(n) = T(n/2) + O(1)` → a=1, b=2, d=0 → log_b(a) = 0 = d → `O(log n)`.

---

3. Math & Bit Manipulation Foundations

3.1 Why This Section Exists

A surprising number of interview problems (especially at companies that love "clever" questions) reduce to basic number theory or bit tricks. Master these once and they become free points.

3.2 GCD, LCM, Primes

```java
// Euclidean algorithm - GCD - O(log(min(a,b)))
int gcd(int a, int b) {
    return b == 0 ? a : gcd(b, a % b);
}

int lcm(int a, int b) {
    return (a / gcd(a, b)) * b; // divide first to avoid overflow
}

// Sieve of Eratosthenes - find all primes up to n - O(n log log n)
boolean[] sieve(int n) {
    boolean[] isComposite = new boolean[n + 1];
    for (int i = 2; (long) i * i <= n; i++) {
        if (!isComposite[i]) {
            for (int j = i * i; j <= n; j += i) {
                isComposite[j] = true;
            }
        }
    }
    return isComposite; // isComposite[i] == false means i is prime (for i >= 2)
}
```

Why start `j` at `i*i`? Because every smaller multiple of `i` (like `2i, 3i, ...`) has already been marked composite by a smaller prime factor. Starting at `i*i` avoids redundant work — this is what gives the sieve its near-linear complexity.

3.3 Fast Exponentiation (Binary Exponentiation)

Naive power `a^b` takes O(b) multiplications. We can do it in O(log b):

```
long power(long base, long exp, long mod) {
    long result = 1;
    base %= mod;
    while (exp > 0) {
        if ((exp & 1) == 1) result = (result * base) % mod;
        base = (base * base) % mod;
        exp >>= 1;
    }
    return result;
}
```

Idea: `a^b = (a^2)^(b/2)` if b even, `a * a^(b-1)` if b odd. Each step halves the exponent → O(log b).

3.4 Bit Manipulation — The Toolkit

Operation	Expression	Meaning
Check if bit `i` is set	`(n >> i) & 1`	Returns 1 if set
Set bit `i`	`n | (1 << i)`	Turns bit on
Clear bit `i`	`n & ~(1 << i)`	Turns bit off
Toggle bit `i`	`n ^ (1 << i)`	Flips bit
Check power of 2	`n > 0 && (n & (n-1)) == 0`	Powers of 2 have exactly one set bit
Remove lowest set bit	`n & (n - 1)`	Used in Brian Kernighan's bit-count algorithm
Isolate lowest set bit	`n & (-n)`	Useful in Fenwick Trees
Count set bits	`Integer.bitCount(n)` (Java)	Or loop with `n & (n-1)`

Operation	Expression	Meaning
XOR self-cancel	`a ^ a = 0`, `a ^ 0 = a`	Foundation of "find the unique number" problems

Why does `n & (n-1)` remove the lowest set bit? Subtracting 1 flips all bits after (and including) the lowest set bit. ANDing with the original clears that lowest set bit while keeping higher bits unchanged. Example: `n = 0b1100`, `n-1 = 0b1011`, AND = `0b1000`.

Classic Problem: Single Number — Given an array where every element appears twice except one, find the unique one.

- **Brute force:** HashMap counting → O(n) time, O(n) space.
- **Optimized:** XOR all elements together. Since `a ^ a = 0` and `a ^ 0 = a`, all pairs cancel out, leaving only the unique number. → **O(n) time, O(1) space.**

```
int singleNumber(int[] nums) {
    int result = 0;
    for (int n : nums) result ^= n;
    return result;
}
```

3.5 Overflow Awareness

Always consider: does `a * b` overflow `int` (max ~2.1 billion)? Use `long` in Java/C++ when multiplying large values, especially in DP problems involving products, or when computing mid in binary search (`mid = low + (high - low) / 2` instead of `(low + high) / 2` to avoid overflow).

3.6 Modular Arithmetic (Used Heavily in DP / Combinatorics Problems)

- `(a + b) % m = ((a % m) + (b % m)) % m`
- `(a * b) % m = ((a % m) * (b % m)) % m`
- `(a - b) % m = ((a % m) - (b % m) + m) % m` (add `m` to avoid negative results)
- Division under modulo requires **modular inverse** (via Fermat's little theorem when `m` is prime): `a / b mod m = a * b^(m-2) mod m`.

4. Arrays & Strings — Basics to Advanced

4.1 What Is an Array?

An array is a contiguous block of memory holding elements of the same type, accessed by index in **O(1)** time (because the memory address of element `i` = `base_address + i * element_size` — pure arithmetic, no searching needed).

- **Access:** O(1)
- **Search (unsorted):** O(n)
- **Search (sorted):** O(log n) via binary search
- **Insert/Delete at end:** O(1) amortized
- **Insert/Delete at arbitrary index:** O(n) (must shift elements)

4.2 Strings

A string is essentially a character array (immutable in Java/Python, mutable in C via `char[]`). Because Java Strings are immutable, repeated concatenation in a loop (`s = s + "x"`) is **O(n²)** overall — use `StringBuilder` for O(n) amortized building.

```
StringBuilder sb = new StringBuilder();
for (char c : chars) sb.append(c); // O(1) amortized per append
String result = sb.toString();
```

4.3 Prefix Sum — The First "Free" Optimization

Problem: Given an array, answer many "sum of range [l, r]" queries.

- **Brute force:** For each query, loop from `l` to `r` → O(n) per query, O(n·q) total for q queries.
- **Optimized:** Precompute prefix sums once: `prefix[i] = arr[0] + arr[1] + ... + arr[i-1]`. Then `sum(l, r) = prefix[r+1] - prefix[l]`. → **O(n)**

preprocessing, O(1) per query.

```
int[] prefix = new int[arr.length + 1];
for (int i = 0; i < arr.length; i++) prefix[i + 1] = prefix[i] + arr[i];
// sum of arr[l..r] inclusive:
int rangeSum = prefix[r + 1] - prefix[l];
```

This is one of the most reused tricks in interviews — memorize it cold.

2D Prefix Sum (for matrix range-sum queries):

```
int[][] prefix = new int[m + 1][n + 1];
for (int i = 0; i < m; i++)
    for (int j = 0; j < n; j++)
        prefix[i+1][j+1] = matrix[i][j] + prefix[i][j+1] + prefix[i+1][j] -
prefix[i][j];
// sum of submatrix (r1,c1) to (r2,c2) inclusive:
int sum = prefix[r2+1][c2+1] - prefix[r1][c2+1] - prefix[r2+1][c1] + prefix[r1]
[c1];
```

4.4 Kadane's Algorithm — Maximum Subarray Sum

Problem: Find the contiguous subarray with the largest sum.

- **Brute force:** Check every subarray → O(n²) or O(n³) if sums recomputed each time.
- **Optimized (Kadane's):** Track `currentSum` — if it drops below 0, reset it to 0 (a negative running sum can only hurt future sums, so discard it). Track the max seen. → **O(n) time, O(1) space.**

```
int maxSubArray(int[] nums) {
    int maxSum = nums[0], currentSum = nums[0];
    for (int i = 1; i < nums.length; i++) {
        currentSum = Math.max(nums[i], currentSum + nums[i]);
        maxSum = Math.max(maxSum, currentSum);
    }
    return maxSum;
}
```

Why it works: At each index, we ask "is it better to extend the previous subarray, or start fresh here?" This is actually a mini dynamic-programming recurrence: `dp[i] =`

`max(nums[i], dp[i-1] + nums[i])`.

4.5 Two-Pointer Technique (Pattern #1)

Used when array is **sorted** (or can be sorted) and you need to find pairs/triplets satisfying a condition, or to compare from both ends.

Problem: Two Sum II (sorted array), find pair summing to target.

- **Brute force:** Nested loop → O(n²).
- **Optimized:** Pointer `left` at start, `right` at end. If `arr[left] + arr[right] > target`, decrease `right` (need smaller sum). If `< target`, increase `left`. If equal, found. → **O(n) time, O(1) space.**

```
int[] twoSumSorted(int[] arr, int target) {
    int left = 0, right = arr.length - 1;
    while (left < right) {
        int sum = arr[left] + arr[right];
        if (sum == target) return new int[]{left, right};
        else if (sum < target) left++;
        else right--;
    }
    return new int[]{-1, -1};
}
```

Problem: 3Sum — find all unique triplets summing to zero.

- **Brute force:** Three nested loops → O(n³).
- **Optimized:** Sort array O(n log n). Fix one element `i`, then two-pointer on the rest → O(n) per fixed element → **O(n²) total**, which is optimal for this problem (still needs to output potentially O(n²) triplets in worst analysis but achievable via O(n²) time).

```
List<List<Integer>> threeSum(int[] nums) {
    Arrays.sort(nums);
    List<List<Integer>> result = new ArrayList<>();
    for (int i = 0; i < nums.length - 2; i++) {
        if (i > 0 && nums[i] == nums[i - 1]) continue; // skip duplicates
        int left = i + 1, right = nums.length - 1;
        while (left < right) {
            int sum = nums[i] + nums[left] + nums[right];
            if (sum == 0) {
                result.add(Arrays.asList(nums[i], nums[left], nums[right]));
                while (left < right && nums[left] == nums[left + 1]) left++;
                while (left < right && nums[right] == nums[right - 1]) right--;
```

```
                left++; right--;
            } else if (sum < 0) left++;
            else right--;
        }
    }
    return result;
}
```

4.6 Sliding Window Technique (Pattern #2)

Used for **contiguous subarray/substring** problems, typically involving a "window" that grows and shrinks. Two flavors: **fixed size** and **variable size**.

Problem: Maximum sum of subarray of size k (fixed window).

- **Brute force:** For each starting index, sum k elements → O(n·k).
- **Optimized:** Compute sum of first window. Slide window by subtracting the element leaving and adding the element entering → **O(n) time, O(1) space.**

```
int maxSumSubarray(int[] arr, int k) {
    int windowSum = 0;
    for (int i = 0; i < k; i++) windowSum += arr[i];
    int maxSum = windowSum;
    for (int i = k; i < arr.length; i++) {
        windowSum += arr[i] - arr[i - k]; // slide: add new, remove old
        maxSum = Math.max(maxSum, windowSum);
    }
    return maxSum;
}
```

Problem: Longest Substring Without Repeating Characters (variable window).

- **Brute force:** Check every substring for uniqueness → O(n³) (or O(n²) with a set).
- **Optimized:** Expand `right` pointer, add char to a HashSet/HashMap. If a duplicate is found, shrink from `left` until duplicate is removed. Track max window size seen. → **O(n) time** (each pointer moves forward at most n times total), **O(min(n, charset)) space.**

```
int lengthOfLongestSubstring(String s) {
    Map<Character, Integer> lastSeen = new HashMap<>();
    int maxLen = 0, left = 0;
    for (int right = 0; right < s.length(); right++) {
        char c = s.charAt(right);
        if (lastSeen.containsKey(c) && lastSeen.get(c) >= left) {
            left = lastSeen.get(c) + 1; // jump left past the duplicate
```

```
            }
            lastSeen.put(c, right);
            maxLen = Math.max(maxLen, right - left + 1);
        }
        return maxLen;
    }
```

Problem: Minimum Window Substring — smallest substring of `s` containing all characters of `t`.

- **Brute force:** Check all substrings → O(n²) or worse.
- **Optimized:** Variable sliding window with a frequency map of `t`. Expand `right` until window is "valid" (contains all of `t`), then shrink `left` as much as possible while staying valid, recording the smallest valid window. → **O(n + m) time, O(m) space** (m = size of charset in t).

This is the hardest sliding window problem and a favorite at top companies — practice it until you can code it without bugs.

4.7 The Dutch National Flag Problem (3-way Partitioning)

Problem: Sort an array of 0s, 1s, 2s (Sort Colors).

- **Brute force:** Use any sort → O(n log n), or counting sort in two passes → O(n) but two passes.
- **Optimized (one pass):** Three pointers `low`, `mid`, `high`. Invariant: everything before `low` is 0, between `low` and `mid` is 1, after `high` is 2. → **O(n) time, O(1) space, single pass.**

```
void sortColors(int[] nums) {
    int low = 0, mid = 0, high = nums.length - 1;
    while (mid <= high) {
        if (nums[mid] == 0) { swap(nums, low++, mid++); }
        else if (nums[mid] == 1) { mid++; }
        else { swap(nums, mid, high--); } // don't increment mid: need to re-check
swapped element
    }
}
```

4.8 Cyclic Sort (Pattern #5)

Used when array contains numbers in range `[1, n]` (or `[0, n-1]`) and we need O(n) time, O(1) space to find missing/duplicate numbers.

Idea: Place each number at its "correct" index (`nums[i]` should be at index `nums[i] - 1`). One pass of swapping achieves this.

Problem: Find the Missing Number (array has n distinct numbers from 0 to n).

- **Brute force:** Sort and scan, or use a HashSet → O(n log n) or O(n) space.
- **Optimized (Cyclic Sort or Sum trick):** `expectedSum = n(n+1)/2`, `missing = expectedSum - actualSum` → **O(n) time, O(1) space.** (XOR trick also works and avoids overflow.)

```
int missingNumber(int[] nums) {
    int n = nums.length;
    int expectedSum = n * (n + 1) / 2;
    int actualSum = 0;
    for (int x : nums) actualSum += x;
    return expectedSum - actualSum;
}
```

Problem: Find All Duplicates in an Array (1 ≤ nums[i] ≤ n, each number appears once or twice).

- **Brute force:** HashMap counting → O(n) time, O(n) space.
- **Optimized (in-place marking):** For each number `x`, go to index `|x|-1` and negate the value there. If it's already negative, `|x|` is a duplicate. → **O(n) time, O(1) extra space.**

```
List<Integer> findDuplicates(int[] nums) {
    List<Integer> result = new ArrayList<>();
    for (int i = 0; i < nums.length; i++) {
        int idx = Math.abs(nums[i]) - 1;
        if (nums[idx] < 0) result.add(idx + 1);
        else nums[idx] = -nums[idx];
    }
    return result;
}
```

4.9 Merge Intervals (Pattern #4)

Used whenever you deal with intervals (start, end) and need to merge, insert, or find overlaps.

Problem: Merge Overlapping Intervals.

- **Brute force:** Compare every pair of intervals → O(n²).
- **Optimized:** Sort intervals by start time O(n log n). Then do a single linear scan, merging when `current.start <= previous.end`. → **O(n log n) time (dominated by sort), O(n) space.**

```
int[][] merge(int[][] intervals) {
    Arrays.sort(intervals, (a, b) -> a[0] - b[0]);
    List<int[]> merged = new ArrayList<>();
    for (int[] interval : intervals) {
        if (merged.isEmpty() || merged.get(merged.size() - 1)[1] < interval[0]) {
            merged.add(interval);
        } else {
            merged.get(merged.size() - 1)[1] = Math.max(merged.get(merged.size() -
1)[1], interval[1]);
        }
    }
    return merged.toArray(new int[0][]);
}
```

Why sorting first is the key insight: Once sorted by start time, you only ever need to compare each interval to the *last merged* interval — never to all previous intervals. This collapses an O(n²) comparison problem into O(n) after the O(n log n) sort.

4.10 Array Rotation

Problem: Rotate array by k positions.

- **Brute force:** Rotate one step at a time, k times → O(n·k).
- **Better:** Use extra array → O(n) time, O(n) space.
- **Optimal (in-place, O(1) space):** Reverse the whole array, then reverse the first k elements, then reverse the rest. → **O(n) time, O(1) space.**

```
void rotate(int[] nums, int k) {
    k %= nums.length;
    reverse(nums, 0, nums.length - 1);
    reverse(nums, 0, k - 1);
    reverse(nums, k, nums.length - 1);
}
void reverse(int[] a, int i, int j) {
```

```
    while (i < j) { int t = a[i]; a[i] = a[j]; a[j] = t; i++; j--; }
}
```

4.11 Matrix Problems

Problem: Rotate Image (matrix) 90 degrees in-place.

- **Optimized approach:** Transpose the matrix (swap `matrix[i][j]` with `matrix[j][i]`), then reverse each row. → **O(n²) time (must touch every cell), O(1) extra space.**

Problem: Spiral Matrix Traversal.

- **Approach:** Maintain four boundaries (top, bottom, left, right); traverse right along top, down along right, left along bottom, up along left, shrinking boundaries each time. → **O(m·n) time, O(1) extra space** (excluding output).

Problem: Search a 2D Matrix (rows and columns sorted).

- **Brute force:** Check every cell → O(m·n).
- **Optimized:** Start from top-right corner. If current > target, move left (eliminate column). If current < target, move down (eliminate row). → **O(m + n) time, O(1) space.**

4.12 String-Specific Patterns

Problem: Valid Anagram — check if two strings are anagrams.

- **Brute force:** Sort both strings and compare → O(n log n).
- **Optimized:** Frequency count array (size 26 for lowercase letters) → **O(n) time, O(1) space** (constant alphabet size).

Problem: Group Anagrams.

- **Approach:** Use sorted string (or character-frequency signature) as a HashMap key, group strings with the same key. → **O(n·k log k) time** (n strings, k = average length; or O(n·k) using frequency-count signature instead of sorting).

Problem: Longest Palindromic Substring.

- **Brute force:** Check every substring for palindrome property → O(n³) (or O(n²) with smarter checks).
- **Optimized (Expand Around Center):** For each index (and each index-pair for even-length), expand outward while characters match → **O(n²) time, O(1) space.**
- **Most Optimal (Manacher's Algorithm):** O(n) time — advanced, rarely required but good to mention you know it exists.

```
String longestPalindrome(String s) {
    int start = 0, maxLen = 0;
    for (int center = 0; center < s.length(); center++) {
        int len1 = expandAroundCenter(s, center, center);     // odd length
        int len2 = expandAroundCenter(s, center, center + 1); // even length
        int len = Math.max(len1, len2);
        if (len > maxLen) {
            maxLen = len;
            start = center - (len - 1) / 2;
        }
    }
    return s.substring(start, start + maxLen);
}
int expandAroundCenter(String s, int left, int right) {
    while (left >= 0 && right < s.length() && s.charAt(left) == s.charAt(right)) {
        left--; right++;
    }
    return right - left - 1;
}
```

Problem: String Matching (does needle occur in haystack?)

- **Brute force:** Try every starting position → O(n·m).
- **Optimized (KMP Algorithm):** Precompute a "failure function" (longest proper prefix that's also a suffix, for each prefix of the pattern) so that on mismatch, you never re-examine characters in the text → **O(n + m) time, O(m) space.** Rabin-Karp (rolling hash) is another O(n+m) average-case alternative worth knowing by name.

---

5. Hashing (HashMap / HashSet)

5.1 How Hashing Works Internally

A HashMap stores key-value pairs in an internal array of "buckets." A **hash function** converts a key into an integer (the hash code), which is then compressed to fit the

bucket array size (typically `hash % capacity`). Ideally, each key lands in its own bucket, giving **O(1) average time** for insert/lookup/delete.

Collisions occur when two keys hash to the same bucket. Two ways to resolve them:

- **Chaining:** Each bucket holds a linked list (or, in Java 8+, a balanced tree once a bucket gets too full) of entries. Lookup degrades to O(k) where k = entries in that bucket.
- **Open addressing (probing):** On collision, search for the next open slot (linear probing, quadratic probing, double hashing).

Why HashMaps are O(1) average, O(n) worst case: If all keys hash to the same bucket (adversarial input or poor hash function), every operation degrades to O(n). Java's `String.hashCode()` and `HashMap` use techniques (like additional bit spreading and treeifying buckets >8 entries into red-black trees) to mitigate this.

Load factor: `size / capacity`. When load factor exceeds a threshold (default 0.75 in Java), the table resizes (usually doubles) and rehashes everything → O(n) one-time cost, but amortized O(1) per insert overall (same idea as dynamic array resizing).

5.2 HashSet vs HashMap vs TreeMap vs LinkedHashMap

Structure	Ordering	Time (avg)	Use Case
`HashMap`	No order	O(1)	Fast lookup, no order needed
`LinkedHashMap`	Insertion order	O(1)	Need order preserved (e.g., LRU cache)
`TreeMap`	Sorted by key	O(log n)	Need sorted iteration / range queries
`HashSet`	No order	O(1)	Membership testing, dedup
`TreeSet`	Sorted	O(log n)	Sorted unique elements, floor/ceiling queries

5.3 Classic Hashing Problems

Problem: Two Sum (unsorted array, find indices summing to target).

- **Brute force:** Nested loop checking every pair → O(n²).
- **Optimized:** Single pass with a HashMap storing `value → index`. For each element, check if `target - element` already exists in the map. → **O(n) time, O(n) space.**

```
int[] twoSum(int[] nums, int target) {
    Map<Integer, Integer> seen = new HashMap<>();
    for (int i = 0; i < nums.length; i++) {
        int complement = target - nums[i];
        if (seen.containsKey(complement)) return new int[]{seen.get(complement),
i};
        seen.put(nums[i], i);
    }
    return new int[]{-1, -1};
}
```

This is *the* canonical "hashing beats brute force" problem — practically every interview candidate sees a variant of this.

Problem: Longest Consecutive Sequence (unsorted array, find length of longest run of consecutive integers).

- **Brute force:** Sort then scan → O(n log n).
- **Optimized (HashSet, O(n)):** Put all numbers in a HashSet. For each number, only start counting a sequence if `num - 1` is NOT in the set (i.e., it's the start of a sequence). Then count forward `num+1, num+2, ...` while present. → **O(n) time** because each number is visited at most twice total (once as a check, once as part of counting a sequence it starts).

```
int longestConsecutive(int[] nums) {
    Set<Integer> set = new HashSet<>();
    for (int n : nums) set.add(n);
    int longest = 0;
    for (int n : set) {
        if (!set.contains(n - 1)) { // n is the start of a sequence
            int length = 1;
            while (set.contains(n + length)) length++;
            longest = Math.max(longest, length);
        }
    }
    return longest;
}
```

Problem: Subarray Sum Equals K — count subarrays summing to k.

- **Brute force:** Check all subarrays with prefix sums → O(n²).
- **Optimized:** Running prefix sum + HashMap of `prefixSum → count of occurrences`. At each index, check if `(currentSum - k)` has occurred before — that means the subarray between those two points sums to k. → **O(n) time, O(n) space.**

```
int subarraySum(int[] nums, int k) {
    Map<Integer, Integer> prefixCount = new HashMap<>();
    prefixCount.put(0, 1); // empty prefix
    int sum = 0, count = 0;
    for (int num : nums) {
        sum += num;
        count += prefixCount.getOrDefault(sum - k, 0);
        prefixCount.merge(sum, 1, Integer::sum);
    }
    return count;
}
```

Problem: Group Anagrams / Valid Anagram — see Section 4.12 (hashing-based signature grouping).

Problem: First Non-Repeating Character.

- **Approach:** Frequency map in one pass O(n), then a second pass to find the first char with count 1 → **O(n) time, O(1) space** (bounded alphabet).

---

6. Linked Lists

6.1 What Is a Linked List and Why Use One?

A linked list is a sequence of nodes where each node stores data plus a pointer/reference to the next node. Unlike arrays:

- **No contiguous memory required** → insert/delete at a known node is O(1) (vs O(n) shifting in arrays).
- **No random access** → accessing the k-th element is O(n) (must traverse from head).
- Extra memory overhead per node (for the pointer).

Types:

- **Singly Linked List:** each node points to the next only.
- **Doubly Linked List:** each node points to next AND previous — allows O(1) removal given a node reference, and backward traversal.
- **Circular Linked List:** tail points back to head — used in round-robin scheduling, Josephus problem.

```
class ListNode {
    int val;
    ListNode next;
    ListNode(int val) { this.val = val; }
}
```

6.2 In-Place Reversal (Pattern #6)

Problem: Reverse a Linked List.

- **Brute force:** Push all values to a stack/array, then rebuild the list (or overwrite values) → O(n) time, O(n) space.
- **Optimized (in-place, iterative):** Maintain `prev`, `curr`, `next` pointers; flip the `next` pointer of each node as you traverse. → **O(n) time, O(1) space.**

```
ListNode reverseList(ListNode head) {
    ListNode prev = null, curr = head;
    while (curr != null) {
        ListNode next = curr.next;
        curr.next = prev;
        prev = curr;
        curr = next;
    }
    return prev; // new head
}
```

Problem: Reverse Linked List II (reverse only nodes between position m and n).

- **Approach:** Same three-pointer technique, but first walk to node `m-1`, then reverse the sublist of length `n-m+1`, then reconnect. → **O(n) time, O(1) space.**

Problem: Reverse Nodes in k-Group.

- **Approach:** Recursively (or iteratively) reverse each group of k nodes using the same pointer-flipping technique, connecting each reversed group to the next. → **O(n) time, O(1) extra space** (excluding recursion stack if done recursively).

6.3 Fast & Slow Pointers / Floyd's Cycle Detection (Pattern #3)

Problem: Detect a Cycle in a Linked List.

- **Brute force:** Store visited nodes in a HashSet, check if a node repeats → O(n) time, O(n) space.
- **Optimized (Floyd's Tortoise and Hare):** Two pointers, `slow` moves 1 step, `fast` moves 2 steps. If there's a cycle, they *will* meet inside it (like two runners on a circular track — the faster one laps the slower one). If `fast` reaches `null`, there's no cycle. → **O(n) time, O(1) space.**

```
boolean hasCycle(ListNode head) {
    ListNode slow = head, fast = head;
    while (fast != null && fast.next != null) {
        slow = slow.next;
        fast = fast.next.next;
        if (slow == fast) return true;
    }
    return false;
}
```

Problem: Find the Start of the Cycle.

- **Approach:** After slow/fast meet, reset one pointer to `head`, keep the other at the meeting point. Move both one step at a time — they meet exactly at the cycle's start. **Why this works mathematically:** if the distance from head to cycle start is `a`, and the meeting point is `b` steps into the cycle, it can be proven `a = (cycle length - b) + k*(cycle length)`, so moving `a` steps from both head and meeting point lands on the same node. → **O(n) time, O(1) space.**

Problem: Find Middle of Linked List.

- **Approach:** slow/fast pointers — when fast reaches the end, slow is at the middle. → **O(n) time, O(1) space.** (Building block for merge sort on linked lists, and for palindrome-checking on linked lists.)

Problem: Linked List Palindrome Check.

- **Approach:** Find middle (fast/slow), reverse second half in-place, compare first half with reversed second half. → **O(n) time, O(1) space** (better than the O(n) space brute force of copying to an array).

6.4 Merging & Sorting Linked Lists

Problem: Merge Two Sorted Lists.

- **Approach:** Classic merge-step from merge sort — compare heads, attach the smaller, advance that pointer. → **O(n + m) time, O(1) extra space** (reusing existing nodes).

Problem: Merge k Sorted Lists.

- **Brute force:** Merge lists one at a time → $O(k^2 \cdot n)$ where n = average length.
- **Optimized (Divide & Conquer or Min-Heap):**
  - *Heap approach:* put the head of each list into a min-heap; repeatedly pop the smallest, push its `next`. → **O(n log k) time, O(k) space.**
  - *Divide & conquer:* pair up lists and merge, like merge sort's merge step, halving the number of lists each round. → **O(n log k) time.**

6.5 Other Common Linked List Problems

Problem: Remove N-th Node From End.

- **Brute force:** Compute length first, then find the node → two passes, O(n) time.
- **Optimized (one-pass, two pointers):** Move `fast` pointer n steps ahead first, then move both `fast` and `slow` together until `fast` hits the end — `slow` is now right before the node to remove. → **O(n) time, one pass, O(1) space.**

Problem: Add Two Numbers (represented as linked lists, digits in reverse order).

- **Approach:** Simulate elementary-school addition digit by digit with a carry, building a new list. → **O(max(n, m)) time, O(max(n, m)) space** for the result.

Problem: Copy List with Random Pointer.

- **Brute force:** HashMap mapping original node → cloned node, two passes → O(n) time, O(n) space.

- **Optimized (O(1) extra space trick):** Interweave cloned nodes right after each original node (`A → A' → B → B'...`), set random pointers using `original.random.next`, then de-interleave into two separate lists. → **O(n) time, O(1) extra space** (excluding output).

---

7. Stacks & Queues

7.1 Stack Basics (LIFO)

A stack supports `push`, `pop`, `peek` — all **O(1)**. Used for: reversing order, matching pairs (parentheses), backtracking (undo), DFS (explicit stack instead of recursion), expression evaluation.

Problem: Valid Parentheses.

- **Approach:** Push opening brackets. On a closing bracket, check if it matches the top of the stack (pop if so); mismatch or empty stack means invalid. → **O(n) time, O(n) space.**

```
boolean isValid(String s) {
    Deque<Character> stack = new ArrayDeque<>();
    Map<Character, Character> pairs = Map.of(')', '(', ']', '[', '}', '{');
    for (char c : s.toCharArray()) {
        if (!pairs.containsKey(c)) stack.push(c);
        else if (stack.isEmpty() || stack.pop() != pairs.get(c)) return false;
    }
    return stack.isEmpty();
}
```

Problem: Min Stack — design a stack supporting push/pop/top/`getMin` all in O(1).

- **Approach:** Maintain a second stack that tracks the minimum "so far" at each level (push `min(newVal, currentMin)` onto the min-stack every time). → **O(1) for every operation, O(n) space.**

Problem: Evaluate Reverse Polish Notation.

- **Approach:** Push operands; on an operator, pop two operands, apply operator, push result back. → **O(n) time, O(n) space.**

7.2 Queue Basics (FIFO)

A queue supports `enqueue` (add to back) and `dequeue` (remove from front), both **O(1)** when backed by a proper structure (circular buffer or linked list — NOT a plain array, where dequeue from front is O(n) due to shifting).

Used for: BFS, task scheduling, rate limiting, producer-consumer buffering.

Circular Queue implementation trick: use a fixed array with `front` and `rear` indices computed modulo capacity to reuse freed slots without shifting.

7.3 Monotonic Stack / Queue (Pattern #16)

A **monotonic stack** keeps elements in strictly increasing or decreasing order, popping elements that violate this order as you insert new ones. It's the go-to pattern for "next greater/smaller element" style problems.

Problem: Next Greater Element (for each element, find the next element to its right that's greater).

- **Brute force:** For each element, scan rightwards until a greater one is found → O(n²).
- **Optimized (Monotonic Decreasing Stack):** Traverse the array; maintain a stack of indices whose values are in decreasing order. For each new element, pop all stack elements smaller than it (they've found their "next greater" — the current element), then push current index. → **O(n) time** because each element is pushed and popped at most once, **O(n) space.**

```
int[] nextGreaterElement(int[] nums) {
    int[] result = new int[nums.length];
    Arrays.fill(result, -1);
    Deque<Integer> stack = new ArrayDeque<>(); // stores indices
    for (int i = 0; i < nums.length; i++) {
        while (!stack.isEmpty() && nums[stack.peek()] < nums[i]) {
            result[stack.pop()] = nums[i];
        }
        stack.push(i);
    }
    return result;
}
```

Problem: Daily Temperatures (how many days until a warmer temperature) — identical monotonic stack pattern, storing "days to wait" instead of the value itself.

Problem: Largest Rectangle in Histogram.

- **Brute force:** For each bar, expand left/right to find the widest rectangle at that height → O(n²).
- **Optimized (Monotonic Increasing Stack):** Maintain a stack of indices with increasing bar heights. When a shorter bar is encountered, pop taller bars and compute the rectangle area they could form (height = popped bar, width = distance between the new stack top and current index). → **O(n) time, O(n) space.** This is considered a "hard" problem but is 100% mechanical once you know the pattern.

Problem: Sliding Window Maximum (max of every window of size k).

- **Brute force:** Recompute max for every window → O(n·k).
- **Optimized (Monotonic Decreasing Deque):** Maintain a deque of indices where values are decreasing. Remove indices that fall out of the window from the front; remove smaller values from the back before pushing new index (they'll never be the max while the current larger element is in the window). Front of deque is always the max of the current window. → **O(n) time** (each index pushed/popped once), **O(k) space.**

```
int[] maxSlidingWindow(int[] nums, int k) {
    Deque<Integer> deque = new ArrayDeque<>(); // stores indices, values decreasing
    int[] result = new int[nums.length - k + 1];
    for (int i = 0; i < nums.length; i++) {
        if (!deque.isEmpty() && deque.peekFirst() <= i - k) deque.pollFirst(); //
out of window
        while (!deque.isEmpty() && nums[deque.peekLast()] < nums[i])
deque.pollLast(); // maintain decreasing
        deque.offerLast(i);
        if (i >= k - 1) result[i - k + 1] = nums[deque.peekFirst()];
    }
    return result;
}
```

7.4 Queue via Two Stacks / Stack via Two Queues (Classic Design Question)

Implement Queue using Stacks: use an `inStack` for enqueue (O(1)) and an `outStack` for dequeue. When `outStack` is empty, pour all of `inStack` into it (reversing order) → dequeue becomes O(1) amortized (each element moved between stacks at most once).

---

8. Recursion & Backtracking

8.1 Recursion Fundamentals

Recursion = a function that calls itself on a smaller subproblem, with a **base case** to stop. Every recursive solution needs:

1. **Base case(s)** — the simplest input(s) you can answer directly.
2. **Recursive case** — how to break the problem into smaller version(s) of itself.
3. **Trust the recursion** ("leap of faith") — assume the recursive call correctly solves the smaller subproblem; focus only on how to combine that result.

The Call Stack: Each recursive call pushes a new stack frame (storing local variables and the return address). Depth of recursion = space complexity (O(depth)). Very deep recursion (e.g., n = 10^6) risks **StackOverflowError** — sometimes you must convert recursion to iteration (using an explicit stack) for such cases.

8.2 Classic Recursion Problems

Problem: Factorial / Fibonacci — the "hello world" of recursion.

- Naive Fibonacci: `fib(n) = fib(n-1) + fib(n-2)` → **O(2^n) time** (exponential, massive redundant recomputation — this is the classic motivator for memoization/DP, see Section 14).
- With memoization: **O(n) time, O(n) space.**

Problem: Print all permutations of a string/array.

- **Approach:** At each recursive level, pick one unused element to place next, recurse on the rest, then backtrack (undo the choice) and try the next option. → **O(n! × n) time** (n! permutations, O(n) to build/copy each one).

Problem: Power Set / Subsets (Pattern #10).

- **Approach:** At each element, recursively branch into "include it" and "exclude it." → **O(2^n) time, O(2^n) space** for all subsets (this is optimal since there are exactly 2^n subsets to output).

```
List<List<Integer>> subsets(int[] nums) {
    List<List<Integer>> result = new ArrayList<>();
    backtrack(nums, 0, new ArrayList<>(), result);
    return result;
}
void backtrack(int[] nums, int start, List<Integer> current, List<List<Integer>>
result) {
    result.add(new ArrayList<>(current)); // every state is a valid subset
    for (int i = start; i < nums.length; i++) {
        current.add(nums[i]);
        backtrack(nums, i + 1, current, result);   // recurse
        current.remove(current.size() - 1);         // backtrack (undo)
    }
}
```

8.3 The Backtracking Template

Backtracking = DFS through a decision tree, where you make a choice, recurse, and **undo the choice** ("backtrack") if it doesn't pan out or after exploring it fully. General template:

```
function backtrack(state):
    if state is a complete/invalid solution:
        record/reject and return
    for each choice available from state:
        make the choice
        backtrack(new state)
        undo the choice   # <-- the critical "backtracking" step
```

Problem: N-Queens — place n queens on an n×n board so none attack each other.

- **Brute force:** Try all C(n², n) placements and check validity → astronomically slow.
- **Optimized (Backtracking with pruning):** Place queens row by row; for each row, only try columns not under attack (track attacked columns and both diagonals with boolean sets). Prune invalid branches immediately instead of building the full board first. → Still exponential worst case but massively pruned in practice; the elegant part is that **checking a placement is O(1)** using precomputed attack-sets, so total work is proportional to the actual search tree size.

Problem: Sudoku Solver.

- **Approach:** Find an empty cell, try digits 1-9, check row/col/box validity in O(1) with precomputed sets, recurse, backtrack on failure. → Exponential worst case, but constraint propagation keeps it fast in practice.

Problem: Combination Sum (find all combinations summing to target, elements reusable).

- **Approach:** Backtrack choosing to include each candidate 0+ times; recurse with reduced target; prune when remaining target < 0; sort first to prune early once candidate > remaining target. → Exponential in the worst case but heavily pruned.

Problem: Word Search (find if a word exists in a 2D grid via adjacent cells).

- **Approach:** DFS/backtracking from every cell matching the first letter; mark visited cells temporarily (or use a visited set), unmark on backtrack. → **O(m·n·4^L)** worst case (L = word length), but pruned quickly on mismatches.

Problem: Generate Parentheses (all valid combinations of n pairs).

- **Approach:** Backtrack tracking `open` and `close` counts used so far; only add `(` if `open < n`, only add `)` if `close < open`. → **O(4^n / √n) time** (the nth Catalan number — this exact bound is good to *mention* you know exists, without needing to derive it live).

8.4 Divide and Conquer

Divide and Conquer breaks a problem into independent subproblems, solves them recursively, and **combines** results — distinct from backtracking (which explores/prunes a decision tree) and from DP (which has *overlapping* subproblems).

Problem: Merge Sort, Quick Sort — see Section 9. **Problem: Maximum Subarray via Divide & Conquer** — split at midpoint, recursively solve left half, right half, and the case where the answer crosses the midpoint (this crossing case is computed in O(n)). → `T(n) = 2T(n/2) + O(n) = O(n log n)` (Kadane's O(n) approach in Section 4.4 is strictly better, but D&C is a good exercise and sometimes explicitly asked for).

Problem: Search in Rotated Sorted Array — see Section 10 (modified binary search, a D&C flavor).

---

9. Sorting Algorithms

9.1 Why Learn Multiple Sorting Algorithms?

Interviewers rarely ask you to "implement bubble sort," but they frequently ask you to **explain trade-offs** (stability, in-place, time complexity in best/worst/average case) and to use sorting as a **subroutine** inside a harder problem (e.g., sort by start time before merging intervals, or use quickselect for "kth largest").

9.2 Comparison-Based Sorts

Algorithm	Best	Average	Worst	Space	Stable?	Notes
Bubble Sort	O(n)	O(n²)	O(n²)	O(1)	Yes	Only good for teaching; swaps adjacent out-of-order pairs repeatedly
Selection Sort	O(n²)	O(n²)	O(n²)	O(1)	No	Repeatedly selects the min and swaps into place; minimal swaps
Insertion Sort	O(n)	O(n²)	O(n²)	O(1)	Yes	Great for nearly-sorted data / small arrays; builds sorted prefix
Merge Sort	O(n log n)	O(n log n)	O(n log n)	O(n)	Yes	Guaranteed O(n log n), needs extra space; great for linked lists too
Quick Sort	O(n log n)	O(n log n)	O(n²)	O(log n)	No	Fastest in practice; worst case with bad pivot choice (mitigated by random/median-of-3 pivot)

Algorithm	Best	Average	Worst	Space	Stable?	Notes
Heap Sort	O(n log n)	O(n log n)	O(n log n)	O(1)	No	In-place, guaranteed O(n log n), but poor cache locality (slower in practice than quicksort)

9.3 Merge Sort — Deep Dive

Idea: Divide the array into halves recursively until size 1 (trivially sorted), then merge sorted halves back together.

```
void mergeSort(int[] arr, int left, int right) {
    if (left >= right) return;
    int mid = left + (right - left) / 2;
    mergeSort(arr, left, mid);
    mergeSort(arr, mid + 1, right);
    merge(arr, left, mid, right);
}
void merge(int[] arr, int left, int mid, int right) {
    int[] temp = new int[right - left + 1];
    int i = left, j = mid + 1, k = 0;
    while (i <= mid && j <= right) temp[k++] = (arr[i] <= arr[j]) ? arr[i++] :
arr[j++];
    while (i <= mid) temp[k++] = arr[i++];
    while (j <= right) temp[k++] = arr[j++];
    System.arraycopy(temp, 0, arr, left, temp.length);
}
```

Why O(n log n)? log n levels of recursion (halving each time), and O(n) work to merge at each level → n × log n.

Why it's stable: the merge step always takes from the left half first when equal, preserving original relative order.

9.4 Quick Sort — Deep Dive

Idea: Pick a pivot, partition the array so elements `< pivot` are left, `> pivot` are right, then recursively sort each partition. Unlike merge sort, partitioning is done **in-place** and sorting happens **before** the recursive calls return (no explicit merge step needed).

```
void quickSort(int[] arr, int low, int high) {
    if (low < high) {
        int pivotIndex = partition(arr, low, high);
        quickSort(arr, low, pivotIndex - 1);
        quickSort(arr, pivotIndex + 1, high);
    }
}
int partition(int[] arr, int low, int high) {
    int pivot = arr[high]; // Lomuto partition scheme
    int i = low - 1;
    for (int j = low; j < high; j++) {
        if (arr[j] < pivot) { i++; swap(arr, i, j); }
    }
    swap(arr, i + 1, high);
    return i + 1;
}
```

Why worst case O(n²)? If the pivot is always the smallest/largest element (e.g., already-sorted array with naive last-element pivot), partitions are maximally unbalanced (size 0 and n-1), giving `T(n) = T(n-1) + O(n) = O(n²)`. **Mitigation:** randomized pivot or median-of-three pivot selection makes worst case astronomically unlikely.

9.5 Heap Sort

Idea: Build a max-heap from the array (O(n)), then repeatedly swap the root (max) with the last element and "sift down" to restore heap property, shrinking the heap by one each time. → O(n log n), in-place, but not stable.

9.6 Non-Comparison Sorts (Linear Time!)

These beat the O(n log n) comparison-sort lower bound by exploiting known structure in the data (not comparing elements pairwise).

Counting Sort: works when values are integers in a small known range `[0, k]`. Count occurrences of each value, then reconstruct the sorted array. → **O(n + k) time, O(k) space.** Used as a subroutine in Radix Sort, and directly useful for problems like "sort an array of 0s/1s/2s" or "sort by frequency."

Radix Sort: sorts numbers digit by digit (least significant to most significant), using counting sort as a stable subroutine for each digit. → **O(d·(n + k)) time** where d = number of digits, k = base (usually 10). Effectively O(n) for fixed-width integers.

Bucket Sort: distribute elements into buckets based on value range, sort each bucket individually (often with insertion sort), then concatenate. → **O(n + k) average time** if data is uniformly distributed.

9.7 Quickselect — Kth Largest/Smallest Element

Problem: Find the Kth Largest Element in an array.

- **Brute force:** Sort the array → O(n log n).
- **Better:** Min-heap of size k → O(n log k) (see Section 12).
- **Most Optimal (Quickselect):** Like quicksort's partition step, but recurse into only **one** side (the side containing the kth element) instead of both. → **O(n) average time**, O(n²) worst case (mitigated with random pivot), O(1) extra space.

```
int findKthLargest(int[] nums, int k) {
    int targetIndex = nums.length - k; // kth largest = (n-k)th smallest (0-
indexed)
    int low = 0, high = nums.length - 1;
    while (true) {
        int pivotIndex = partition(nums, low, high);
        if (pivotIndex == targetIndex) return nums[pivotIndex];
        else if (pivotIndex < targetIndex) low = pivotIndex + 1;
        else high = pivotIndex - 1;
    }
}
```

Why average O(n)? Each partition step is O(current range), and on average the range shrinks by half each time (geometric series: n + n/2 + n/4 + ... = 2n = O(n)), unlike quicksort which must recurse into *both* halves.

9.8 When to Use Which Sort (Interview Answer Template)

- "I'd use **merge sort** when stability matters or when sorting a linked list (no random access penalty)."
- "I'd use **quick sort** for general in-memory array sorting — best average performance and in-place."
- "I'd use **counting/radix sort** when the value range is small/known, e.g., sorting ages or scores — beats O(n log n)."

- "I'd use **heap sort** when I need guaranteed O(n log n) with O(1) space and don't care about stability."
- "I'd use **quickselect** instead of full sort when I only need the kth element, not the full sorted order — saves time from O(n log n) to O(n)."

---

10. Searching & Binary Search Patterns

10.1 Linear Search vs Binary Search

Linear search: O(n), works on unsorted data. Binary search: O(log n), **requires sorted (or "monotonic"/"sorted-like") data** — or more precisely, requires the ability to define a boolean predicate that is `false` for a prefix and `true` for a suffix (or vice versa) over the search space.

10.2 The Binary Search Template (Memorize This)

```
int binarySearch(int[] arr, int target) {
    int low = 0, high = arr.length - 1;
    while (low <= high) {
        int mid = low + (high - low) / 2; // avoids overflow
        if (arr[mid] == target) return mid;
        else if (arr[mid] < target) low = mid + 1;
        else high = mid - 1;
    }
    return -1; // not found
}
```

Key insight for Modified Binary Search (Pattern #11): Binary search isn't just for "find exact value in sorted array." It works **any time you can define a monotonic true/false condition** over a range and want to find the boundary. Ask yourself: *"Can I eliminate half the search space based on a check at the midpoint?"* If yes → binary search applies, even if the array isn't sorted in the traditional sense (e.g., "search in rotated sorted array," "find peak element," "search in answer space" for optimization problems).

10.3 Classic Binary Search Variants

Problem: Find First and Last Position of Element in Sorted Array.

- **Approach:** Two binary searches — one biased to find the leftmost occurrence (when `arr[mid] == target`, keep searching left by setting `high = mid - 1`), one for the rightmost (`low = mid + 1`). → **O(log n) time** (two binary searches), much better than O(n) linear scan.

Problem: Search in Rotated Sorted Array.

- **Brute force:** Linear scan → O(n).
- **Optimized:** At each step, determine which half (`low..mid` or `mid..high`) is properly sorted (comparing `arr[low]` vs `arr[mid]`). If target lies within the sorted half's range, search there; otherwise search the other half. → **O(log n) time, O(1) space.**

Problem: Find Minimum in Rotated Sorted Array.

- **Approach:** Compare `arr[mid]` with `arr[high]`. If `arr[mid] > arr[high]`, the minimum is in the right half (`low = mid + 1`); else it's in the left half including mid (`high = mid`). → **O(log n) time.**

Problem: Find Peak Element (element greater than both neighbors).

- **Approach:** If `arr[mid] < arr[mid+1]`, a peak must exist to the right (search right half); else search left half (including mid). → **O(log n) time** — this works because you're never discarding the side that's guaranteed to contain a peak.

Problem: Search a 2D Matrix (fully sorted, row-major order).

- **Approach:** Treat the 2D matrix as a flattened 1D sorted array; binary search using `mid / cols` and `mid % cols` to map back to 2D coordinates. → **O(log(m·n)) time.**

10.4 Binary Search on the Answer (A Very High-Leverage Trick)

Many optimization problems ("minimize the maximum X" / "maximize the minimum Y") can be solved by **binary searching over the range of possible answers**, using a feasibility check function.

Problem: Koko Eating Bananas (find minimum eating speed k so Koko finishes all bananas within h hours).

- **Brute force:** Try every speed from 1 to max(piles) → O(max(piles) · n).
- **Optimized:** Binary search on the speed `k` (range: 1 to max(piles)). For each candidate `k`, compute hours needed in O(n); if feasible (`hours <= h`), try smaller `k` (search left); else search right. → **O(n log(max(piles))) time.**

```
int minEatingSpeed(int[] piles, int h) {
    int low = 1, high = Arrays.stream(piles).max().getAsInt();
    while (low < high) {
        int mid = low + (high - low) / 2;
        if (hoursNeeded(piles, mid) <= h) high = mid; // feasible, try smaller
        else low = mid + 1;
    }
    return low;
}
long hoursNeeded(int[] piles, int speed) {
    long hours = 0;
    for (int p : piles) hours += (p + speed - 1) / speed; // ceiling division
    return hours;
}
```

Problem: Capacity to Ship Packages Within D Days — identical binary-search-on-answer pattern (binary search on the ship capacity).

Problem: Split Array Largest Sum — same pattern (binary search on the "largest subarray sum" value).

How to recognize this pattern in an interview: the phrase "minimize the maximum" or "find the minimum X such that condition Y holds" is a huge signal. Ask: *"Is the feasibility of a candidate answer monotonic?"* (i.e., if `X` works, does every value `> X` also work, or every value `< X`?) If yes, binary search on the answer applies.

10.5 Ternary Search (Less Common, Good to Know)

Used for finding the extremum (min/max) of a **unimodal function** (one that increases then decreases, or vice versa). Divide the search range into three parts using two midpoints, discard the third that can't contain the extremum. → O(log n), though in practice binary search on the derivative's sign is often preferred when possible.

---

11. Trees

11.1 What Is a Tree? Basic Terminology

A tree is a hierarchical structure of nodes where each node has a value and pointers to child nodes, with no cycles, and exactly one path between any two nodes.

- **Root:** the topmost node.
- **Leaf:** a node with no children.
- **Height of a node:** longest path from that node down to a leaf.
- **Depth of a node:** distance from the root to that node.
- **Balanced tree:** height is O(log n) — guarantees fast operations.
- **Binary Tree:** each node has at most 2 children.
- **Binary Search Tree (BST):** left subtree values < node < right subtree values, for every node.
- **Complete Binary Tree:** all levels full except possibly the last, filled left to right (basis of array-backed heaps).
- **Full Binary Tree:** every node has 0 or 2 children.

```
class TreeNode {
    int val;
    TreeNode left, right;
    TreeNode(int val) { this.val = val; }
}
```

11.2 Tree Traversals

Depth-First Traversals (Pattern #8 — Tree DFS):

- **Preorder** (Root → Left → Right): used to *copy* a tree or produce a prefix expression.
- **Inorder** (Left → Root → Right): for a BST, this produces values in **sorted order** — extremely important property.
- **Postorder** (Left → Right → Root): used when children must be processed before the parent (e.g., deleting a tree, computing subtree sizes/heights).

```
void preorder(TreeNode node, List<Integer> result) {
    if (node == null) return;
```

```
    result.add(node.val);
    preorder(node.left, result);
    preorder(node.right, result);
}
// Iterative preorder using an explicit stack (avoids recursion stack overflow
risk):
List<Integer> preorderIterative(TreeNode root) {
    List<Integer> result = new ArrayList<>();
    if (root == null) return result;
    Deque<TreeNode> stack = new ArrayDeque<>();
    stack.push(root);
    while (!stack.isEmpty()) {
        TreeNode node = stack.pop();
        result.add(node.val);
        if (node.right != null) stack.push(node.right); // push right first
        if (node.left != null) stack.push(node.left);   // so left is processed
first
    }
    return result;
}
```

Breadth-First Traversal / Level Order (Pattern #7 — Tree BFS): visit nodes level by level using a **queue**.

```
List<List<Integer>> levelOrder(TreeNode root) {
    List<List<Integer>> result = new ArrayList<>();
    if (root == null) return result;
    Queue<TreeNode> queue = new LinkedList<>();
    queue.offer(root);
    while (!queue.isEmpty()) {
        int size = queue.size(); // snapshot: number of nodes at this level
        List<Integer> level = new ArrayList<>();
        for (int i = 0; i < size; i++) {
            TreeNode node = queue.poll();
            level.add(node.val);
            if (node.left != null) queue.offer(node.left);
            if (node.right != null) queue.offer(node.right);
        }
        result.add(level);
    }
    return result;
}
```

All traversals are O(n) time (visit every node once), O(n) space (worst case for a skewed tree's recursion stack, or O(w) for BFS where w = max width of tree).

11.3 Core Tree Problems (DFS-based)

Problem: Maximum Depth of Binary Tree.

- **Approach:** `depth(node) = 1 + max(depth(left), depth(right))`, base case `depth(null) = 0`. → **O(n) time, O(h) space** (h = height, for recursion stack).

Problem: Diameter of Binary Tree (longest path between any two nodes, may not pass through root).

- **Brute force:** For every node, compute height of left + height of right, take max over all nodes → O(n²) if height recomputed each time.
- **Optimized:** Single DFS pass that computes height AND updates a global/passed-by-reference "max diameter" simultaneously (`diameter at node = leftHeight + rightHeight`). → **O(n) time, O(h) space.**

Problem: Balanced Binary Tree Check.

- **Approach:** Bottom-up DFS returning -1 as a sentinel the moment imbalance is detected anywhere, short-circuiting further computation. → **O(n) time** (vs O(n²) if you naively recompute height at every node).

Problem: Lowest Common Ancestor (LCA) of a Binary Tree.

- **Approach:** Recursively search left and right subtrees for the two target nodes. If both are found in different subtrees, the current node is the LCA; if only one side finds something, propagate that result up. → **O(n) time, O(h) space.**
- **For a BST specifically:** use the BST property — if both targets are less than current node, go left; if both greater, go right; otherwise current node is the LCA. → **O(h) time**, better than general tree LCA.

Problem: Validate Binary Search Tree.

- **Common bug:** checking only `node.left.val < node.val < node.right.val` locally is WRONG — it misses violations from grandchildren.
- **Correct approach:** pass down a valid `(min, max)` range to each recursive call, narrowing it as you descend. → **O(n) time, O(h) space.** Alternative: do an inorder traversal and verify it's strictly increasing.

Problem: Serialize and Deserialize Binary Tree.

- **Approach:** Preorder traversal with explicit "null" markers for missing children, joined into a string; deserialize by reading tokens in the same preorder sequence, recursively reconstructing. → **O(n) time, O(n) space.**

Problem: Path Sum / Path Sum II / Binary Tree Maximum Path Sum.

- **Approach (Max Path Sum, the hardest variant):** DFS returning "best downward path sum starting at this node" to the parent (only extending if positive), while separately tracking the best "path through this node as the peak" (left + right + node value) in a global/outer variable. → **O(n) time, O(h) space.**

11.4 Binary Search Trees (BST) — Operations

Operation	Average (balanced)	Worst case (skewed)
Search	O(log n)	O(n)
Insert	O(log n)	O(n)
Delete	O(log n)	O(n)

Insert: recurse left/right based on comparison until you find a null spot.

Delete (3 cases):

1. Node is a leaf → simply remove it.
2. Node has one child → replace node with its child.
3. Node has two children → find the **inorder successor** (smallest value in right subtree, i.e., leftmost node of right subtree), copy its value into the current node, then recursively delete that successor from the right subtree (it has at most one child, so this reduces to case 1 or 2).

Why BSTs can degrade to O(n): if you insert already-sorted data, the tree becomes a straight line (like a linked list) — no balancing. This motivates **self-balancing trees**.

11.5 Self-Balancing Trees (Concept-Level Understanding)

You are rarely asked to *implement* these from scratch, but you MUST be able to explain the concept:

- **AVL Tree:** after every insert/delete, checks the "balance factor" (height difference between left and right subtree) at each ancestor; if it exceeds 1, performs **rotations** (single or double: LL, RR, LR, RL) to restore balance. Guarantees O(log n) height strictly.

- **Red-Black Tree:** a looser balancing scheme (each node colored red/black with specific invariants) that guarantees O(log n) height with fewer rotations than AVL on average — this is what Java's `TreeMap`/`TreeSet` and C++'s `std::map` use internally.
- **B-Trees / B+ Trees:** generalize BSTs to have many children per node (not just 2) — used in databases and filesystems because they minimize disk reads (each node = one disk block, so a B-tree of order 100 has height ~$\log_{100}(n)$, drastically fewer disk seeks than a binary tree).

11.6 Tries (Prefix Trees)

A Trie stores strings by sharing common prefixes as paths in a tree, where each edge represents one character.

```
class TrieNode {
    TrieNode[] children = new TrieNode[26];
    boolean isEndOfWord;
}
class Trie {
    TrieNode root = new TrieNode();
    void insert(String word) {
        TrieNode node = root;
        for (char c : word.toCharArray()) {
            int idx = c - 'a';
            if (node.children[idx] == null) node.children[idx] = new TrieNode();
            node = node.children[idx];
        }
        node.isEndOfWord = true;
    }
    boolean search(String word) {
        TrieNode node = find(word);
        return node != null && node.isEndOfWord;
    }
    boolean startsWith(String prefix) {
        return find(prefix) != null;
    }
    private TrieNode find(String s) {
        TrieNode node = root;
        for (char c : s.toCharArray()) {
            int idx = c - 'a';
            if (node.children[idx] == null) return null;
            node = node.children[idx];
        }
        return node;
    }
}
```

Complexity: insert/search/prefix-check are all **O(L)** where L = string length — independent of how many words are stored! This beats a HashSet for prefix-based queries (HashSet can't efficiently answer "does any word start with 'ab'?" without scanning everything).

Use cases: autocomplete, spell-checkers, IP routing (longest prefix match), word search puzzles (combined with backtracking to prune the search space early — if the current path isn't a valid trie prefix, stop exploring immediately).

11.7 Tree Construction Problems

Problem: Construct Binary Tree from Preorder and Inorder Traversal.

- **Approach:** The first element of preorder is always the root. Find that value's position in inorder — everything to its left is the left subtree, everything to the right is the right subtree (recursively). Use a HashMap of `value → index in inorder` to make lookups O(1) instead of O(n). → **O(n) time (with the hashmap optimization), O(n) space.**

11.8 N-ary Trees & Graph-adjacent Tree Problems

Problem: Maximum Width of Binary Tree.

- **Approach:** BFS while assigning each node a positional index (as if it were in a complete binary tree: `left child = 2*i`, `right child = 2*i+1`); width of a level = `lastIndex - firstIndex + 1`. → **O(n) time, O(w) space.**

Problem: Binary Tree Right Side View.

- **Approach:** BFS, take the last node of each level. Or DFS (right-first) and record only the first node visited at each depth. → **O(n) time.**

---

12. Heaps / Priority Queues

12.1 What Is a Heap?

A heap is a complete binary tree satisfying the **heap property**: in a **min-heap**, every parent ≤ its children (so the root is always the minimum); in a **max-heap**, every parent ≥ its children.

Because it's a *complete* tree, it's efficiently stored as a flat array: for node at index `i`, `left child = 2i+1`, `right child = 2i+2`, `parent = (i-1)/2`. No pointers needed!

Operation	Time
Peek min/max	O(1)
Insert	O(log n)
Extract min/max	O(log n)
Build heap from array	O(n) — *not* O(n log n)! (Careful, this is a common interview trick question)

Why "build heap" is O(n) and not O(n log n): Naively you might think "n inserts × O(log n) each = O(n log n)." But the actual bottom-up heapify process does O(log n) work only for the ~n/2 nodes near the root, while the ~n/2 leaf nodes need **zero** work. Summing this geometric-like series across all levels gives O(n) total, not O(n log n).

```
// Java's built-in priority queue - min-heap by default
PriorityQueue<Integer> minHeap = new PriorityQueue<>();
PriorityQueue<Integer> maxHeap = new PriorityQueue<>(Collections.reverseOrder());
// Custom comparator, e.g., min-heap of int[] pairs by second element:
PriorityQueue<int[]> pq = new PriorityQueue<>((a, b) -> a[1] - b[1]);
```

12.2 Top K Elements Pattern (Pattern #12)

Core idea: to find the "top k" (largest or smallest), you DON'T need to sort the entire array (O(n log n)). Maintain a heap of size **k** and process elements one at a time — this gives **O(n log k)**, which beats O(n log n) whenever k << n.

Problem: Kth Largest Element in a Stream / Array.

- **Brute force:** Sort → O(n log n) every time new elements arrive (bad for a stream).
- **Optimized:** Maintain a **min-heap of size k**. For each new element: push it in; if heap size exceeds k, pop the minimum (the smallest of the current top-k gets

discarded). The heap's root is always the kth largest. → **O(log k) per insertion, O(n log k) total for n elements.**

```
class KthLargest {
    private PriorityQueue<Integer> minHeap;
    private int k;
    KthLargest(int k, int[] nums) {
        this.k = k;
        minHeap = new PriorityQueue<>();
        for (int n : nums) add(n);
    }
    int add(int val) {
        minHeap.offer(val);
        if (minHeap.size() > k) minHeap.poll();
        return minHeap.peek();
    }
}
```

Problem: Top K Frequent Elements.

- **Brute force:** Sort by frequency → O(n log n).
- **Optimized (Heap):** Count frequencies with a HashMap O(n), then use a min-heap of size k on frequency → **O(n log k) time.**
- **Most Optimal (Bucket Sort by frequency):** since frequency can only range from 1 to n, create `n+1` buckets indexed by frequency, place each element in its bucket, then read off the top k from the highest-frequency buckets down. → **O(n) time!** (This is a great "can you beat O(n log k)?" follow-up answer.)

Problem: K Closest Points to Origin.

- **Approach:** Max-heap of size k storing `(distance, point)`; if a new point is closer than the heap's max, pop max and push the new point. → **O(n log k) time.**

12.3 Two Heaps Pattern (Pattern #9)

Used to efficiently track the **median** of a running data stream, or to balance two halves of data.

Problem: Find Median from Data Stream.

- **Brute force:** Insert into a sorted list/array each time (O(n) insert due to shifting), or sort from scratch every query → O(n log n) per query.
- **Optimized (Two Heaps):** Maintain a **max-heap** (`lo`) for the smaller half of numbers, and a **min-heap** (`hi`) for the larger half, keeping their sizes balanced

(differ by at most 1). The median is either the top of the larger heap, or the average of both tops. → **O(log n) per insertion, O(1) per median query.**

```
class MedianFinder {
    PriorityQueue<Integer> lo = new PriorityQueue<>(Collections.reverseOrder()); //
max-heap, smaller half
    PriorityQueue<Integer> hi = new PriorityQueue<>(); // min-heap, larger half

    void addNum(int num) {
        lo.offer(num);
        hi.offer(lo.poll()); // balance: move the max of lo into hi
        if (hi.size() > lo.size()) lo.offer(hi.poll()); // keep lo >= hi in size
    }
    double findMedian() {
        if (lo.size() > hi.size()) return lo.peek();
        return (lo.peek() + hi.peek()) / 2.0;
    }
}
```

Why this maintains the invariant: every insertion always routes through both heaps (first into `lo`, then the largest of `lo` moves to `hi`), so `lo` always holds the smaller half and `hi` the larger half, with sizes never differing by more than 1.

Problem: Sliding Window Median — same two-heap idea, but with **lazy deletion** (mark elements as removed when they slide out of the window, and clean them up when they surface at the top of a heap) since heaps don't support O(log n) arbitrary removal natively.

12.4 K-Way Merge Pattern (Pattern #13)

Problem: Merge k Sorted Lists/Arrays — see Section 6.4. Use a min-heap holding one "current smallest candidate" per list; repeatedly extract the global min and push the next element from that same list. → **O(n log k) time** where n = total elements across all lists, k = number of lists.

Problem: Kth Smallest Element in a Sorted Matrix (rows and columns sorted).

- **Brute force:** Flatten and sort → O(n² log n²) for an n×n matrix.
- **Optimized (Heap, K-way merge style):** Push the first element of each row into a min-heap; pop the min k times, each time pushing the next element from the same row. → **O(k log n) time** (n = number of rows), much better when k << n².
- **Alternative (Binary search on answer):** binary search over the value range, counting how many elements are ≤ mid using the sorted-rows-and-columns

property (O(n) count per check) → **O(n log(max-min)) time** — often even better.

---

13. Graphs

13.1 Graph Representations

A graph is a set of **vertices (nodes)** connected by **edges**. Graphs can be **directed** or **undirected**, **weighted** or **unweighted**, and may contain **cycles**.

Adjacency List (most common in interviews): `Map<Integer, List<Integer>>` or `List<List<Integer>>` — space O(V + E), efficient for sparse graphs, easy to iterate over a node's neighbors.

Adjacency Matrix: `boolean[][]` or `int[][]` of size V×V — O(V²) space, O(1) edge lookup, good for dense graphs but wasteful for sparse ones.

```
List<List<Integer>> adjList = new ArrayList<>();
for (int i = 0; i < n; i++) adjList.add(new ArrayList<>());
for (int[] edge : edges) {
    adjList.get(edge[0]).add(edge[1]);
    adjList.get(edge[1]).add(edge[0]); // omit this line if directed
}
```

13.2 Breadth-First Search (BFS)

BFS explores level by level using a queue — guarantees the **shortest path in an unweighted graph**.

```
void bfs(int start, List<List<Integer>> adj) {
    boolean[] visited = new boolean[adj.size()];
    Queue<Integer> queue = new LinkedList<>();
    queue.offer(start);
    visited[start] = true;
    while (!queue.isEmpty()) {
        int node = queue.poll();
        // process node
        for (int neighbor : adj.get(node)) {
            if (!visited[neighbor]) {
                visited[neighbor] = true; // mark visited when ENQUEUING, not when
dequeuing (avoids duplicate enqueues)
                queue.offer(neighbor);
```

```
                }
            }
        }
    }
```

Complexity: O(V + E) time, O(V) space. Critical detail: always mark a node visited the moment it's enqueued, not when dequeued — otherwise the same node can be added to the queue multiple times before being processed, wasting time (and in graphs with many cross-edges, this can blow up complexity).

13.3 Depth-First Search (DFS)

DFS explores as far as possible along a branch before backtracking, using recursion (or an explicit stack).

```
void dfs(int node, List<List<Integer>> adj, boolean[] visited) {
    visited[node] = true;
    // process node
    for (int neighbor : adj.get(node)) {
        if (!visited[neighbor]) dfs(neighbor, adj, visited);
    }
}
```

Complexity: O(V + E) time, O(V) space (recursion stack, worst case for a path-like graph).

When to use BFS vs DFS:

- **BFS** → shortest path/minimum steps in unweighted graphs, level-order problems, "minimum number of operations."
- **DFS** → exploring all paths, detecting cycles, topological sort, connected components, backtracking-style graph problems, when you need to explore deeply before deciding (e.g., checking if a path exists at all).

13.4 Connected Components & Islands

Problem: Number of Islands (grid of 1s/0s, count connected groups of 1s).

- **Approach:** For each unvisited land cell, run BFS/DFS to mark the entire connected component as visited, incrementing a counter each time you start a new BFS/DFS. → **O(m·n) time, O(m·n) space** (visited array + recursion/queue).

Problem: Number of Connected Components in an Undirected Graph.

- **Approach:** Same idea — DFS/BFS from each unvisited node, or use **Union-Find** (see 13.8) which is often cleaner for this exact question. → O(V + E) with DFS/BFS, or ~O(V + E·α(V)) with Union-Find.

13.5 Cycle Detection

Undirected Graph: during DFS, if you visit a neighbor that's already visited AND it's not the immediate parent you came from, there's a cycle.

Directed Graph: simple "visited" tracking isn't enough (a node can be reached via multiple paths without a cycle). Use **3-color marking**: white (unvisited), gray (currently in the recursion stack / being processed), black (fully processed). If DFS encounters a **gray** node, that's a **back edge** → **cycle**.

```
boolean hasCycleDirected(int node, List<List<Integer>> adj, int[] color) {
    color[node] = 1; // gray
    for (int neighbor : adj.get(node)) {
        if (color[neighbor] == 1) return true; // back edge to a node in current
path
        if (color[neighbor] == 0 && hasCycleDirected(neighbor, adj, color)) return
true;
    }
    color[node] = 2; // black, fully done
    return false;
}
```

13.6 Topological Sort (Pattern #14)

Only defined for **Directed Acyclic Graphs (DAGs)**. Produces a linear ordering of vertices such that for every directed edge u → v, u comes before v. Classic use case: **course scheduling / build dependency ordering.**

Method 1 — Kahn's Algorithm (BFS-based):

1. Compute in-degree (number of incoming edges) for every node.
2. Push all nodes with in-degree 0 into a queue (they have no unmet dependencies).
3. Pop a node, add to result, decrement in-degree of its neighbors; if any neighbor's in-degree hits 0, push it into the queue.

4. If the result contains fewer than V nodes at the end → a cycle exists (topological sort impossible).

```
List<Integer> topologicalSort(int n, List<List<Integer>> adj) {
    int[] inDegree = new int[n];
    for (List<Integer> neighbors : adj) for (int v : neighbors) inDegree[v]++;
    Queue<Integer> queue = new LinkedList<>();
    for (int i = 0; i < n; i++) if (inDegree[i] == 0) queue.offer(i);
    List<Integer> result = new ArrayList<>();
    while (!queue.isEmpty()) {
        int node = queue.poll();
        result.add(node);
        for (int neighbor : adj.get(node)) {
            if (--inDegree[neighbor] == 0) queue.offer(neighbor);
        }
    }
    return result.size() == n ? result : new ArrayList<>(); // empty = cycle
detected
}
```

Method 2 — DFS-based: run DFS, and push each node onto a stack **after** all its descendants have been processed (postorder); reverse the stack at the end to get the topological order.

Complexity: O(V + E) for both methods.

Problem: Course Schedule (can all courses be finished given prerequisites)? → literally "does a topological sort exist?" i.e., "is the graph acyclic?" Use Kahn's algorithm; if result size < n, return false.

Problem: Course Schedule II (return a valid ordering) → return the `result` list directly from Kahn's algorithm above.

13.7 Shortest Path Algorithms

Algorithm	Graph type	Complexity	Notes
BFS	Unweighted	O(V + E)	Simplest, only for unweighted or equal-weight edges
Dijkstra	Weighted, non-negative	O((V + E) log V) with a min-heap	Greedy: always expand the closest unvisited node

Algorithm	Graph type	Complexity	Notes
Bellman-Ford	Weighted, allows negative edges	O(V·E)	Can detect negative cycles
Floyd-Warshall	All-pairs shortest path	$O(V^3)$	Good for small dense graphs, all pairs at once
A*	Weighted, with heuristic	Varies	Dijkstra + heuristic function to guide search toward the goal faster

Dijkstra's Algorithm (Deep Dive):

```
int[] dijkstra(int src, List<List<int[]>> adj, int n) { // adj: node -> list of [neighbor, weight]
    int[] dist = new int[n];
    Arrays.fill(dist, Integer.MAX_VALUE);
    dist[src] = 0;
    PriorityQueue<int[]> pq = new PriorityQueue<>((a, b) -> a[1] - b[1]); // [node, distance]
    pq.offer(new int[]{src, 0});
    while (!pq.isEmpty()) {
        int[] current = pq.poll();
        int node = current[0], d = current[1];
        if (d > dist[node]) continue; // stale entry, skip (lazy deletion)
        for (int[] edge : adj.get(node)) {
            int neighbor = edge[0], weight = edge[1];
            if (dist[node] + weight < dist[neighbor]) {
                dist[neighbor] = dist[node] + weight;
                pq.offer(new int[]{neighbor, dist[neighbor]});
            }
        }
    }
    return dist;
}
```

Why Dijkstra fails with negative edges: its greedy assumption is "once a node is popped with the smallest known distance, that distance is final." A negative edge encountered later could still improve an already-finalized node's distance, breaking this assumption.

Bellman-Ford (handles negative edges): relax **all** edges, **V-1** times (in the worst case, a shortest path uses at most V-1 edges). A **V-th pass** that still finds an improvement indicates a **negative cycle**.

```
int[] bellmanFord(int src, int[][] edges, int n) { // edges: [u, v, weight]
    int[] dist = new int[n];
    Arrays.fill(dist, Integer.MAX_VALUE);
    dist[src] = 0;
    for (int i = 0; i < n - 1; i++) {
        for (int[] edge : edges) {
            int u = edge[0], v = edge[1], w = edge[2];
            if (dist[u] != Integer.MAX_VALUE && dist[u] + w < dist[v]) dist[v] =
dist[u] + w;
        }
    }
    // optional Vth pass to detect negative cycle
    return dist;
}
```

Floyd-Warshall (all-pairs shortest paths): dynamic programming — `dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j])` for every intermediate node `k`. Simple triple nested loop, O(V³), fine for V up to ~400-500.

13.8 Union-Find (Disjoint Set Union / DSU)

Used to efficiently track which elements belong to the same group/set, and to merge groups — the backbone of Kruskal's MST algorithm and many "connected components" / "redundant connection" problems.

```
class UnionFind {
    int[] parent, rank;
    UnionFind(int n) {
        parent = new int[n];
        rank = new int[n];
        for (int i = 0; i < n; i++) parent[i] = i;
    }
    int find(int x) {
        if (parent[x] != x) parent[x] = find(parent[x]); // path compression
        return parent[x];
    }
    boolean union(int x, int y) {
        int rootX = find(x), rootY = find(y);
        if (rootX == rootY) return false; // already in same set (would form a
cycle)
        if (rank[rootX] < rank[rootY]) { int t = rootX; rootX = rootY; rootY = t; }
        parent[rootY] = rootX; // union by rank: attach smaller tree under bigger
tree's root
        if (rank[rootX] == rank[rootY]) rank[rootX]++;
        return true;
    }
}
```

Why near-O(1) per operation? With **path compression** (flatten the tree every time `find` is called) and **union by rank/size** (always attach the smaller tree under the bigger), the amortized complexity per operation becomes **O(α(n))**, where α is the inverse Ackermann function — grows so slowly it's effectively a constant (≤ 4 for any realistic input size).

Problem: Redundant Connection (find the edge that, if removed, makes the graph a tree — i.e., the one edge creating a cycle).

- **Approach:** Process edges one at a time; for each edge `(u, v)`, if `find(u) == find(v)` already, this edge creates a cycle — return it. Otherwise `union(u, v)`. → **O(E · α(V)) time**, effectively O(E).

Problem: Number of Provinces (count connected components given a matrix of direct connections). → Union-Find or DFS, both O(n²) to read the matrix (dominates complexity) + near-O(n) for the union-find part.

13.9 Minimum Spanning Tree (MST)

An MST connects all vertices with the minimum possible total edge weight, using exactly V-1 edges, no cycles.

Kruskal's Algorithm: sort all edges by weight; greedily add the smallest edge as long as it doesn't form a cycle (checked via Union-Find). → **O(E log E) time** (dominated by sorting).

```
int kruskalMST(int n, int[][] edges) { // edges: [u, v, weight]
    Arrays.sort(edges, (a, b) -> a[2] - b[2]);
    UnionFind uf = new UnionFind(n);
    int totalWeight = 0, edgesUsed = 0;
    for (int[] edge : edges) {
        if (uf.union(edge[0], edge[1])) {
            totalWeight += edge[2];
            edgesUsed++;
            if (edgesUsed == n - 1) break; // MST complete
        }
    }
    return totalWeight;
}
```

Prim's Algorithm: starts from any node, repeatedly adds the cheapest edge that connects the growing tree to a new vertex (using a min-heap, very similar structurally to Dijkstra). → **O(E log V) time** with a binary heap.

When to use which: Kruskal is simpler and preferred for sparse graphs (edge list already available); Prim is preferred for dense graphs (adjacency matrix / when starting from a specific node matters).

13.10 Bipartite Graph Check

Problem: Is Graph Bipartite? (can nodes be 2-colored such that no edge connects same-colored nodes?)

- **Approach:** BFS/DFS coloring — color the start node color A, all its neighbors color B, their neighbors color A, etc. If you ever find an edge between two same-colored nodes, it's not bipartite. → **O(V + E) time.** (Equivalent to checking for odd-length cycles.)

13.11 Multi-Source BFS

Problem: Rotting Oranges (all rotten oranges spread to adjacent fresh oranges every minute — find time until all rot, or -1 if impossible).

- **Approach:** Push **all** initially-rotten oranges into the BFS queue at once (multi-source BFS), then process level by level — level index = minutes elapsed. → **O(m·n) time, O(m·n) space.**

Problem: Walls and Gates / 01 Matrix (distance to nearest 0).

- **Approach:** Multi-source BFS starting from all 0-cells simultaneously — guarantees each cell's first-visited distance is its shortest distance to *any* 0. → **O(m·n) time.**

13.12 0-1 BFS (Advanced but Very Useful Trick)

When edge weights are only 0 or 1, you can find shortest paths in **O(V + E)** (better than Dijkstra's O((V+E) log V)) using a **deque**: push to the front for a 0-weight edge, push to the back for a 1-weight edge. This keeps the deque roughly sorted by distance without needing a heap.

---

14. Dynamic Programming

14.1 What Is Dynamic Programming, Really?

DP = **recursion + remembering answers to subproblems you've already solved** (so you never recompute them). It applies when a problem has:

1. **Optimal substructure:** the optimal solution to the problem can be built from optimal solutions to its subproblems.
2. **Overlapping subproblems:** the same subproblems are solved again and again in a naive recursive solution (this is the key difference from Divide & Conquer, where subproblems are independent/non-overlapping).

14.2 The Universal DP Framework (Use This Every Time)

1. **Define the state.** What parameters uniquely describe a subproblem? (e.g., `dp[i]` = "answer considering first i elements", or `dp[i][j]` = "answer using first i items with capacity j").
2. **Write the recurrence.** How does `dp[state]` relate to smaller states? (This is the hardest and most important step — it comes directly from thinking about the *last decision* made.)
3. **Identify the base case(s).**
4. **Decide the order of computation** (top-down memoization, or bottom-up tabulation).
5. **Optimize space** if possible (often you only need the previous row/few states, not the entire table).

14.3 Memoization (Top-Down) vs Tabulation (Bottom-Up)

- **Memoization:** write the natural recursive solution, add a cache (HashMap or array) that stores already-computed results, and check the cache before recomputing. Easier to write correctly (closer to brute force); has recursion overhead and stack depth risk.

- **Tabulation:** build the answer iteratively from the base case upward, filling a table. No recursion overhead, but requires figuring out the correct iteration order upfront.

Example: Fibonacci, Naive → Memoized → Tabulated → Space-Optimized

```
// Naive: O(2^n) time
int fibNaive(int n) {
    if (n <= 1) return n;
    return fibNaive(n - 1) + fibNaive(n - 2);
}

// Memoized (top-down): O(n) time, O(n) space
int fibMemo(int n, int[] memo) {
    if (n <= 1) return n;
    if (memo[n] != -1) return memo[n];
    return memo[n] = fibMemo(n - 1, memo) + fibMemo(n - 2, memo);
}

// Tabulated (bottom-up): O(n) time, O(n) space
int fibTab(int n) {
    if (n <= 1) return n;
    int[] dp = new int[n + 1];
    dp[0] = 0; dp[1] = 1;
    for (int i = 2; i <= n; i++) dp[i] = dp[i - 1] + dp[i - 2];
    return dp[n];
}

// Space-optimized: O(n) time, O(1) space (only need last 2 values)
int fibOptimal(int n) {
    if (n <= 1) return n;
    int prev2 = 0, prev1 = 1;
    for (int i = 2; i <= n; i++) {
        int curr = prev1 + prev2;
        prev2 = prev1; prev1 = curr;
    }
    return prev1;
}
```

This progression (naive → memo → tabulation → space-optimized) is exactly how you should narrate EVERY DP answer in an interview. It shows complete mastery of the trade-offs.

14.4 1D DP Problems

Problem: Climbing Stairs (1 or 2 steps at a time, count distinct ways to reach the top).

- **Recurrence:** `dp[i] = dp[i-1] + dp[i-2]` (you reach step i either from i-1 with a single step, or from i-2 with a double step). Literally Fibonacci in disguise. →

O(n) time, O(1) space (optimized).

Problem: House Robber (can't rob two adjacent houses, maximize total).

- **Recurrence:** `dp[i] = max(dp[i-1], dp[i-2] + nums[i])` — either skip house i (take best of first i-1 houses), or rob house i (best of first i-2 houses, plus this house's value). → **O(n) time, O(1) space.**

```
int rob(int[] nums) {
    int prev2 = 0, prev1 = 0;
    for (int num : nums) {
        int curr = Math.max(prev1, prev2 + num);
        prev2 = prev1; prev1 = curr;
    }
    return prev1;
}
```

Problem: House Robber II (houses arranged in a circle — first and last are adjacent).

- **Approach:** Run House Robber twice — once excluding the first house, once excluding the last house — and take the max. → **O(n) time, O(1) space.**

Problem: Maximum Product Subarray.

- **Tricky twist:** a negative number can flip a very negative running product into the maximum (two negatives make a positive). Track **both** `maxProduct` and `minProduct` ending at each index. `dp_max[i] = max(nums[i], nums[i]*dp_max[i-1], nums[i]*dp_min[i-1])`, similarly for `dp_min`. → **O(n) time, O(1) space.**

Problem: Coin Change (minimum coins to make amount).

- **Brute force (pure recursion):** try every coin at every amount → exponential.
- **DP:** `dp[amount] = min(dp[amount - coin] + 1)` for every coin ≤ amount; `dp[0] = 0`; unreachable amounts = infinity. → **O(amount × coins) time, O(amount) space.**

```
int coinChange(int[] coins, int amount) {
    int[] dp = new int[amount + 1];
    Arrays.fill(dp, amount + 1); // "infinity" sentinel
    dp[0] = 0;
    for (int i = 1; i <= amount; i++) {
        for (int coin : coins) {
            if (coin <= i) dp[i] = Math.min(dp[i], dp[i - coin] + 1);
        }
```

```
    }
    return dp[amount] > amount ? -1 : dp[amount];
}
```

Problem: Coin Change II (number of ways to make amount, unbounded coins).

- **Recurrence:** iterate coins in the OUTER loop, amounts in the inner loop: `dp[amount] += dp[amount - coin]`. **Why outer-loop-on-coins matters:** it ensures each combination is counted once (as a *combination*, not a *permutation* — e.g., [1,2] and [2,1] aren't double counted). → **O(amount × coins) time, O(amount) space.**

14.5 0/1 Knapsack Pattern

The Problem: Given items with weights and values, and a knapsack of capacity W, maximize value without exceeding capacity — each item used **at most once**.

State: `dp[i][w]` = max value achievable using the first i items with capacity w.
Recurrence: `dp[i][w] = max(dp[i-1][w], dp[i-1][w - weight[i]] + value[i])` (skip item i, or take it if it fits).

```
int knapsack01(int[] weights, int[] values, int capacity) {
    int n = weights.length;
    int[][] dp = new int[n + 1][capacity + 1];
    for (int i = 1; i <= n; i++) {
        for (int w = 0; w <= capacity; w++) {
            dp[i][w] = dp[i - 1][w]; // don't take item i-1
            if (weights[i - 1] <= w) {
                dp[i][w] = Math.max(dp[i][w], dp[i - 1][w - weights[i - 1]] +
values[i - 1]);
            }
        }
    }
    return dp[n][capacity];
}
// Space-optimized: O(capacity) space — iterate w DECREASING to avoid using an item
twice
int knapsack01Optimized(int[] weights, int[] values, int capacity) {
    int[] dp = new int[capacity + 1];
    for (int i = 0; i < weights.length; i++) {
        for (int w = capacity; w >= weights[i]; w--) { // must go backwards!
            dp[w] = Math.max(dp[w], dp[w - weights[i]] + values[i]);
        }
    }
    return dp[capacity];
}
```

Why iterate `w` backwards in the optimized version: going forward would let you reuse item `i` multiple times in the same row update (since `dp[w - weights[i]]` might already reflect item `i` being used) — going backward ensures `dp[w - weights[i]]` still reflects the *previous* row's state (item i not yet used).

This exact 0/1 knapsack pattern (with variations) solves:

Problem: Subset Sum (does any subset sum to exactly target?) — knapsack where "value" doesn't matter, only whether a weight-sum is achievable: `dp[w] = dp[w] || dp[w - num]`.

Problem: Partition Equal Subset Sum — subset sum where target = totalSum / 2 (if totalSum is odd, immediately return false).

Problem: Target Sum (assign +/- to each number to reach a target) — transforms into a subset-sum problem: find subset P (positive assignments) such that `sum(P) = (target + totalSum) / 2`.

14.6 Unbounded Knapsack Pattern

Same as 0/1 knapsack, but each item can be used **unlimited** times. Key difference: iterate `w` **forwards** (not backwards), since reusing the same item multiple times is exactly what we want.

```
int unboundedKnapsack(int[] weights, int[] values, int capacity) {
    int[] dp = new int[capacity + 1];
    for (int i = 0; i < weights.length; i++) {
        for (int w = weights[i]; w <= capacity; w++) { // forwards!
            dp[w] = Math.max(dp[w], dp[w - weights[i]] + values[i]);
        }
    }
    return dp[capacity];
}
```

This pattern directly solves **Coin Change**, **Coin Change II**, **Rod Cutting**.

14.7 Longest Common Subsequence (LCS) Family

Problem: Longest Common Subsequence of two strings.

- **State:** `dp[i][j]` = LCS length of `s1[0..i)` and `s2[0..j)`.
- **Recurrence:** if `s1[i-1] == s2[j-1]`, `dp[i][j] = dp[i-1][j-1] + 1`; else `dp[i][j] = max(dp[i-1][j], dp[i][j-1])`. → **O(n·m) time, O(n·m) space** (reducible to O(min(n,m)) space using rolling rows).

```
int longestCommonSubsequence(String s1, String s2) {
    int n = s1.length(), m = s2.length();
    int[][] dp = new int[n + 1][m + 1];
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= m; j++) {
            if (s1.charAt(i - 1) == s2.charAt(j - 1)) dp[i][j] = dp[i - 1][j - 1] +
1;
            else dp[i][j] = Math.max(dp[i - 1][j], dp[i][j - 1]);
        }
    }
    return dp[n][m];
}
```

Derived Problems (same table, different reading):

- **Longest Common Substring:** same DP but requires contiguity — recurrence becomes `dp[i][j] = dp[i-1][j-1] + 1` ONLY on a match, else `dp[i][j] = 0` (reset, no carry-over). Track the max value seen anywhere in the table.
- **Edit Distance** (minimum insert/delete/replace operations to convert s1 to s2): `dp[i][j] = dp[i-1][j-1]` if chars match; else `1 + min(dp[i-1][j], dp[i][j-1], dp[i-1][j-1])` (delete, insert, replace respectively). → **O(n·m) time.**
- **Longest Palindromic Subsequence** = LCS of the string with its own reverse.
- **Shortest Common Supersequence length** = n + m - LCS(s1, s2).
- **Minimum Insertions/Deletions to make two strings equal** = `(n - LCS) + (m - LCS)`.

14.8 Longest Increasing Subsequence (LIS) Family

Problem: Longest Increasing Subsequence.

- **Brute force DP:** `dp[i]` = length of LIS ending at index i = `1 + max(dp[j])` for all `j < i` where `nums[j] < nums[i]`. → **O(n²) time, O(n) space.**
- **Optimized (Binary Search + Patience Sorting):** maintain a `tails` array where `tails[k]` = smallest possible tail value of an increasing subsequence of length

k+1. For each new number, binary search for its position in `tails` (replace if found, append if it's a new max) → **O(n log n) time.**

```
int lengthOfLIS(int[] nums) {
    List<Integer> tails = new ArrayList<>();
    for (int num : nums) {
        int pos = Collections.binarySearch(tails, num);
        if (pos < 0) pos = -(pos + 1); // insertion point
        if (pos == tails.size()) tails.add(num);
        else tails.set(pos, num);
    }
    return tails.size();
}
```

Why this works (the non-obvious part): `tails` doesn't necessarily represent a *real* subsequence, but its **length always equals the true LIS length so far**, because keeping tail values as small as possible always gives the best chance of extending with future numbers.

Derived Problems:

- **Russian Doll Envelopes:** sort by width ascending (and height descending for ties, to avoid same-width envelopes counting as nested), then find LIS on heights.
- **Maximum Sum Increasing Subsequence:** same O(n²) DP, but recurrence tracks sum instead of length.
- **Number of LIS:** track both `length[i]` and `count[i]` (number of LIS ending at i) simultaneously.

14.9 DP on Grids

Problem: Unique Paths (robot moving right/down only, count paths from top-left to bottom-right).

- **Recurrence:** `dp[i][j] = dp[i-1][j] + dp[i][j-1]` (came from above or from the left). Base case: first row and first column = 1. → **O(m·n) time, O(n) space** (rolling row).

Problem: Minimum Path Sum.

- **Recurrence:** `dp[i][j] = grid[i][j] + min(dp[i-1][j], dp[i][j-1])`. → **O(m·n) time, O(n) space.**

Problem: Dungeon Game (minimum initial health to survive a grid).

- **Key insight:** must be solved **backwards** (from bottom-right to top-left), since the health requirement at a cell depends on the requirement of cells *after* it, not before.

14.10 Palindrome DP

Problem: Longest Palindromic Substring — see Section 4.12 (Expand Around Center is usually preferred for O(n²)/O(1) space, but a DP formulation also exists: `dp[i][j] = (s[i]==s[j]) && dp[i+1][j-1]`, O(n²) time and space).

Problem: Palindrome Partitioning II (minimum cuts to partition a string into palindromes).

- **Approach:** Precompute `isPalindrome[i][j]` for all substrings (O(n²)), then `dp[i]` = min cuts for `s[0..i]` = `min(dp[j] + 1)` for all `j < i` where `s[j+1..i]` is a palindrome. → **O(n²) time, O(n²) space.**

14.11 Matrix Chain Multiplication Pattern (Interval DP)

The Problem: Given a chain of matrices, find the optimal way to parenthesize multiplications to minimize scalar multiplications.

- **State:** `dp[i][j]` = min cost to multiply matrices from i to j.
- **Recurrence:** `dp[i][j] = min over k (dp[i][k] + dp[k+1][j] + cost of multiplying the two resulting matrices)`.
- This "try every split point k" recurrence is the template for **all interval DP problems**: `dp[i][j] = best over k in (i, j) of combine(dp[i][k], dp[k][j])`. → Generally **O(n³) time** (O(n²) states, O(n) split points each).

Other Interval DP Problems: Burst Balloons, Minimum Cost to Merge Stones, Boolean Parenthesization.

14.12 DP on Trees

Problem: House Robber III (binary tree version — can't rob a node and its direct children).

- **Approach:** Each node returns a pair `(maxIfRobbed, maxIfNotRobbed)` to its parent. `robbed = node.val + leftNotRobbed + rightNotRobbed`; `notRobbed = max(leftRobbed, leftNotRobbed) + max(rightRobbed, rightNotRobbed)`. → **O(n) time, O(h) space** — a single post-order DFS pass, no repeated subtree computation.

Problem: Diameter of Binary Tree (revisited as tree DP) — see Section 11.3, structurally identical to "combine children's DP results at each node."

14.13 Bitmask DP (for Small n, Typically n ≤ 20)

Used when the state needs to track "which subset of items has been used/visited" — represented as an integer bitmask.

Problem: Traveling Salesman Problem (TSP).

- **State:** `dp[mask][i]` = minimum cost to visit exactly the set of cities in `mask`, ending at city `i`.
- **Recurrence:** `dp[mask][i] = min over j in mask (dp[mask without i][j] + cost[j][i])`.
- **Complexity: O(n² · 2^n)** — exponential, but far better than the O(n!) brute force of trying every permutation, and totally fine for n ≤ ~18-20.

Problem: Partition to K Equal Sum Subsets.

- **Approach:** `dp[mask]` = true if the subset represented by `mask` can be perfectly partitioned so far; try adding each unused element to the current bucket. → Exponential but pruned heavily using the bitmask to avoid recomputation of identical subsets.

14.14 Digit DP (For "Count Numbers in Range With Property X")

Used for problems like "count numbers between L and R whose digits satisfy some property" (e.g., digit sum, no repeated digits) when the range is too large to iterate directly (e.g., up to 10^18).

Core idea: process digits one at a time (left to right), tracking:

- Current position.
- Whether the number built so far is still "tight" (equal to the prefix of the upper bound so far, restricting future digit choices) or already strictly less (free to choose any digit 0-9).
- Any problem-specific state (e.g., running digit sum, count of a specific digit, leading-zero flag).

This collapses what looks like an O(R - L) brute force into **O(digits × states)**, i.e., roughly O(log(R) × 10 × extra-state-size) — a huge win for astronomically large ranges. This is an advanced/rare topic — knowing the *name and idea* is usually sufficient unless applying to a company known for asking it.

14.15 State Machine DP (Stock Buy/Sell Problems)

Problem: Best Time to Buy and Sell Stock (one transaction).

- **Approach:** Track `minPriceSoFar` and `maxProfitSoFar` in one linear pass. → **O(n) time, O(1) space.**

Problem: Best Time to Buy and Sell Stock II (unlimited transactions).

- **Approach:** Greedy — sum up every positive day-to-day price difference (equivalent to buying/selling on every upswing). → **O(n) time, O(1) space.**

Problem: Best Time to Buy and Sell Stock III / IV (at most k transactions) — the general state machine.

- **State:** `dp[i][k][holding]` = max profit on day i, having completed/started at most k transactions, currently holding a stock or not.
- **Recurrence:** `dp[i][k][0] = max(dp[i-1][k][0], dp[i-1][k][1] + price[i])` (rest, or sell), `dp[i][k][1] = max(dp[i-1][k][1], dp[i-1][k-1][0] - price[i])` (rest, or buy — starts a new transaction). → **O(n·k) time**, reducible to O(1) space per k-layer via rolling variables.
- **With cooldown / transaction fee:** just add extra states/transitions (e.g., a "cooldown" state after selling, or subtract a fee upon selling) — same skeleton.

This "draw the state machine, define transitions between states" approach generalizes to a huge class of DP problems beyond just stocks (e.g., "paint house," "typing with a broken keyboard").

14.16 How to Recognize DP in an Interview (Trigger Words)

- "Find the **maximum/minimum**..." combined with a constraint (weight limit, count of allowed operations).
- "**Count the number of ways**..." to do something.
- "**Can you achieve...**" (a boolean feasibility, e.g., subset sum, partition).
- The problem asks about **subsequences** (not requiring contiguity) — LCS/LIS family.
- Small-ish constraints for a naive-looking exponential recursive solution (e.g., $n \leq 1000$ rules out $O(2^n)$, but suggests $O(n^2)$ DP is expected).
- If your first instinct is "try every possibility recursively," but the same sub-inputs recur — that's your cue to memoize.

---

15. Greedy Algorithms

15.1 What Makes an Algorithm "Greedy"?

A greedy algorithm builds a solution piece by piece, always choosing the option that looks best **right now**, without reconsidering past choices. It only produces a globally optimal answer when the problem has the **greedy-choice property** (a locally optimal choice leads to a globally optimal solution) — you often need to *prove* (or at least convince an interviewer) why greedy works, since it doesn't for every problem (e.g., greedy fails for 0/1 Knapsack but works for the Fractional Knapsack).

15.2 How to Identify When Greedy Works

- Ask: "If I make the locally best choice now, could a different choice ever lead to a strictly better overall outcome?" If you can convince yourself (via an exchange argument — showing any optimal solution can be transformed into the greedy one without making it worse) that it can't, greedy is safe.
- Interval scheduling, activity selection, and Huffman coding are the classic proof-friendly examples.

15.3 Classic Greedy Problems

Problem: Activity Selection / Non-overlapping Intervals (max number of non-overlapping intervals, or min removals to make non-overlapping).

- **Greedy rule:** Sort intervals by **end time**. Always pick the interval that finishes earliest among remaining valid choices — this leaves maximum room for future intervals. → **O(n log n) time** (dominated by sort).

```
int eraseOverlapIntervals(int[][] intervals) {
    Arrays.sort(intervals, (a, b) -> a[1] - b[1]); // sort by end time
    int count = 0, prevEnd = Integer.MIN_VALUE;
    for (int[] interval : intervals) {
        if (interval[0] >= prevEnd) { prevEnd = interval[1]; } // keep this
interval
        else count++; // must remove this one (overlaps)
    }
    return count;
}
```

Problem: Jump Game (can you reach the last index, given max jump length at each index?)

- **Greedy:** track the farthest reachable index seen so far while scanning left to right; if you ever reach an index beyond the farthest-reachable point before getting there, return false. → **O(n) time, O(1) space.**

Problem: Jump Game II (minimum jumps to reach the end).

- **Greedy (BFS-like level tracking):** track `currentEnd` (farthest reachable with current number of jumps) and `farthest` (farthest reachable with one more jump); when you reach `currentEnd`, increment jump count and update `currentEnd = farthest`. → **O(n) time, O(1) space.**

Problem: Gas Station (can you complete a circular route given gas[i] and cost[i] at each station?)

- **Key insight:** if total gas ≥ total cost, a solution always exists. Track running tank; whenever it goes negative, the start candidate must be after the current station (none of the stations up to here could be valid starts either). → **O(n) time, O(1) space** — surprisingly elegant given how complex the problem sounds.

Problem: Task Scheduler (CPU tasks with a cooldown period between same tasks, minimize total time).

- **Greedy (math formula) approach:** the answer is `max(n, (maxFreq - 1) * (cooldown + 1) + countOfTasksWithMaxFreq)`, derived by picturing the most frequent task's occurrences with mandatory gaps, and filling gaps with other tasks. → **O(n) time** (counting frequencies).

Problem: Huffman Encoding (build optimal prefix-free binary codes for characters based on frequency, minimizing total encoded length).

- **Greedy + Heap:** repeatedly merge the two least-frequent nodes into a new parent node (frequency = sum), until one tree remains. → **O(n log n) time** using a min-heap.

Problem: Minimum Number of Platforms (given train arrival/departure times, find max trains present at once).

- **Approach:** Sort arrivals and departures separately; two-pointer merge (like merging two sorted lists) — increment a counter on an arrival, decrement on a departure, track the max. → **O(n log n) time** (dominated by sorting).

Problem: Fractional Knapsack (items can be split, unlike 0/1 knapsack).

- **Greedy:** sort items by value/weight ratio descending, greedily take as much as possible of the highest-ratio items first. → **O(n log n) time.** (Contrast with 0/1 Knapsack, where greedy provably fails because items can't be split — you need full DP there.)

15.4 Greedy vs DP — How to Tell Them Apart

	Greedy	DP
Decision reconsidered?	Never — commit and move on	Yes — explores all valid choices via subproblems
Proof needed	Yes (exchange argument / matroid theory)	Correctness follows from recurrence being exhaustive
Speed	Usually faster (O(n log n) or O(n))	Usually slower (O(n²), O(n·capacity), etc.)

	Greedy	DP
Example	Interval scheduling, Fractional Knapsack	0/1 Knapsack, LCS, Edit Distance

Interview tip: if you're not 100% sure greedy is correct, mention it as a hypothesis, try to find a counter-example quickly (small manual test case), and if you can't and it "feels right" structurally (matches an interval-scheduling or exchange-argument pattern), propose it — but always mention "I'd want to verify this greedy choice doesn't have a counterexample" to show awareness.

---

16. Advanced Data Structures

16.1 Segment Tree

Solves **range query + range/point update** problems in O(log n), where a simple prefix-sum array (O(1) query but O(n) update) or brute-force (O(n) query) would be too slow when **both** queries and updates happen frequently.

Structure: a binary tree where each node represents a range `[l, r]` of the array; leaves represent single elements; each internal node's value is a "combination" (sum, min, max, gcd, etc.) of its two children's ranges.

```
class SegmentTree {
    int[] tree;
    int n;
    SegmentTree(int[] arr) {
        n = arr.length;
        tree = new int[4 * n]; // safe upper bound on size
        build(arr, 1, 0, n - 1);
    }
    void build(int[] arr, int node, int start, int end) {
        if (start == end) { tree[node] = arr[start]; return; }
        int mid = (start + end) / 2;
        build(arr, 2 * node, start, mid);
        build(arr, 2 * node + 1, mid + 1, end);
        tree[node] = tree[2 * node] + tree[2 * node + 1]; // combine (sum here)
    }
    void update(int node, int start, int end, int idx, int val) {
        if (start == end) { tree[node] = val; return; }
        int mid = (start + end) / 2;
        if (idx <= mid) update(2 * node, start, mid, idx, val);
        else update(2 * node + 1, mid + 1, end, idx, val);
        tree[node] = tree[2 * node] + tree[2 * node + 1];
```

```
    }
    int query(int node, int start, int end, int l, int r) { // range sum [l, r]
        if (r < start || end < l) return 0; // no overlap
        if (l <= start && end <= r) return tree[node]; // total overlap
        int mid = (start + end) / 2; // partial overlap
        return query(2 * node, start, mid, l, r) + query(2 * node + 1, mid + 1,
end, l, r);
    }
}
```

Complexity: build O(n), update O(log n), range query O(log n). **Lazy propagation** extends this to support O(log n) **range updates** too (e.g., "add 5 to every element in [l, r]") by deferring updates to children until they're actually needed.

16.2 Fenwick Tree / Binary Indexed Tree (BIT)

A simpler, more compact alternative to a segment tree for **prefix sum queries + point updates** — same O(log n) complexity but much less code, using the bit-trick `i & (-i)` to isolate the lowest set bit.

```
class FenwickTree {
    int[] tree;
    int n;
    FenwickTree(int n) { this.n = n; tree = new int[n + 1]; }
    void update(int i, int delta) { // add delta at index i (1-indexed)
        for (; i <= n; i += i & (-i)) tree[i] += delta;
    }
    int query(int i) { // prefix sum [1, i]
        int sum = 0;
        for (; i > 0; i -= i & (-i)) sum += tree[i];
        return sum;
    }
    int rangeQuery(int l, int r) { return query(r) - query(l - 1); }
}
```

Why it works: each index in the Fenwick array is responsible for a specific range determined by its lowest set bit, so both update and query only ever touch O(log n) nodes by repeatedly adding/subtracting the lowest set bit.

Use cases: counting inversions in an array, range-sum with point updates, order statistics (count of elements ≤ x seen so far).

16.3 LRU Cache (Extremely Common Design Question)

Problem: Design a Least Recently Used (LRU) Cache with O(1) `get` and `put`.

- **Approach:** Combine a **HashMap** (key → node reference, O(1) lookup) with a **Doubly Linked List** (maintains usage order — move-to-front on access, evict from the back when full). Neither structure alone achieves O(1) for both operations: an array/list alone can't do O(1) lookup by key, and a HashMap alone can't maintain/update order in O(1).

```
class LRUCache {
    class Node { int key, value; Node prev, next; Node(int k, int v) { key = k;
value = v; } }
    Map<Integer, Node> map = new HashMap<>();
    int capacity;
    Node head = new Node(-1, -1), tail = new Node(-1, -1); // dummy sentinels
    LRUCache(int capacity) { this.capacity = capacity; head.next = tail; tail.prev
= head; }

    int get(int key) {
        if (!map.containsKey(key)) return -1;
        Node node = map.get(key);
        remove(node); insertAtFront(node); // mark as most recently used
        return node.value;
    }
    void put(int key, int value) {
        if (map.containsKey(key)) { remove(map.get(key)); }
        if (map.size() == capacity) { map.remove(tail.prev.key); remove(tail.prev);
} // evict LRU
        Node node = new Node(key, value);
        map.put(key, node);
        insertAtFront(node);
    }
    void remove(Node node) { node.prev.next = node.next; node.next.prev =
node.prev; }
    void insertAtFront(Node node) {
        node.next = head.next; node.prev = head;
        head.next.prev = node; head.next = node;
    }
}
```

Complexity: O(1) for both get and put. Java also offers `LinkedHashMap` with `removeEldestEntry` overridden, which gives you this behavior "for free" if allowed to use built-ins.

16.4 LFU Cache (Least Frequently Used)

More advanced: evict the **least frequently used** item (ties broken by least recently used). Requires a HashMap of key→node, a HashMap of frequency→doubly-linked-

list-of-nodes-with-that-frequency, and tracking the current minimum frequency. → **O(1)** for get/put, but noticeably trickier to implement correctly — a great "have you seen this before?" filter question at senior levels.

16.5 Sparse Table (Static Range Queries, No Updates)

For range **min/max/gcd** queries on a **static** array (no updates), a sparse table precomputes answers for all ranges of length `2^k` starting at each index. → **O(n log n) preprocessing, O(1) query** for idempotent operations like min/max/gcd (overlapping ranges don't cause double-counting issues, unlike sum).

16.6 Skip List

A probabilistic data structure using multiple layers of linked lists with "express lanes" to skip ahead — gives O(log n) expected search/insert/delete without the complexity of tree rotations (used in Redis sorted sets). Good to know the *name and concept* even if rarely implemented live.

---

17. The Master Pattern Cheat-Sheet

> This is the section to re-read the night before an interview. For each pattern: **when to use it**, the **template**, and a **problem checklist**.

Pattern 1 — Two Pointers

Use when: array/string is sorted (or sortable), and you're looking for pairs/triplets, or comparing from both ends. **Trigger words:** "pair that sums to," "sorted array," "palindrome check," "remove duplicates in place." **Template:**

```
left = 0, right = n - 1
while left < right:
    evaluate condition using arr[left], arr[right]
    move left forward and/or right backward based on condition
```

Problem checklist: Two Sum II, 3Sum, 4Sum, Container With Most Water, Trapping Rain Water, Sort Colors, Valid Palindrome, Remove Duplicates from Sorted Array.

Pattern 2 — Sliding Window

Use when: contiguous subarray/substring problems ("longest," "smallest," "max sum of size k"). **Trigger words:** "contiguous," "substring," "subarray," "window of size k," "at most k distinct." **Template (variable window):**

```
left = 0
for right in 0..n-1:
    add arr[right] to window state
    while window is invalid:
        remove arr[left] from window state
        left += 1
    update answer using window [left, right]
```

Problem checklist: Max Sum Subarray Size K, Longest Substring Without Repeating Characters, Minimum Window Substring, Longest Substring with At Most K Distinct Characters, Fruit Into Baskets, Permutation in String, Find All Anagrams in a String.

Pattern 3 — Fast & Slow Pointers (Floyd's Cycle Detection)

Use when: linked list or "sequence that could cycle" problems; finding middle elements. **Trigger words:** "cycle," "duplicate number in array using pointers," "middle of linked list," "happy number." **Template:**

```
slow = head, fast = head
while fast and fast.next:
    slow = slow.next
    fast = fast.next.next
    if slow == fast: cycle found
```

Problem checklist: Linked List Cycle I/II, Middle of Linked List, Happy Number, Find the Duplicate Number, Palindrome Linked List.

Pattern 4 — Merge Intervals

Use when: dealing with intervals/ranges that may overlap. **Trigger words:** "intervals," "meeting rooms," "schedule," "overlap." **Template:**

```
sort intervals by start time
for each interval:
    if it overlaps with the last merged interval: merge them
    else: add as a new interval
```

Problem checklist: Merge Intervals, Insert Interval, Non-overlapping Intervals, Meeting Rooms I/II, Interval List Intersections.

Pattern 5 — Cyclic Sort

Use when: array contains numbers in range `[1,n]` or `[0,n-1]`, need O(n)/O(1) space to find missing/duplicate/first-missing-positive. **Template:**

```
i = 0
while i < n:
    correct = arr[i] - 1
    if arr[i] != arr[correct]: swap(arr[i], arr[correct])
    else: i += 1
```

Problem checklist: Missing Number, Find All Missing Numbers, Find the Duplicate Number, Find All Duplicates, First Missing Positive, Set Mismatch.

Pattern 6 — In-Place Reversal of Linked List

Use when: need to reverse a linked list (or part of it) without extra space. **Template:**

```
prev = null, curr = head
while curr:
    next = curr.next
    curr.next = prev
    prev = curr
    curr = next
```

Problem checklist: Reverse Linked List, Reverse Linked List II, Reverse Nodes in k-Group, Swap Nodes in Pairs, Rotate List.

Pattern 7 — Tree BFS

Use when: level-order processing of a tree/graph. **Template:**

```
queue = [root]
while queue:
    level_size = len(queue)
    for _ in range(level_size):
        node = queue.pop_front()
        process(node)
        queue.push(node.children)
```

Problem checklist: Binary Tree Level Order Traversal, Zigzag Level Order, Binary Tree Right Side View, Average of Levels, Minimum Depth of Binary Tree, Connect Level Order Siblings.

Pattern 8 — Tree DFS

Use when: path-related problems, or need to explore all root-to-leaf paths. **Template:**

```
function dfs(node, path):
    if node is null: return
    path.add(node.val)
    if node is leaf: process(path)
    dfs(node.left, path); dfs(node.right, path)
    path.remove(last)  # backtrack
```

Problem checklist: Path Sum I/II/III, Binary Tree Maximum Path Sum, All Root-to-Leaf Paths, Diameter of Binary Tree, Lowest Common Ancestor.

Pattern 9 — Two Heaps

Use when: need median of a stream, or need to balance two halves of data. **Template:** max-heap for smaller half + min-heap for larger half, rebalance after every insert so sizes differ by ≤1. **Problem checklist:** Find Median from Data Stream, Sliding Window Median, IPO (Maximize Capital), Scheduling to Minimize Idle Time.

Pattern 10 — Subsets (Backtracking)

Use when: need all subsets/permutations/combinations. **Template:** see Section 8.2/8.3 backtracking template. **Problem checklist:** Subsets I/II, Permutations I/II, Combination Sum I/II/III, Generate Parentheses, Letter Combinations of Phone Number, Palindrome Partitioning.

Pattern 11 — Modified Binary Search

Use when: search space is sorted, rotated, or has a monotonic feasibility condition ("binary search on the answer"). **Template:** see Section 10.2/10.4. **Problem checklist:** Search in Rotated Sorted Array, Find Minimum in Rotated Sorted Array, Find Peak Element, Koko Eating Bananas, Capacity to Ship Packages, Split Array Largest Sum, Median of Two Sorted Arrays.

Pattern 12 — Top K Elements

Use when: need k largest/smallest/most-frequent, without full sort. **Template:** min-heap of size k (for "k largest") or max-heap of size k (for "k smallest"). **Problem checklist:** Kth Largest Element, Top K Frequent Elements, K Closest Points to Origin, Sort Characters by Frequency, Kth Largest Element in a Stream.

Pattern 13 — K-Way Merge

Use when: merging k sorted lists/arrays. **Template:** min-heap holding one candidate per list, pop min, push next from same source. **Problem checklist:** Merge k Sorted Lists, Kth Smallest Element in a Sorted Matrix, Smallest Range Covering Elements from k Lists, Find K Pairs with Smallest Sums.

Pattern 14 — Topological Sort

Use when: ordering with dependency constraints (DAG). **Template:** see Section 13.6 (Kahn's Algorithm). **Problem checklist:** Course Schedule I/II, Alien Dictionary, Minimum Height Trees, Sequence Reconstruction, Parallel Courses.

Pattern 15 — Dynamic Programming

Use when: optimization/counting over overlapping subproblems. **Sub-patterns to master (in priority order):** 1D DP (House Robber, Climbing Stairs) → 0/1 Knapsack (Subset Sum, Partition) → Unbounded Knapsack (Coin Change) → LCS family (Edit Distance, Palindromic Subsequence) → LIS family → Grid DP → Interval DP (MCM) → Tree DP → State Machine DP (Stocks) → Bitmask DP (TSP). **Problem checklist:** see Section 14 in full — every subsection has 2-4 anchor problems.

Pattern 16 — Monotonic Stack/Queue

Use when: "next greater/smaller element," or need max/min of a sliding window efficiently. **Template:** see Section 7.3. **Problem checklist:** Next Greater Element I/II, Daily Temperatures, Largest Rectangle in Histogram, Trapping Rain Water (stack version), Sliding Window Maximum, Remove K Digits, Online Stock Span.

---

18. Top Interview Questions by Pattern

> Every problem below follows the format: **Problem** → **Brute Force** → **Optimized** → **Code** → **Pattern**. These are curated to *not* repeat problems already fully solved in Sections 3-16 (cross-referenced there), so treat Sections 3-16 + this section together as your combined ~150-problem question bank.

18.1 Pattern 1: Two Pointers — More Problems

Q1. Container With Most Water. Given heights of vertical lines, find two lines that, with the x-axis, form a container holding the most water.

- *In simple words:* pick two lines; the water held = shorter line's height × distance between them. Find the pair maximizing this.
- **Brute force:** check every pair of lines → O(n²).
- **Optimized:** two pointers starting at both ends; always move the pointer at the **shorter** line inward (moving the taller one can never increase area, since width shrinks and height is capped by the shorter line anyway). → **O(n) time, O(1) space.**

```java
int maxArea(int[] height) {
    int left = 0, right = height.length - 1, maxArea = 0;
    while (left < right) {
        int area = Math.min(height[left], height[right]) * (right - left);
        maxArea = Math.max(maxArea, area);
        if (height[left] < height[right]) left++; else right--;
    }
    return maxArea;
}
```

Q2. Trapping Rain Water. Given elevation heights, compute total water trapped after rain.

- *In simple words:* water above index i = `min(maxLeft, maxRight) - height[i]`, summed over all i.
- **Brute force:** for each index, scan left and right to find max → O(n²).
- **Better:** precompute `leftMax[]` and `rightMax[]` arrays → O(n) time, O(n) space.
- **Optimized (two pointers, O(1) space):** maintain `leftMax`, `rightMax` on the fly; move the pointer on the side with the smaller max (that side's water level is already determined). → **O(n) time, O(1) space.**

```java
int trap(int[] height) {
    int left = 0, right = height.length - 1, leftMax = 0, rightMax = 0, water = 0;
    while (left < right) {
        if (height[left] < height[right]) {
            leftMax = Math.max(leftMax, height[left]);
            water += leftMax - height[left];
            left++;
        } else {
            rightMax = Math.max(rightMax, height[right]);
            water += rightMax - height[right];
            right--;
        }
    }
    return water;
}
```

Q3. Valid Palindrome (ignore non-alphanumeric, case-insensitive).

- **Optimized:** two pointers from both ends, skip non-alphanumeric chars, compare lowercase versions. → **O(n) time, O(1) space.**

Q4. Remove Duplicates from Sorted Array (in-place).

- **Optimized:** slow pointer marks the position for the next unique element; fast pointer scans ahead. → **O(n) time, O(1) space.**

Q5. 4Sum.

- **Approach:** sort array O(n log n); fix first two elements with nested loops, two-pointer on the remaining → O(n³) total, with duplicate-skipping at each level.

18.2 Pattern 2: Sliding Window — More Problems

Q1. Fruit Into Baskets (longest subarray with at most 2 distinct values).

- *In simple words:* it's "Longest Substring with At Most K Distinct Characters" with k=2 in disguise.
- **Optimized:** sliding window with a frequency map of size ≤ 2; shrink from left when a 3rd distinct type appears. → **O(n) time, O(1) space** (bounded map size).

Q2. Permutation in String (does s1's permutation exist as a substring in s2?)

- **Brute force:** generate all permutations of s1, search each in s2 → factorial time, terrible.
- **Optimized:** fixed-size sliding window of length `len(s1)` over s2; maintain character frequency counts for the window and for s1; compare counts (or use a "matches" counter to avoid O(26) comparison every slide). → **O(n) time, O(1) space** (26-letter alphabet).

Q3. Find All Anagrams in a String — identical sliding window pattern to Q2, but collect **all** starting indices where the window's frequency matches s1's, instead of stopping at the first.

Q4. Longest Repeating Character Replacement (longest substring after replacing at most k characters to make all chars the same).

- **Optimized:** sliding window tracking the count of the most frequent character in the window (`maxFreq`). Window is valid if `windowSize - maxFreq <= k` (i.e., replacements needed ≤ k); shrink from left otherwise. **Trick:** `maxFreq` never needs to *decrease* even as the window shrinks — because a shrinking window that no longer beats the current best simply won't grow the answer, so it's safe to leave `maxFreq` stale. → **O(n) time, O(1) space.**

Q5. Max Consecutive Ones III (longest subarray of 1s after flipping at most k 0s).

- **Optimized:** sliding window tracking count of zeros in the window; shrink from left when zero-count exceeds k. → **O(n) time, O(1) space.** Structurally identical to Longest Repeating Character Replacement.

18.3 Pattern 3: Fast & Slow Pointers — More Problems

Q1. Happy Number (repeatedly replace a number with the sum of squares of its digits; does it reach 1, or loop forever?)

- *In simple words:* this "sequence of transformations" either terminates at 1 or enters a cycle — exactly the shape Floyd's cycle detection was built for.
- **Brute force:** HashSet to detect repeats → O(log n) per step (digit operations), O(unknown) space.
- **Optimized:** slow/fast pointers where "next" = apply the digit-square-sum transformation; if slow == fast and value isn't 1, it's a cycle (not happy). → **O(1) extra space.**

Q2. Find the Duplicate Number (array of n+1 integers, values in [1,n], exactly one duplicate, can't modify array, O(1) space required).

- *In simple words:* treat `nums[i]` as a "pointer" to index `nums[i]` — this creates a linked-list-like structure with a guaranteed cycle (because there's a duplicate, at least two indices point to the same place). Find the cycle's entry point using Floyd's algorithm — that entry point IS the duplicate. → **O(n) time, O(1) space.**

18.4 Pattern 4: Merge Intervals — More Problems

Q1. Insert Interval (insert a new interval into an already-sorted, non-overlapping list, merging as needed).

- **Optimized:** single linear pass — add all intervals ending before the new one starts unchanged; merge all overlapping ones into the new interval; add the rest unchanged. → **O(n) time** (already sorted, no need to re-sort), O(n) space for output.

Q2. Meeting Rooms II (minimum number of meeting rooms required, given intervals).

- **Brute force:** for each meeting, check overlap with all others → O(n²).
- **Optimized (two approaches, both O(n log n)):**
  - *Two sorted arrays:* sort start times and end times separately; two-pointer scan, incrementing a room counter on a start, decrementing on an end whenever `end <= start`; track the max concurrent rooms.
  - *Min-heap of end times:* for each meeting (sorted by start), if the earliest-ending room (heap top) has already ended, reuse it (pop + push); else allocate a new room (just push). Heap size at the end = rooms needed.

Q3. Interval List Intersections (given two lists of sorted, non-overlapping intervals, find their intersection).

- **Optimized:** two-pointer walk through both lists simultaneously; compute overlap of current pair (`max(starts)` to `min(ends)`), advance whichever interval ends first. → **O(n + m) time.**

18.5 Pattern 5: Cyclic Sort — More Problems

Q1. First Missing Positive (find the smallest missing positive integer, O(n) time, O(1) space).

- *In simple words:* the answer must be in range `[1, n+1]` for an array of size n (if all of 1..n are present, answer is n+1).
- **Brute force:** sort then scan → O(n log n), or HashSet → O(n) space.
- **Optimized (Cyclic Sort):** place each value `v` (where `1 <= v <= n`) at index `v-1` via swapping. After this pass, scan for the first index `i` where `nums[i] != i+1` — that's the answer. → **O(n) time, O(1) space** (each element swapped at most once into its correct place, so the swap loop is bounded by O(n) total work).

Q2. Set Mismatch (array has one number replaced by a duplicate; find the duplicate and the missing number).

- **Optimized:** cyclic sort, or the "negative marking" trick from Section 4.8, or a single-pass sum/sum-of-squares math trick. → **O(n) time, O(1) space.**

18.6 Pattern 6: In-Place Linked List Reversal — More Problems

Q1. Swap Nodes in Pairs.

- **Optimized:** iterative pointer rewiring, three nodes at a time (previous pair's tail, current pair's two nodes). → **O(n) time, O(1) space.**

Q2. Rotate List (rotate a linked list to the right by k places).

- **Optimized:** find length, connect tail to head (make it circular temporarily), find the new tail at position `(length - k % length - 1)`, break the circle there. → **O(n) time, O(1) space.**

Q3. Reorder List (L0 → Ln → L1 → Ln-1 → ...).

- **Optimized:** find middle (fast/slow), reverse second half, merge the two halves alternately. → **O(n) time, O(1) space** — a great combination of three sub-patterns (fast/slow + reversal + merge) in one problem.

18.7 Pattern 7: Tree BFS — More Problems

Q1. Binary Tree Zigzag Level Order Traversal.

- **Optimized:** standard level-order BFS, but reverse alternate levels (or use a deque, adding to front/back alternately per level). → **O(n) time, O(n) space.**

Q2. Populating Next Right Pointers in Each Node (connect nodes at the same level).

- **Brute force:** standard BFS with a queue → O(n) time, O(n) space (queue).
- **Optimized (O(1) extra space, using already-established next pointers):** process level by level using the `next` pointers themselves as the traversal mechanism (no queue needed) — leverages the previous level's connections to reach every node in the current level. → **O(n) time, O(1) extra space.**

Q3. Minimum Depth of Binary Tree.

- **Common pitfall:** a node with only one child is NOT a valid "leaf-reaching" base case for min depth (must continue down the non-null side). BFS is actually preferable here over DFS because BFS can **stop early** the moment it finds the first leaf (guaranteed shortest due to level-order property). → **O(n) worst case, but often much faster in practice.**

Q4. Average of Levels in Binary Tree. — direct application of the BFS level-order template, averaging each level's values.

18.8 Pattern 8: Tree DFS — More Problems

Q1. Path Sum II (return all root-to-leaf paths summing to a target).

- **Optimized:** DFS carrying the running path and remaining sum; when a leaf is hit with remaining sum == 0, record the path; backtrack (remove last element) after exploring both children. → **O(n) time** in general, O(n²) if paths must be copied (each path copy costs O(n)).

Q2. Sum Root to Leaf Numbers (each root-to-leaf path represents a number; sum them all).

- **Optimized:** DFS passing down `currentNumber = currentNumber * 10 + node.val`; add to total when a leaf is reached. → **O(n) time, O(h) space.**

Q3. Binary Tree Maximum Path Sum — see Section 11.3 (this is the canonical hard Tree DFS problem; make sure you can code it cold).

Q4. All Paths From Source to Target (DAG version, not tree) — DFS backtracking, recording the path each time the target is reached. → **O(2^V · V)** worst case for a densely connected DAG (exponentially many paths possible).

18.9 Pattern 9: Two Heaps — More Problems

Q1. IPO / Maximize Capital (pick up to k projects to maximize capital, each project has a required capital and a profit; can only pick projects you can currently afford).

- **Optimized:** sort projects by required capital. Use a **min-heap** for capital-to-unlock, and a **max-heap** for profit. At each step, push all currently-affordable projects (capital ≤ current funds) from the sorted list into the max-heap (by profit), then pop the max-profit one and add its profit to current funds. Repeat k times. → **O(n log n) time.**

Q2. Sliding Window Median — see Section 12.3 (two heaps + lazy deletion, an advanced but mechanical extension of Find Median from Data Stream).

18.10 Pattern 10: Subsets / Backtracking — More Problems

Q1. Permutations (all permutations of a distinct-element array).

- **Optimized:** backtrack, swapping elements into the "current position" and swapping back after recursing (avoids needing a separate "used" array). → **O(n! × n) time.**

Q2. Combination Sum II (candidates may have duplicates, each used at most once, no duplicate combinations in output).

- **Key trick:** sort first; when iterating choices at each recursion level, skip a candidate if it equals the previous one **at the same recursion depth** (`i > start && candidates[i] == candidates[i-1]`) — this is THE standard duplicate-avoidance trick for backtracking problems with duplicate inputs. Memorize this exact line.

Q3. Letter Combinations of a Phone Number.

- **Optimized:** backtrack, at each digit trying every letter it maps to, building the string one character at a time. → **O(4^n × n) time** worst case (digits 7 and 9 map to 4 letters).

Q4. Palindrome Partitioning (partition a string so every substring is a palindrome; return all partitions).

- **Optimized:** backtrack trying every prefix starting at the current index; only recurse further if that prefix is a palindrome (checked via precomputed O(n²) `isPalindrome` table, or checked on the fly). → Exponential worst case, pruned by the palindrome check.

18.11 Pattern 11: Modified Binary Search — More Problems

Q1. Median of Two Sorted Arrays (the classic "hard" binary search problem).

- *In simple words:* find a partition point in the smaller array such that combined with a corresponding partition in the larger array, everything on the left side is ≤ everything on the right side, and both sides have equal (or off-by-one) counts.
- **Brute force:** merge both arrays → O(n + m) time, O(n + m) space.
- **Better:** merge without extra array, track only the middle position(s) → O(n + m) time, O(1) space.

- **Optimized:** binary search on the partition index of the **smaller** array (ensures O(log(min(n,m)))); for each candidate partition, compute the corresponding partition in the other array using the fact that total left-side elements = `(n+m+1)/2`; check the boundary condition (`leftMax1 <= rightMin2` and `leftMax2 <= rightMin1`), adjust the binary search range accordingly. → **O(log(min(n, m))) time, O(1) space.**

Q2. Search Insert Position (find index to insert target to keep array sorted) — a direct, un-modified binary search returning the "insertion point" (the `low` pointer at termination) rather than -1 on failure.

Q3. Find K Closest Elements (given a sorted array, find k elements closest to x).

- **Brute force:** compute distance for every element, sort by distance → O(n log n).
- **Optimized:** binary search for the **left boundary** of a window of size k, using a comparison of `x - arr[mid]` vs `arr[mid+k] - x` to decide which side to shrink. → **O(log(n-k)) time.**

18.12 Pattern 12: Top K Elements — More Problems

Q1. Sort Characters By Frequency.

- **Optimized:** count frequencies with a HashMap O(n), then either sort by frequency O(n log n) or bucket-sort by frequency for O(n) (same idea as Top K Frequent Elements' optimal solution in Section 12.2).

Q2. Kth Largest Element in an Array (one-time query, not a stream) — see Section 9.7 (Quickselect gives O(n) average, beating the O(n log k) heap approach when it's a single one-off query rather than a stream).

Q3. Reorganize String (rearrange characters so no two adjacent are the same).

- **Optimized:** count frequencies; if any character's frequency > `(n+1)/2`, it's impossible. Otherwise, greedily place the most frequent remaining character at each position using a max-heap, always alternating between the top two most-frequent characters to avoid adjacency violations. → **O(n log k) time** (k = alphabet size).

18.13 Pattern 13: K-Way Merge — More Problems

Q1. Smallest Range Covering Elements from K Lists.

- **Optimized:** min-heap holding the current element from each of the k lists, plus tracking the current max across the heap. Repeatedly pop the min, update the best range using `(currentMin, currentMax)`, then push the next element from the same list the popped element came from (updating `currentMax` if needed). Stop when any list is exhausted. → **O(n log k) time** (n = total elements across all lists).

Q2. Find K Pairs with Smallest Sums (from two sorted arrays, find k pairs with smallest sum).

- **Brute force:** generate all n×m pairs, sort → O(nm log(nm)).
- **Optimized:** min-heap seeded with pairs `(arr1[i], arr2[0])` for the first `min(k, n)` values of i; each time you pop a pair `(arr1[i], arr2[j])`, push `(arr1[i], arr2[j+1])` if it exists. → **O(k log k) time** (bounded heap operations since we stop after k pops).

18.14 Pattern 14: Topological Sort — More Problems

Q1. Alien Dictionary (given a sorted list of words in an alien language, derive the character ordering).

- **Approach:** compare adjacent words to derive "comes before" edges between the first differing characters; build a graph, run topological sort (Kahn's or DFS). **Watch for edge case:** if a word is a prefix of an earlier word (e.g., "abc" appears after "ab"), the order is invalid. → **O(C) time** where C = total characters across all words (graph has at most 26 nodes so building/topo-sorting it is cheap; dominant cost is comparing words).

Q2. Minimum Height Trees (find node(s) that, if chosen as root, minimize the tree's height).

- *In simple words:* the answer is always the 1 or 2 "most central" nodes.
- **Brute force:** run BFS from every node to compute height → O(V²).

- **Optimized ("peel the onion" / topological-sort-like leaf trimming):** repeatedly remove all current leaf nodes (degree 1) layer by layer, like a BFS from the outside in; the last 1-2 remaining nodes are the answer. → **O(V + E) time.**

18.15 Pattern 15: Dynamic Programming — A Bigger Problem Bank

Q1. Word Break (can a string be segmented into dictionary words?)

- **Brute force:** try every prefix recursively → exponential without memoization.
- **Optimized DP:** `dp[i]` = true if `s[0..i)` can be segmented; `dp[i] = OR over all j<i of (dp[j] && s[j..i) is in dictionary)`; `dp[0] = true`. → **O(n²) time** (n² substring checks, each O(1) with a HashSet of words, or O(n) if checking substring equality naively), **O(n) space.**

Q2. Decode Ways (count ways to decode a digit string into letters, 'A'=1..'Z'=26).

- **Recurrence:** `dp[i] = dp[i-1] (if s[i-1..i) is a valid single-digit decode, i.e., not '0') + dp[i-2] (if s[i-2..i) is a valid two-digit decode, i.e., "10".."26")`. → **O(n) time, O(1) space** (rolling variables).

Q3. Interleaving String (can s3 be formed by interleaving s1 and s2, preserving relative order within each?)

- **State:** `dp[i][j]` = true if `s3[0..i+j)` can be formed from `s1[0..i)` and `s2[0..j)`.
- **Recurrence:** `dp[i][j] = (dp[i-1][j] && s1[i-1]==s3[i+j-1]) || (dp[i][j-1] && s2[j-1]==s3[i+j-1])`. → **O(n·m) time, O(n·m) space** (reducible to O(min(n,m))).

Q4. Maximal Square (largest square of 1s in a binary matrix).

- **Recurrence:** `dp[i][j] = min(dp[i-1][j], dp[i][j-1], dp[i-1][j-1]) + 1` if `matrix[i][j] == 1` (the square ending at this cell is limited by the smallest of its three neighbors' squares — a beautiful "limited by your weakest supporting neighbor" recurrence). Track max `dp[i][j]` seen; answer is `max²`. → **O(m·n) time, O(n) space** (rolling row).

Q5. Regular Expression Matching (support `.` and `*`).

- **State:** `dp[i][j]` = does `s[0..i)` match `p[0..j)`.
- **Recurrence:** handle `*` by considering "zero occurrences of preceding char" (`dp[i][j-2]`) OR "one more occurrence" (`dp[i-1][j]` if the preceding pattern char matches `s[i-1]`). Handle `.` as a wildcard matching any single character. → **O(n·m) time, O(n·m) space** — this is considered one of the hardest "standard" DP problems; practice it specifically if targeting top-tier companies.

Q6. Perfect Squares (minimum number of perfect square numbers summing to n) — structurally identical to unbounded Coin Change, where "coins" = perfect squares ≤ n. → **O(n·√n) time.**

Q7. Egg Drop Problem (minimum trials to find the critical floor with k eggs, n floors).

- **Classic DP:** `dp[eggs][floors]` with a recurrence trying every floor to drop from → O(k·n²), optimizable to O(k·n log n) with binary search on the recurrence's monotonic structure, or reformulated as "given t trials and k eggs, what's the max floors distinguishable" for an O(k log n) solution. Good to at least explain the O(k·n²) baseline and mention the optimization exists.

18.16 Pattern 16: Monotonic Stack — More Problems

Q1. Trapping Rain Water — Stack Version (alternative to the two-pointer solution in 18.1 Q2).

- **Approach:** maintain a decreasing monotonic stack of indices (by height). When a taller bar is found, pop the stack — the popped bar is a "valley bottom"; compute trapped water as `(min(height[left_wall], height[current]) - height[popped]) * width`. → **O(n) time, O(n) space** (worth knowing both the two-pointer O(1)-space version AND this stack-based version, as interviewers sometimes ask for both).

Q2. Remove K Digits (remove k digits to make the smallest possible number).

- **Approach:** monotonic increasing stack of digits; while the current digit is smaller than the stack's top AND removals remain, pop (removing a "bad" larger digit that appears before a smaller one hurts the number's value). → **O(n) time, O(n) space.**

Q3. Online Stock Span (for each day, how many consecutive previous days had price ≤ today's price?)

- **Approach:** monotonic decreasing stack storing `(price, span)` pairs; when the current price ≥ stack top's price, pop and accumulate its span before pushing the current one. → **O(1) amortized per call, O(n) total space.**

Q4. 132 Pattern (does there exist i<j<k with `nums[i] < nums[k] < nums[j]`?)

- **Approach:** traverse right to left, maintaining a monotonic decreasing stack of candidates for "nums[j]," and a running `third` value = the largest number ever popped off the stack (a valid candidate for "nums[k]" that's less than some previous "nums[j]"). If current number < `third`, found the pattern. → **O(n) time, O(n) space** — a genuinely clever, non-obvious use of monotonic stacks worth studying carefully.

---

19. The No-Grind Practice Roadmap

19.1 Philosophy Recap

You do not need 500+ solved problems. You need **deep, re-derivable understanding of ~16 patterns**, reinforced by roughly **120-150 well-chosen problems** (all listed in Sections 4-18 above), plus **timed mock practice** to build interview-speed execution. Quality of understanding beats quantity of problems every single time.

19.2 The 8-Week Plan (Adjust Pace to Your Timeline)

Week 1 — Foundations & Complexity Fluency

- Day 1-2: Read Sections 1-3 (how to use this guide, Big O, math/bit tricks). Practice computing complexity of 10 random code snippets until it's instant.
- Day 3-5: Section 4 (Arrays & Strings) — implement Kadane's, Two Pointers, Sliding Window from scratch, no peeking, 3 times each.
- Day 6-7: Section 5 (Hashing) — Two Sum, Longest Consecutive Sequence, Subarray Sum Equals K. Mock timed session: 3 problems in 45 minutes.

Week 2 — Linked Lists, Stacks, Queues

- Day 1-3: Section 6 — reversal, fast/slow pointers, merge k lists. Draw pointer diagrams by hand before coding (this is the #1 way to avoid null-pointer bugs).
- Day 4-5: Section 7 — monotonic stack/queue problems (Next Greater Element → Daily Temperatures → Largest Rectangle in Histogram, in that order of difficulty).
- Day 6-7: Mixed mock: 1 linked list + 1 stack problem, timed 60 minutes total, explain out loud.

Week 3 — Recursion, Backtracking, Sorting, Binary Search

- Day 1-2: Section 8 — subsets, permutations, N-Queens, combination sum.
- Day 3-4: Section 9 — implement merge sort and quicksort from memory (yes, actually memorize these two).
- Day 5-7: Section 10 — binary search template drilled until automatic; then all rotated-array variants; then "binary search on the answer" problems (Koko, Ship Packages).

Week 4 — Trees & Heaps

- Day 1-3: Section 11 — all traversals, BST validate/insert/delete, LCA (both general and BST versions), diameter, serialize/deserialize.
- Day 4-5: Tries — implement from scratch, then Word Search II (Trie + backtracking combo).
- Day 6-7: Section 12 — Top K pattern, Two Heaps (median), K-way merge.

Week 5 — Graphs

- Day 1-2: BFS/DFS templates, number of islands, connected components.
- Day 3-4: Topological sort (both Kahn's and DFS versions), course schedule variants.
- Day 5-6: Dijkstra, Union-Find, Kruskal's MST.
- Day 7: Mock: one graph problem, 45 minutes, explain time/space trade-offs of BFS vs Dijkstra vs Union-Find for the given problem.

Week 6 — Dynamic Programming Part 1

- Day 1-2: 1D DP (climbing stairs, house robber) → narrate the naive → memo → tabulation → space-optimized progression every time.
- Day 3-4: 0/1 Knapsack family (subset sum, partition equal subset, target sum).
- Day 5-7: Unbounded Knapsack (coin change I/II) + LCS family (edit distance, LCS, palindromic subsequence).

Week 7 — Dynamic Programming Part 2 & Greedy

- Day 1-2: LIS family (including the O(n log n) binary search trick).
- Day 3-4: Grid DP + Interval DP (matrix chain style).
- Day 5: State machine DP (stock problems) — this teaches you to "draw the states" for ANY DP problem.
- Day 6-7: Section 15 Greedy — interval scheduling, jump game, gas station.

Week 8 — Advanced Topics, Mocks, and Polish

- Day 1-2: Section 16 — LRU cache (implement from scratch, no shortcuts), Segment Tree / Fenwick Tree concept + one implementation.
- Day 3-5: Full mock interviews (45-60 min each) covering a random mix of patterns — at least 6 total this week.
- Day 6-7: Review your personal "mistake log" (see 19.4), redo any problem you got wrong under time pressure.

19.3 Daily Practice Structure (Recommended per Problem)

1. **Read the problem. Do NOT look at the solution.** (2-3 min)
2. **State the brute force out loud**, along with its time/space complexity. (2 min)
3. **Ask yourself the pattern-recognition question:** "What's the constraint telling me? Is this sorted? Is it asking for contiguous elements? Is it counting ways vs finding max/min? Does it smell like one of the 16 patterns?" (2-3 min)
4. **Attempt the optimized solution.** Give yourself 15-20 minutes genuinely stuck before looking at hints.
5. **Code it fully**, including edge cases (empty input, single element, all duplicates, negative numbers).
6. **Compare against the reference solution** — note ANY difference in approach, not just bugs.
7. **Log it** if you struggled (see below).

19.4 The Mistake Log (Your Most Valuable Practice Asset)

Keep a simple table (spreadsheet or markdown file) with columns: `Problem | Pattern | What I Got Wrong | Why | Fixed Understanding`. Review this log

weekly. Most candidates who fail interviews repeat the **same category of mistake** (e.g., off-by-one errors in binary search, forgetting to handle empty input, using O(n) space when O(1) was expected) — the log makes these visible so you can consciously correct them.

19.5 How Many Problems Per Pattern Is "Enough"?

Pattern	Minimum to Feel Confident	Signal You've Mastered It
Two Pointers	5	Can solve 3Sum-style problems without referencing a template
Sliding Window	6	Can distinguish fixed vs variable window instantly, code Minimum Window Substring cold
Fast/Slow Pointers	3	Can explain WHY Floyd's algorithm mathematically finds the cycle start
Merge Intervals	4	Immediately think "sort by start/end" for any interval problem
Cyclic Sort	3	Recognize "range [1,n]" as the trigger instantly
LinkedList Reversal	4	Can reverse a sublist or k-group without drawing it out
Tree BFS	4	Comfortable tracking "level size" snapshots
Tree DFS	6	Comfortable with pre/in/post-order AND passing state down/up
Two Heaps	3	Can explain the rebalancing invariant without notes
Subsets/Backtracking	6	Can write the template from memory, adapt constraints per problem
Modified Binary Search	6	Recognize "binary search on the answer" from problem phrasing

Pattern	Minimum to Feel Confident	Signal You've Mastered It
Top K Elements	4	Know when heap beats full sort (k << n)
K-Way Merge	3	Comfortable extending heap-of-candidates idea to N sources
Topological Sort	3	Comfortable with both Kahn's and DFS-postorder-reversed methods
Dynamic Programming	15-20 (across all sub-families)	Can state the recurrence for a NEW problem within 5 minutes
Monotonic Stack	5	Recognize "next greater/smaller" phrasing instantly

Total: ~85-100 problems minimum, ~150 for very high confidence — a tiny fraction of "500+," but with FAR deeper understanding per problem.

19.6 Mock Interview Practice

- Practice **explaining while coding** — silence is the #1 thing that tanks otherwise-correct solutions in real interviews.
- Use a timer. Real interviews give you ~35-45 minutes for one medium/hard problem including explanation, coding, and testing.
- Practice on a **plain text editor or whiteboard app** (no autocomplete) periodically, since many interviews (especially onsite/virtual whiteboard rounds) don't give you IDE features.
- Get a peer or use a mock-interview platform to practice explaining trade-offs to another person — self-practice alone won't build this muscle.

---

20. Complexity Cheat Sheet & Common Mistakes

20.1 Data Structure Operation Complexities (Master Table)

Data Structure	Access	Search	Insert	Delete	Space
Array (unsorted)	O(1)	O(n)	O(n)	O(n)	O(n)
Array (sorted)	O(1)	O(log n)	O(n)	O(n)	O(n)
Dynamic Array (ArrayList)	O(1)	O(n)	O(1) amortized (end), O(n) (middle)	O(n)	O(n)
Singly Linked List	O(n)	O(n)	O(1) (at known node)	O(1) (at known node)	O(n)
Doubly Linked List	O(n)	O(n)	O(1)	O(1)	O(n)
Stack	O(n)	O(n)	O(1)	O(1)	O(n)
Queue	O(n)	O(n)	O(1)	O(1)	O(n)
HashMap/HashSet	N/A	O(1) avg, O(n) worst	O(1) avg	O(1) avg	O(n)
Balanced BST (TreeMap)	O(log n)	O(log n)	O(log n)	O(log n)	O(n)
Binary Heap	O(1) peek	O(n)	O(log n)	O(log n) (root)	O(n)
Trie	O(L)	O(L)	O(L)	O(L)	O(total chars)
Segment Tree	O(log n) query	O(log n)	O(log n) update	O(log n)	O(n)
Fenwick Tree (BIT)	O(log n)	O(log n)	O(log n)	O(log n)	O(n)
Union-Find	N/A	O(α(n))	O(α(n)) union	N/A	O(n)

20.2 Algorithm Complexities (Master Table)

Algorithm	Time	Space
Linear Search	O(n)	O(1)

Algorithm	Time	Space
Binary Search	O(log n)	O(1)
Bubble/Selection/Insertion Sort	O(n²)	O(1)
Merge Sort	O(n log n)	O(n)
Quick Sort	O(n log n) avg, O(n²) worst	O(log n)
Heap Sort	O(n log n)	O(1)
Counting/Radix Sort	O(n + k) / O(d(n+k))	O(n + k)
BFS/DFS (graph)	O(V + E)	O(V)
Dijkstra (heap-based)	O((V+E) log V)	O(V)
Bellman-Ford	O(V·E)	O(V)
Floyd-Warshall	O(V³)	O(V²)
Kruskal's MST	O(E log E)	O(V)
Prim's MST	O(E log V)	O(V)
Topological Sort	O(V + E)	O(V)
0/1 Knapsack DP	O(n·W)	O(n·W) → O(W) optimized
LCS / Edit Distance	O(n·m)	O(n·m) → O(min(n,m)) optimized
LIS (binary search version)	O(n log n)	O(n)
Quickselect (kth element)	O(n) avg	O(1)

20.3 Common Mistakes That Cost Interview Points

1. **Forgetting edge cases:** empty array/string, single element, all elements identical, negative numbers, integer overflow. **Always verbally list these before finishing.**
2. **Off-by-one errors in binary search:** confusing `<` vs `<=` in the while condition, or `mid` vs `mid+1`/`mid-1` in boundary updates. Always trace through a tiny example (size 1, size 2) mentally.

3. **Modifying a collection while iterating over it** (e.g., removing from an ArrayList inside a for-each loop) — causes `ConcurrentModificationException` in Java or skipped elements in other languages. Use an iterator's `remove()` or iterate backwards, or collect indices to remove separately.
4. **Integer overflow** in Java/C++ when computing `mid = (low + high) / 2` (use `low + (high - low) / 2` instead) or when summing large arrays (use `long` accumulator).
5. **Confusing time complexity of string concatenation** — repeated `+` on Strings in a loop is O(n²) in Java; always use `StringBuilder`.
6. **Assuming HashMap iteration order is deterministic** — it isn't (use `LinkedHashMap` if you need insertion order, `TreeMap` for sorted order).
7. **Forgetting to handle recursion base cases properly**, leading to infinite recursion or missing an edge case (e.g., forgetting `null` checks in tree recursion).
8. **Not discussing space complexity** — many candidates only state time complexity; interviewers specifically probe for space trade-offs.
9. **Jumping straight to code without explaining approach** — even a correct solution loses points if the interviewer can't follow your reasoning.
10. **Not testing the code with a manual walkthrough** after writing it — always trace through at least one example by hand before declaring "I'm done."
11. **Using recursion for very deep inputs** without considering stack overflow risk — mention "I could convert this to an iterative approach with an explicit stack if recursion depth is a concern."
12. **Misjudging when greedy is safe** — proposing a greedy solution without any justification, when the interviewer expected a DP proof-driven approach (or vice versa, overcomplicating with DP when greedy provably works).

20.4 How to Estimate Complexity From Constraints (Quick Reference)

Constraint on n	Expected complexity
n ≤ 10-12	O(n!) or O(2^n · n) — brute force permutations/subsets fine
n ≤ 20-25	O(2^n) — bitmask DP fine
n ≤ 500	O(n³) fine
n ≤ 5,000	O(n²) fine

Constraint on n	Expected complexity
n ≤ 10^5 - 10^6	O(n log n) expected
n ≤ 10^8	O(n) expected, tight
n > 10^8	O(log n) or O(1) expected, or the problem uses a formula/math trick

21. Interview Day Tips & Communication Templates

21.1 The Ideal Interview Flow (Timing Guide for a 45-Minute Slot)

1. **Clarify the problem (3-5 min).** Restate it in your own words. Ask about constraints (input size, data types, can input be empty/negative, are there duplicates, is the array sorted). This alone signals seniority.
2. **Discuss approach out loud before coding (5-8 min).** State brute force + complexity, then propose the optimization and why it works. Get a nod from the interviewer before diving into code.
3. **Code (15-20 min).** Write clean, modular code. Narrate as you go ("I'll use a HashMap here to get O(1) lookups instead of scanning").
4. **Test your code (5-8 min).** Walk through the example given, then a self-constructed edge case (empty, single element, duplicates).
5. **Discuss complexity and potential follow-ups (2-5 min).** State final time/space complexity. Be ready for "can you optimize further?" or "what if the input doesn't fit in memory?"

21.2 Useful Verbal Templates

- **When starting:** *"Let me first make sure I understand the problem. We're given [X], and we need to return [Y], is that right? A few clarifying questions: [constraints]."*
- **When proposing brute force:** *"The straightforward approach would be to [X], which takes O(...) time because [reason]. I think we can do better."*

- **When identifying a pattern:** *"This looks like a sliding window problem because we need a contiguous subarray/substring satisfying a condition."*
- **When stuck:** *"Let me think about what information I'm recomputing repeatedly — is there a data structure that could cache that?"* (This often leads you back to hashing/DP on your own, and shows the interviewer your problem-solving process even mid-struggle.)
- **When finished:** *"This solution runs in O(n) time and O(1) extra space. Let me trace through the example to verify... [walk through]. I don't see any bugs — do you want me to consider any additional edge cases or discuss alternative approaches?"*

21.3 What Interviewers Are Silently Evaluating

- Do you ask clarifying questions, or do you assume and potentially solve the wrong problem?
- Do you write **working**, not just "roughly correct," code? (Compile-level correctness matters — watch bracket matching, variable scoping.)
- Do you use meaningful variable names (`left`, `right`, `windowSum` — not `x`, `y`, `z` for everything)?
- Do you proactively mention complexity without being asked?
- How do you react to hints? (Gracefully incorporating a hint is FAR better than getting defensive or shutting down.)
- Do you discuss trade-offs unprompted (e.g., "we could trade space for time here by caching...")?

21.4 Handling the "I Don't Know This Problem" Moment

Never freeze silently. Instead:

1. Restate the problem and constraints to buy thinking time and confirm understanding.
2. Work through a small example **by hand** on paper/whiteboard — patterns often emerge visually that don't emerge from just reading the statement.
3. State the brute force, even if you suspect it's not what they want — it's a starting point and shows you're not stuck.

4. Explicitly ask yourself out loud: "Is this an array/string/tree/graph/DP-shaped problem?" and connect it to the closest pattern from Section 17.
5. If truly stuck, ask a guiding question: *"Would a hint about the expected time complexity help me narrow down the approach?"* — most interviewers are happy to nudge you; this is normal and doesn't disqualify you.

21.5 System Design Adjacent Note

Pure DSA interviews are usually distinct from System Design interviews, but senior candidates are sometimes asked to extend a DSA answer with a real-world lens (e.g., "how would this scale if the array had 10 billion elements and didn't fit in memory?" → external sorting, streaming algorithms, distributed hashing). Being aware that such follow-ups exist (even without deep system design prep) helps you respond calmly rather than being caught off guard.

---

22. Final Words & Resources

22.1 Summary

This guide covered, from the ground up:

- **Complexity analysis** (Big O/Θ/Ω, amortized analysis, Master Theorem).
- **Every core data structure**: arrays, strings, linked lists, stacks, queues, hash maps, trees (binary, BST, balanced, tries), heaps, graphs, and advanced structures (segment trees, Fenwick trees, union-find, LRU/LFU caches).
- **Every core algorithm family**: sorting (comparison-based and linear-time), searching (binary search and its many disguises), recursion/backtracking, greedy, and dynamic programming across all its major sub-patterns (knapsack, LCS/LIS, interval DP, tree DP, bitmask DP, digit DP, state machine DP).
- **16 master interview patterns** that cover the overwhelming majority of interview questions asked in practice, each with a reusable template.
- **~150 anchor problems**, each explained as: problem in plain words → brute force with complexity → optimized approach with complexity → working code.
- **A realistic, non-500-problem practice roadmap**, a mistake-tracking system, and interview-day communication templates.

22.2 The Single Most Important Habit to Build

Before writing any code, always articulate: (1) what is the brute force, (2) why is it slow, (3) what redundant work can be eliminated, and (4) which data structure or pattern eliminates that redundant work. This four-step habit, repeated across ~100-150 problems, is what actually builds durable interview skill — far more than the raw count of problems solved.

22.3 Suggested Practice Platforms (Names Only, for Reference)

- LeetCode (pattern-tagged problem sets, mock interview timer feature).
- NeetCode's curated pattern-based problem lists (a well-known free curation aligned with this same "patterns over volume" philosophy).
- Pramp / interviewing.io style peer mock interview platforms for live practice.

22.4 A Note on Confidence

Every experienced engineer has, at some point, blanked on a "simple" problem in an interview. What separates people who pass consistently isn't perfect recall of solutions — it's the **process**: staying calm, talking through the brute force, systematically looking for the pattern, and communicating clearly even when unsure. Trust the process in this guide, put in the focused (not just voluminous) practice, and you will be well prepared.

The guide continues below with bonus advanced material: additional graph algorithms, advanced string algorithms, combinatorics, worked numeric dry-runs of key algorithms, design-question patterns, and a language cheat sheet — all still relevant to interview prep, especially at senior levels or product-based companies that go beyond the "standard" 16 patterns.

---

23. Additional Graph Algorithms (Beyond the Basics)

23.1 Strongly Connected Components (SCC)

A **Strongly Connected Component** is a maximal set of vertices in a **directed** graph where every vertex can reach every other vertex in the set. Used in problems about mutual reachability, compiler dependency analysis, and social network "clique-like" reachability groups.

Kosaraju's Algorithm (two-pass DFS):

1. Do a DFS on the original graph, pushing nodes onto a stack in **postorder** (finish time order).
2. Reverse all edges of the graph (transpose graph).
3. Pop nodes from the stack one at a time; for each unvisited node, DFS on the **transposed** graph — each such DFS tree is exactly one SCC.

Why this works (intuition): a node finishing DFS last (highest postorder) in the original graph is guaranteed to be in a "source-like" SCC when processed on the transposed graph, so processing in decreasing finish-time order correctly isolates each SCC without cross-contamination.

```
void kosaraju(int n, List<List<Integer>> adj, List<List<Integer>> reverseAdj) {
    boolean[] visited = new boolean[n];
    Deque<Integer> finishOrder = new ArrayDeque<>();
    for (int i = 0; i < n; i++) if (!visited[i]) fillOrder(i, adj, visited,
finishOrder);

    Arrays.fill(visited, false);
    List<List<Integer>> sccs = new ArrayList<>();
    while (!finishOrder.isEmpty()) {
        int node = finishOrder.pop();
        if (!visited[node]) {
            List<Integer> component = new ArrayList<>();
            dfsCollect(node, reverseAdj, visited, component);
            sccs.add(component);
        }
    }
}
void fillOrder(int node, List<List<Integer>> adj, boolean[] visited, Deque<Integer>
stack) {
    visited[node] = true;
    for (int next : adj.get(node)) if (!visited[next]) fillOrder(next, adj,
visited, stack);
    stack.push(node); // postorder
}
void dfsCollect(int node, List<List<Integer>> adj, boolean[] visited, List<Integer>
component) {
    visited[node] = true;
    component.add(node);
    for (int next : adj.get(node)) if (!visited[next]) dfsCollect(next, adj,
visited, component);
}
```

Complexity: O(V + E) — three linear passes (original DFS, transpose, transposed DFS).

Tarjan's Algorithm (single-pass, using low-link values): assigns each node a discovery time and a "low-link" value (the smallest discovery time reachable from that node's subtree, including back edges). A node is the root of an SCC if its low-link equals its own discovery time. Also **O(V + E)**, single DFS pass, but more intricate to implement correctly than Kosaraju's — knowing Kosaraju's well is usually sufficient for interviews; mention Tarjan's by name as the single-pass alternative.

23.2 Articulation Points & Bridges

An **articulation point** (cut vertex) is a node whose removal disconnects the graph. A **bridge** is an edge whose removal disconnects the graph. Both use the same low-link DFS technique as Tarjan's SCC algorithm.

Core idea: during DFS, track `disc[node]` (discovery time) and `low[node]` (lowest discovery time reachable via back edges from the subtree rooted at node).

- **Bridge condition:** edge `(u, v)` (tree edge, v is child of u) is a bridge if `low[v] > disc[u]` — meaning v's subtree has no back edge reaching u or higher.
- **Articulation point condition:** node `u` (non-root) is an articulation point if it has a child `v` with `low[v] >= disc[u]`. The root is an articulation point if it has more than one DFS child.

Complexity: O(V + E). Used in network reliability analysis (which server/link is a single point of failure), and appears as "Critical Connections in a Network" on LeetCode.

23.3 Eulerian Path & Circuit

An **Eulerian path** visits every **edge** exactly once (as opposed to Hamiltonian path, which visits every **vertex** exactly once — Hamiltonian is NP-hard, Eulerian is efficiently solvable).

Conditions for existence (undirected graph):

- Eulerian **circuit** (starts and ends at the same vertex) exists iff every vertex has even degree, and the graph is connected (ignoring isolated vertices).
- Eulerian **path** (may start/end at different vertices) exists iff exactly 0 or 2 vertices have odd degree.

Hierholzer's Algorithm constructs the actual Eulerian path/circuit in **O(E) time**: repeatedly follow unused edges until stuck (forming a cycle back to start or a dead end), then splice in additional cycles found from any vertex along the current path that still has unused edges.

Problem: Reconstruct Itinerary (given tickets as directed edges, find the itinerary using every ticket exactly once, lexicographically smallest if multiple exist) — directly an Eulerian path problem on a directed multigraph, solved via Hierholzer's with a priority-queue/sorted adjacency list for lexicographic ordering.

23.4 Network Flow (Concept-Level)

Max-Flow Min-Cut Theorem: the maximum flow from a source to a sink in a flow network equals the minimum capacity of edges that, if removed, disconnect source from sink.

Ford-Fulkerson / Edmonds-Karp: repeatedly find an **augmenting path** (a path from source to sink with available capacity) using BFS (Edmonds-Karp variant, guarantees $O(V \cdot E^2)$), and push flow along it, updating a **residual graph** (which also allows "undoing" flow via reverse edges) until no augmenting path remains.

This is a rare-but-possible advanced interview topic (more common in competitive programming or at companies with strong algorithmic focus). Knowing the *concept* (max-flow = min-cut, augmenting paths, residual graphs) is usually sufficient; full implementation is rarely expected live unless explicitly targeting such companies.

23.5 Graph Coloring & Bipartiteness (Extended)

M-Coloring Problem (can a graph be colored with M colors such that no two adjacent vertices share a color?) — solved via backtracking, trying each color for each vertex, checking adjacent vertices' colors, backtracking on conflict. NP-hard in general (exponential), but fine for small graphs in interviews.

2-coloring = Bipartiteness check (Section 13.10) is the special, efficiently-solvable case (M=2).

---

24. Advanced String Algorithms (Full Implementations)

24.1 KMP (Knuth-Morris-Pratt) Pattern Matching — Full Detail

Goal: find all occurrences of pattern `p` (length m) in text `s` (length n) in **O(n + m)** time, instead of the brute force **O(n·m)**.

Step 1 — Build the "LPS" (Longest Proper Prefix which is also Suffix) array for the pattern. `lps[i]` = length of the longest proper prefix of `p[0..i]` that is also a suffix of `p[0..i]`.

```
int[] buildLPS(String pattern) {
    int m = pattern.length();
    int[] lps = new int[m];
    int len = 0, i = 1;
    while (i < m) {
        if (pattern.charAt(i) == pattern.charAt(len)) {
            lps[i++] = ++len;
        } else if (len != 0) {
            len = lps[len - 1]; // fall back without moving i (key insight!)
        } else {
            lps[i++] = 0;
        }
    }
    return lps;
}
```

Step 2 — Use the LPS array to skip re-comparisons during matching:

```
List<Integer> kmpSearch(String text, String pattern) {
    List<Integer> matches = new ArrayList<>();
    int[] lps = buildLPS(pattern);
    int i = 0, j = 0; // i -> text, j -> pattern
    while (i < text.length()) {
        if (text.charAt(i) == pattern.charAt(j)) { i++; j++; }
        if (j == pattern.length()) { matches.add(i - j); j = lps[j - 1]; }
        else if (i < text.length() && text.charAt(i) != pattern.charAt(j)) {
            if (j != 0) j = lps[j - 1]; // don't re-check matched prefix, jump
using LPS
            else i++;
        }
    }
```

```
    return matches;
}
```

Why O(n+m): `i` (text pointer) never moves backward — it only ever increases. `j` (pattern pointer) can decrease, but only via `lps[j-1]`, and the total decrease across the whole algorithm is bounded by the total increase, so amortized total work is O(n+m).

24.2 Z-Algorithm (Alternative to KMP, Often Simpler to Reason About)

The **Z-array** for a string `s`: `Z[i]` = length of the longest substring starting at `i` that matches a prefix of `s`. Useful for pattern matching by constructing `pattern + "#" + text` and scanning the Z-array for values equal to `pattern.length()`.

```
int[] zArray(String s) {
    int n = s.length();
    int[] z = new int[n];
    int l = 0, r = 0;
    for (int i = 1; i < n; i++) {
        if (i < r) z[i] = Math.min(r - i, z[i - l]);
        while (i + z[i] < n && s.charAt(z[i]) == s.charAt(i + z[i])) z[i]++;
        if (i + z[i] > r) { l = i; r = i + z[i]; }
    }
    return z;
}
```

Complexity: O(n). Same "window [l, r] reused across iterations" idea as the sliding window pattern — avoids recomputation by reusing previously computed match info whenever the current index falls inside the last known match window.

24.3 Rabin-Karp (Rolling Hash) Pattern Matching

Idea: compute a hash of the pattern and of every window of the same length in the text; only do a full character comparison when hashes match (to rule out hash collisions). The key trick is computing each window's hash in **O(1)** from the previous window's hash (a "rolling hash"), rather than recomputing from scratch.

```
void rabinKarp(String text, String pattern) {
    int n = text.length(), m = pattern.length();
    long base = 256, mod = 1_000_000_007L;
    long patternHash = 0, windowHash = 0, h = 1;
    for (int i = 0; i < m - 1; i++) h = (h * base) % mod; // base^(m-1) % mod

    for (int i = 0; i < m; i++) {
        patternHash = (patternHash * base + pattern.charAt(i)) % mod;
        windowHash = (windowHash * base + text.charAt(i)) % mod;
    }
    for (int i = 0; i <= n - m; i++) {
        if (patternHash == windowHash && text.substring(i, i + m).equals(pattern))
{
            System.out.println("Match at index " + i);
        }
        if (i < n - m) {
            windowHash = (base * (windowHash - text.charAt(i) * h) + text.charAt(i
+ m)) % mod;
            if (windowHash < 0) windowHash += mod; // handle negative modulo
        }
    }
}
```

Complexity: O(n + m) average case (assuming few hash collisions), **O(n·m) worst case** (adversarial input causing many collisions — mitigated with a good random base/modulus or double hashing). Rolling hashes are also directly useful for problems like "longest duplicate substring" (combined with binary search on length) and "repeated DNA sequences."

24.4 Manacher's Algorithm (O(n) Longest Palindromic Substring)

Beats the O(n²) expand-around-center approach (Section 4.12) by reusing previously computed palindrome radius information, similar in spirit to the Z-algorithm's "reuse the window" trick.

Core idea: transform the string by inserting separators (e.g., `"aba"` → `"^#a#b#a#$"`) to handle even/odd length palindromes uniformly. Maintain a "center" and "right boundary" of the rightmost-reaching palindrome found so far; for each new position, use its mirror position's known radius as a starting guess (only expanding further if needed) instead of starting from radius 0.

Complexity: O(n) time, O(n) space. This is a genuinely advanced algorithm — most interviews accept the O(n²) expand-around-center solution, but mentioning "there's an

O(n) Manacher's algorithm for this, though it's rarely required" demonstrates depth.

24.5 Suffix Arrays & Suffix Trees (Concept-Level)

A **suffix array** is a sorted array of all suffixes of a string (represented by starting indices). Enables O(log n) substring search (via binary search on the sorted suffixes) and is foundational for problems like "longest repeated substring," "longest common substring across many strings," and full-text search indexes (used in bioinformatics for genome search).

Building a suffix array naively is O(n² log n) (sort n suffixes, each comparison O(n)); the **O(n log n)** prefix-doubling technique (sorting suffixes by progressively longer prefixes, reusing previous round's rankings) or the **O(n)** DC3/SA-IS algorithms are the practical approaches — implementation specifics are rarely asked live, but the *existence and use-case* of suffix arrays/trees is a good thing to mention in a senior-level string-processing discussion.

---

25. Combinatorics, Probability, and Math for Interviews

25.1 Permutations & Combinations Formulas

- **Permutations** (order matters): `P(n, r) = n! / (n - r)!`
- **Combinations** (order doesn't matter): `C(n, r) = n! / (r! (n - r)!)`
- **Pascal's Triangle identity:** `C(n, r) = C(n-1, r-1) + C(n-1, r)` — directly gives an O(n²) DP table for computing all `C(n, r)` values without factorials (avoids overflow for large n too).

```
long[][] buildPascalsTriangle(int n) {
    long[][] C = new long[n + 1][n + 1];
    for (int i = 0; i <= n; i++) {
        C[i][0] = 1;
        for (int j = 1; j <= i; j++) {
            C[i][j] = C[i - 1][j - 1] + (j <= i - 1 ? C[i - 1][j] : 0);
        }
    }
```

```
    return C;
}
```

25.2 Catalan Numbers

The nth Catalan number `C_n = C(2n, n) / (n + 1)` counts an enormous number of combinatorial structures that show up repeatedly in interviews:

- Number of valid balanced parenthesizations of n pairs (Generate Parentheses, Section 8.3).
- Number of distinct Binary Search Trees with n nodes (Unique Binary Search Trees).
- Number of ways to triangulate a convex polygon with n+2 sides.
- Number of ways to fully parenthesize a matrix chain product (Section 14.11).

Recurrence (useful for DP formulation of "Unique BSTs"): `C_n = sum over i=0..n-1 of (C_i * C_(n-1-i))`, `C_0 = 1`. This recurrence directly gives the DP solution for "Unique Binary Search Trees": `dp[n] = sum over i=0..n-1 of dp[i] * dp[n-1-i]` (choosing each value as root, left subtree from i smaller values, right subtree from n-1-i larger values). → **O(n²) time, O(n) space.**

25.3 Reservoir Sampling

Problem: Randomly select k items from a stream of unknown/large length, each with equal probability, using O(k) space.

- **Approach:** keep the first k items in a "reservoir." For each subsequent item at index `i` (0-indexed, i ≥ k), generate a random index `j` in `[0, i]`; if `j < k`, replace `reservoir[j]` with the new item. → **O(n) time, O(k) space**, and a clean probability proof shows every item has exactly `k/n` probability of ending up in the final reservoir.

25.4 Random Pick with Weight

Problem: given weights for n items, pick a random index with probability proportional to its weight.

- **Approach:** build a prefix-sum array of weights (Section 4.3's prefix sum trick again!); generate a random number in `[1, totalWeight]`; binary search the prefix-sum array for the first value ≥ the random number. → **O(log n) time per pick, O(n) preprocessing.**

25.5 Shuffle an Array (Fisher-Yates Algorithm)

Problem: implement a truly uniform random shuffle.

- **Approach:** iterate from the last index to the first; at each index `i`, swap `arr[i]` with `arr[random(0, i)]`. → **O(n) time, O(1) extra space**, and provably uniform (every permutation is equally likely) — a common trap is swapping with a fully random index in `[0, n)` every time, which does NOT produce a uniform shuffle.

25.6 Probability "Gotcha" Interview Questions (Concept-Level)

- **Birthday Paradox:** with just 23 people, there's a >50% chance two share a birthday — illustrates that collision probability in hashing grows faster than intuition suggests (relevant when discussing hash collision likelihood).
- **Monty Hall Problem:** switching doors doubles your win probability (1/3 → 2/3) — a classic "explain a counterintuitive probability result" interview question, more common in quant/data-science-adjacent interviews than pure SWE ones.

---

26. Data Structure Design Questions (Object-Oriented + DSA Combined)

These questions test whether you can combine multiple data structures thoughtfully to satisfy several operations efficiently at once — extremely common at product companies.

26.1 Design Twitter (Simplified Feed System)

Requirements: post a tweet, follow/unfollow a user, get the 10 most recent tweets in a user's feed (own + followed users), most recent first.

- **Approach:** each user has a HashSet of followees, and a list of `(timestamp, tweetId)`. To get the feed, use the **K-Way Merge** pattern (Section 12.4/18.13) — a max-heap merging the most recent tweets across the user and all followees, popping the top 10. → **O(k log f) per feed request** (f = number of followees, k = 10).

26.2 Design Hit Counter

Requirements: record a hit with a timestamp; return the number of hits in the past 300 seconds.

- **Approach:** a queue of timestamps; on each `getHits` call, pop timestamps older than `(current - 300)` from the front before returning the queue size. → **O(1) amortized per hit, O(n) worst case per query** (bounded by 300-second window size in practice). For high-throughput systems, a **circular buffer of 300 buckets** (one per second, storing a count) gives true **O(1)** per operation.

26.3 Design Rate Limiter (Sliding Window Counter)

Requirements: allow at most N requests per rolling time window per client.

- **Approaches (in order of sophistication):**
  - *Fixed window counter:* simplest, but allows burst traffic at window boundaries (up to 2N requests near the edge).
  - *Sliding window log:* store exact timestamps (like Hit Counter above) — accurate but O(n) space per client.
  - *Sliding window counter (approximation):* weight the previous window's count by the overlap fraction with the current window — O(1) space, good approximation, and this exact scheme is what most production rate limiters (e.g., API gateways) use.
  - *Token bucket:* tokens refill at a fixed rate up to a cap; each request consumes a token; naturally allows controlled bursts. This is the industry-standard algorithm for API rate limiting.

26.4 Design a Parking Lot (Classic OOD + DSA)

Requirements: park/unpark vehicles of different sizes (motorcycle, car, bus) into appropriately-sized spots, find the nearest available spot.

- **Approach:** maintain separate collections (e.g., TreeSet or min-heap of available spot IDs) per spot-size category, so "find the nearest/lowest-numbered available spot" is O(log n); a HashMap tracks `vehicle → spot` for O(1) unpark lookups.

26.5 Design Snake Game

Requirements: move a snake on a grid, grow when eating food, detect collision with itself or walls.

- **Approach:** a **Deque** represents the snake's body (head at front, tail at back) for O(1) head-insertion and tail-removal; a **HashSet** of body cell coordinates for O(1) self-collision checks (must be kept in sync with the deque on every move).

26.6 Design Tic-Tac-Toe (Optimized Win-Check)

Requirements: support `move(row, col, player)`, return the winner if the move creates one, efficiently (not re-scanning the whole board every move).

- **Approach:** maintain running counts per row, per column, and both diagonals (`+1` for player 1's mark, `-1` for player 2's mark, in an int array/variable); after each move, check only the affected row/column/diagonal count for `== n` or `== -n`. → **O(1) per move**, instead of O(n) or O(n²) re-scanning.

26.7 Design a File System / Trie-based Autocomplete

Requirements: insert file paths, list contents of a directory, or provide search-as-you-type suggestions.

- **Approach:** a **Trie** where each node represents a path segment (or character, for autocomplete); traversing down the trie following the query's segments/characters, then DFS from that node to collect all matching results. → **O(L)** for insert/lookup (L = path length or query length), independent of how many total entries exist.

26.8 Design an LFU/LRU-backed Key-Value Store with TTL

Requirements: combine LRU eviction (Section 16.3) with time-to-live expiration.

- **Approach:** augment the LRU's doubly linked list nodes with an `expiryTime` field; lazily check expiry on `get` (return null/miss if expired, and evict then), and optionally run a background sweep (or a min-heap ordered by expiry time) for proactive cleanup. → **O(1) amortized per operation**, with the min-heap approach adding O(log n) for proactive expiry sweeps.

---

27. Worked Numeric Dry-Runs of Key Algorithms

> Reading pseudocode is not the same as *tracing* it. Below are hand-traced examples for the algorithms most commonly botched under interview pressure due to subtle off-by-one or state-tracking errors.

27.1 Dry-Run: Binary Search

Array: `[1, 3, 5, 7, 9, 11]`, target = `7`.

Step	low	high	mid	arr[mid]	Action
1	0	5	2	5	5 < 7 → low = mid+1 = 3
2	3	5	4	9	9 > 7 → high = mid-1 = 3
3	3	3	3	7	Found! return 3

Notice `low <= high` is the loop condition — if it were `low < high`, the case `low == high == 3` would never execute and we'd incorrectly return "not found."

27.2 Dry-Run: Sliding Window (Longest Substring Without Repeating Characters)

String: `"abcabcbb"`.

right	char	Action	left	window	maxLen
0	a	new, add	0	"a"	1
1	b	new, add	0	"ab"	2
2	c	new, add	0	"abc"	3
3	a	dup (last seen at 0 ≥ left) → left = 1	1	"bca"	3
4	b	dup (last seen at 1 ≥ left) → left = 2	2	"cab"	3
5	c	dup (last seen at 2 ≥ left) → left = 3	3	"abc"	3
6	b	dup (last seen at 4 ≥ left) → left = 5	5	"cb"	3
7	b	dup (last seen at 6 ≥ left) → left = 7	7	"b"	3

Final answer: **3** (from "abc").

27.3 Dry-Run: Kadane's Algorithm

Array: `[-2, 1, -3, 4, -1, 2, 1, -5, 4]`.

i	nums[i]	currentSum = max(nums[i], currentSum+nums[i])	maxSum
0	-2	-2	-2
1	1	max(1, -2+1=-1) = 1	1
2	-3	max(-3, 1-3=-2) = -2	1
3	4	max(4, -2+4=2) = 4	4
4	-1	max(-1, 4-1=3) = 3	4
5	2	max(2, 3+2=5) = 5	5
6	1	max(1, 5+1=6) = 6	6
7	-5	max(-5, 6-5=1) = 1	6
8	4	max(4, 1+4=5) = 5	6

Final answer: **6** (subarray `[4, -1, 2, 1]`).

27.4 Dry-Run: BFS Shortest Path (Unweighted Graph)

Graph edges: `0-1, 0-2, 1-3, 2-3, 3-4`. Start = 0.

Queue after processing	Distances so far
Start: [0]	dist[0]=0
Process 0 → enqueue 1, 2	dist[1]=1, dist[2]=1
Process 1 → enqueue 3	dist[3]=2
Process 2 → 3 already visited, skip	(no change)
Process 3 → enqueue 4	dist[4]=3
Process 4 → no unvisited neighbors	done

Final: `dist = [0, 1, 1, 2, 3]`. Notice node 3 is reached via node 1 first (distance 2) and the attempt via node 2 is correctly ignored since it's already visited — this is exactly why marking visited at **enqueue time** (Section 13.2) matters, otherwise node 3 might be enqueued twice.

27.5 Dry-Run: 0/1 Knapsack DP Table

Items: weight=[1,3,4,5], value=[1,4,5,7], capacity=7.

`dp[i][w]` table (rows = items considered 0..4, columns = capacity 0..7):

i \ w	0	1	2	3	4	5	6	7
0 (none)	0	0	0	0	0	0	0	0
1 (w=1,v=1)	0	1	1	1	1	1	1	1
2 (w=3,v=4)	0	1	1	4	5	5	5	5
3 (w=4,v=5)	0	1	1	4	5	6	6	9
4 (w=5,v=7)	0	1	1	4	5	7	8	9

Final answer: `dp[4][7] = 9` (achieved by taking items with weight 3 and weight 4, total value 4+5=9, total weight 7 — fits exactly).

27.6 Dry-Run: Monotonic Stack (Next Greater Element)

Array: `[2, 1, 2, 4, 3]`.

i	nums[i]	Stack before	Action	Stack after	result so far
0	2	[]	push 0	[0]	[-,-,-,-,-]
1	1	[0]	nums[0]=2 not < 1, push 1	[0,1]	unchanged
2	2	[0,1]	nums[1]=1 < 2, pop 1 → result[1]=2; nums[0]=2 not < 2, push 2	[0,2]	[-,2,-,-,-]
3	4	[0,2]	nums[2]=2<4, pop 2→result[2]=4; nums[0]=2<4, pop 0→result[0]=4; push 3	[3]	[4,2,4,-,-]
4	3	[3]	nums[3]=4 not < 3, push 4	[3,4]	[4,2,4,-,-]

End of array, remaining stack entries (indices 3, 4) get `-1` (no greater element to their right). Final: `[4, 2, 4, -1, -1]`.

---

28. Language Cheat Sheet (Java / Python / C++ Quick Reference)

28.1 Java Collections Cheat Sheet

Need	Use	Key methods	Complexity
Dynamic array	`ArrayList<T>`	`add`, `get`, `remove`	O(1) amortized add/get, O(n) remove
Fast lookup, no order	`HashMap<K,V>` / `HashSet<T>`	`put/get`, `add/contains`	O(1) avg
Sorted map/set	`TreeMap<K,V>` / `TreeSet<T>`	`firstKey`, `floor`, `ceiling`	O(log n)
Insertion-order map	`LinkedHashMap<K,V>`	same as HashMap + ordering	O(1) avg

Need	Use	Key methods	Complexity
Stack	`Deque<T>` (use `ArrayDeque`, NOT `Stack` class)	`push`, `pop`, `peek`	O(1)
Queue	`Deque<T>` or `LinkedList<T>`	`offer`, `poll`, `peek`	O(1)
Priority Queue (min-heap default)	`PriorityQueue<T>`	`offer`, `poll`, `peek`	O(log n) insert/remove, O(1) peek
Double-ended queue	`ArrayDeque<T>`	`offerFirst/Last`, `pollFirst/Last`	O(1)
Immutable list/set/map	`List.of(...)`, `Set.of(...)`, `Map.of(...)`	—	—

Common gotchas in Java:

- Never use the legacy `Stack` class (it's synchronized/slow, extends `Vector`) — use `ArrayDeque` as a stack instead.
- `PriorityQueue` is a min-heap by default; for a max-heap use `new PriorityQueue<>(Collections.reverseOrder())` or a custom comparator.
- Autoboxing: comparing `Integer` objects with `==` compares references, not values, above the cached range (-128 to 127) — always use `.equals()` or unbox to `int` first.
- `String` is immutable — use `StringBuilder` for repeated concatenation (Section 4.2).
- Arrays don't have a built-in `.equals` deep comparison — use `Arrays.equals()` (1D) or `Arrays.deepEquals()` (nested).
- 2D array default fill is `0`/`false`/`null` — use `Arrays.fill` per row (or a loop) if you need a different default like `Integer.MAX_VALUE`.

28.2 Python Equivalents

Need	Python structure	Key methods
Dynamic array	`list`	`append`, `pop`, slicing

Need	Python structure	Key methods
Hash map / set	`dict` / `set`	O(1) avg for lookups
Ordered dict (insertion order, Python 3.7+)	`dict` (already ordered by default)	—
Stack	`list` (`append`/`pop`)	O(1)
Queue	`collections.deque`	`append`/`popleft` — O(1) both ends (a plain `list.pop(0)` is O(n)!)
Priority queue (min-heap)	`heapq` module	`heappush`, `heappop` (for max-heap, negate values)
Sorted container	`sortedcontainers.SortedList` (3rd party)	O(log n) insert/search
Counting	`collections.Counter`	frequency map with extra helpers like `most_common()`
Default values	`collections.defaultdict`	avoids manual key-existence checks

Common gotchas in Python:

- `list.pop(0)` and `list.insert(0, x)` are **O(n)**, not O(1) — use `collections.deque` for queue-like behavior.
- Mutable default arguments (`def f(x, cache={})`) persist across calls — a classic bug source.
- Integer division: use `//` for floor division; `/` always returns a float.
- Recursion limit is ~1000 by default (`sys.setrecursionlimit` can raise it, but deep recursion is still a real risk for large inputs — prefer iterative solutions for very deep recursion).

28.3 C++ STL Cheat Sheet

Need	C++ structure	Key methods	Complexity
Dynamic array	`vector<T>`	`push_back`, `pop_back`, `[]`	O(1) amortized push_back

Need	C++ structure	Key methods	Complexity
Hash map/set	`unordered_map` / `unordered_set`	O(1) avg	
Sorted map/set	`map` / `set`	O(log n)	
Stack	`stack<T>`	`push`, `pop`, `top`	O(1)
Queue	`queue<T>`	`push`, `pop`, `front`	O(1)
Priority queue (max-heap default!)	`priority_queue<T>`	`push`, `pop`, `top`	O(log n)
Double-ended queue	`deque<T>`	`push_front/back`, `pop_front/back`	O(1)
Bit set	`bitset<N>`	fixed-size, fast bit ops	O(1) per bit

Common gotchas in C++: `priority_queue` is a **max-heap by default** (opposite of Java's `PriorityQueue`!) — use `priority_queue<int, vector<int>, greater<int>>` for a min-heap. Iterator invalidation after modifying a container while iterating is a classic source of undefined behavior.

28.4 Choosing Your Interview Language

Pick the language you're most fluent in — interviewers care about correct logic and clean code, not language trivia. That said:

- **Java/C++** are common defaults at large companies (strong typing catches some bugs early, but syntax is more verbose).
- **Python** is often praised for speed of writing/reading during time-boxed interviews (concise syntax, built-in high-level structures like `heapq`, `Counter`, `deque`).
- Whatever you choose, **be fluent in its standard library's data structure names and complexities** — fumbling with "how do I even create a queue in this language" wastes precious interview minutes.

---

29. Quick-Reference Problem Index

(Master Table)

> A scannable, revision-friendly index of every problem referenced in this guide, tagged by pattern and core idea. Use this as a final checklist before an interview — if you can recall the core idea for every row without looking, you are ready.

#	Problem	Pattern	Core Idea (one line)
1	Two Sum	Hashing	HashMap of value→index, check complement
2	Two Sum II (sorted)	Two Pointers	left/right pointers move based on sum vs target
3	3Sum	Two Pointers	sort + fix one + two-pointer on rest
4	4Sum	Two Pointers	sort + fix two + two-pointer on rest
5	Container With Most Water	Two Pointers	always move the shorter line's pointer
6	Trapping Rain Water	Two Pointers / Monotonic Stack	min(leftMax,rightMax) - height[i]
7	Valid Palindrome	Two Pointers	skip non-alnum, compare from both ends
8	Remove Duplicates from Sorted Array	Two Pointers	slow pointer marks next unique slot
9	Sort Colors (Dutch Flag)	Two/Three Pointers	low/mid/high partition in one pass
10	Max Sum Subarray of Size K	Sliding Window (fixed)	slide by adding new, removing old
11	Longest Substring Without Repeating Chars	Sliding Window (variable)	shrink left on duplicate
12	Minimum Window Substring	Sliding Window (variable)	expand until valid, shrink while valid
13	Fruit Into Baskets	Sliding Window	at most 2 distinct values

#	Problem	Pattern	Core Idea (one line)
14	Permutation in String	Sliding Window (fixed)	fixed window, compare frequency maps
15	Find All Anagrams in a String	Sliding Window (fixed)	same as above, collect all matches
16	Longest Repeating Character Replacement	Sliding Window	windowSize - maxFreq <= k
17	Max Consecutive Ones III	Sliding Window	shrink when zero-count > k
18	Kadane's Max Subarray	1D DP / Greedy	reset running sum when negative
19	Maximum Product Subarray	1D DP	track both max and min running product
20	Missing Number	Cyclic Sort / Math	expectedSum - actualSum, or XOR
21	Find All Duplicates	Cyclic Sort	negate value at index abs(x)-1
22	First Missing Positive	Cyclic Sort	place each value at index value-1
23	Set Mismatch	Cyclic Sort	same negative-marking / cyclic idea
24	Merge Intervals	Merge Intervals	sort by start, merge if overlap
25	Insert Interval	Merge Intervals	linear scan, merge overlapping
26	Non-overlapping Intervals	Merge Intervals / Greedy	sort by end time, greedy keep
27	Meeting Rooms II	Merge Intervals / Heap	min-heap of end times, or two sorted arrays
28	Interval List Intersections	Merge Intervals	two-pointer walk on both lists
29	Rotate Array	Arrays	reverse whole, reverse parts

#	Problem	Pattern	Core Idea (one line)
30	Rotate Image (Matrix)	Arrays	transpose then reverse rows
31	Spiral Matrix	Arrays	shrinking boundary traversal
32	Search a 2D Matrix	Binary Search	treat as flattened sorted array
33	Valid Anagram	Hashing	frequency count array
34	Group Anagrams	Hashing	sorted string / signature as key
35	Longest Palindromic Substring	Two Pointers (expand)	expand around every center
36	Longest Consecutive Sequence	Hashing	only start counting at sequence start
37	Subarray Sum Equals K	Hashing / Prefix Sum	prefix sum + count map
38	Reverse Linked List	LinkedList Reversal	prev/curr/next pointer flip
39	Reverse Linked List II	LinkedList Reversal	reverse a sub-range
40	Reverse Nodes in k-Group	LinkedList Reversal	reverse each group of k
41	Linked List Cycle	Fast & Slow Pointers	Floyd's tortoise and hare
42	Find Cycle Start	Fast & Slow Pointers	reset one pointer to head after meeting
43	Middle of Linked List	Fast & Slow Pointers	fast reaches end, slow at middle
44	Palindrome Linked List	Fast & Slow + Reversal	find mid, reverse half, compare
45	Merge Two Sorted Lists	Merge / Two Pointers	classic merge step

#	Problem	Pattern	Core Idea (one line)
46	Merge k Sorted Lists	K-Way Merge	min-heap of list heads
47	Remove N-th Node From End	Two Pointers	fast pointer n steps ahead
48	Add Two Numbers	LinkedList	simulate digit addition with carry
49	Copy List with Random Pointer	LinkedList / Hashing	interleave clones or use HashMap
50	Swap Nodes in Pairs	LinkedList Reversal	pairwise pointer rewiring
51	Rotate List	LinkedList	make circular, break at new tail
52	Reorder List	LinkedList (combo)	find mid + reverse + merge
53	Valid Parentheses	Stack	push open, match on close
54	Min Stack	Stack	secondary stack tracks running min
55	Evaluate Reverse Polish Notation	Stack	push operands, apply on operator
56	Next Greater Element	Monotonic Stack	decreasing stack of indices
57	Daily Temperatures	Monotonic Stack	same as above, store day distance
58	Largest Rectangle in Histogram	Monotonic Stack	increasing stack, pop to compute area
59	Sliding Window Maximum	Monotonic Deque	decreasing deque of indices
60	Remove K Digits	Monotonic Stack	pop bigger digits before smaller ones
61	Online Stock Span	Monotonic Stack	decreasing stack of (price, span)
62	132 Pattern	Monotonic Stack	track "third" value from right to left

#	Problem	Pattern	Core Idea (one line)
63	Subsets	Backtracking	include/exclude at every element
64	Permutations	Backtracking	swap-based or used[] array
65	Combination Sum	Backtracking	reuse allowed, prune with sorted early exit
66	Combination Sum II	Backtracking	skip duplicates at same recursion depth
67	N-Queens	Backtracking	attacked columns/diagonals tracked in O(1)
68	Sudoku Solver	Backtracking	row/col/box validity sets
69	Word Search	Backtracking / DFS	DFS with temporary visited marking
70	Generate Parentheses	Backtracking	track open/close counts used
71	Letter Combinations of Phone Number	Backtracking	try every letter mapping per digit
72	Palindrome Partitioning	Backtracking	only recurse on palindrome prefixes
73	Merge Sort	Sorting	divide, recurse, merge
74	Quick Sort	Sorting	partition around pivot, recurse both sides
75	Heap Sort	Sorting	build max-heap, repeatedly extract max
76	Kth Largest Element	Quickselect / Heap	partition-based O(n) average
77	Binary Search (template)	Binary Search	low<=high, mid=low+(high-low)/2
78	First and Last Position in Sorted Array	Binary Search	two biased binary searches

#	Problem	Pattern	Core Idea (one line)
79	Search in Rotated Sorted Array	Modified Binary Search	identify sorted half each step
80	Find Minimum in Rotated Sorted Array	Modified Binary Search	compare mid vs high
81	Find Peak Element	Modified Binary Search	move toward the increasing side
82	Koko Eating Bananas	Binary Search on Answer	binary search speed, check feasibility
83	Capacity to Ship Packages Within D Days	Binary Search on Answer	binary search capacity
84	Split Array Largest Sum	Binary Search on Answer	binary search max subarray sum
85	Median of Two Sorted Arrays	Modified Binary Search	binary search partition on smaller array
86	Find K Closest Elements	Modified Binary Search	binary search left boundary of window
87	Binary Tree Level Order Traversal	Tree BFS	queue, snapshot level size
88	Zigzag Level Order Traversal	Tree BFS	reverse alternate levels
89	Binary Tree Right Side View	Tree BFS/DFS	last node per level
90	Populating Next Right Pointers	Tree BFS	use existing next pointers, O(1) space
91	Minimum Depth of Binary Tree	Tree BFS	stop early at first leaf
92	Maximum Depth of Binary Tree	Tree DFS	1 + max(left, right)
93	Diameter of Binary Tree	Tree DFS	combine left+right height at each node
94	Balanced Binary Tree	Tree DFS	-1 sentinel short-circuits imbalance

#	Problem	Pattern	Core Idea (one line)
95	Lowest Common Ancestor (general tree)	Tree DFS	propagate found targets upward
96	Lowest Common Ancestor (BST)	Tree DFS / BST property	compare values to decide direction
97	Validate BST	Tree DFS	pass down (min,max) range
98	Serialize/Deserialize Binary Tree	Tree DFS	preorder + null markers
99	Path Sum II	Tree DFS / Backtracking	track path + remaining sum
100	Binary Tree Maximum Path Sum	Tree DFS	return best downward path, track global peak
101	Sum Root to Leaf Numbers	Tree DFS	carry running number down
102	Construct Tree from Preorder/Inorder	Tree Construction	hashmap of value→inorder index
103	Maximum Width of Binary Tree	Tree BFS	positional indexing per node
104	Trie Insert/Search/StartsWith	Trie	26-array children per node
105	Kth Largest Element in a Stream	Top K / Heap	min-heap of size k
106	Top K Frequent Elements	Top K / Heap / Bucket Sort	heap O(n log k) or bucket sort O(n)
107	K Closest Points to Origin	Top K / Heap	max-heap of size k on distance
108	Sort Characters By Frequency	Top K / Heap	frequency map + sort/bucket
109	Reorganize String	Top K / Greedy Heap	alternate top two most-frequent chars
110	Find Median from Data Stream	Two Heaps	max-heap (lower) + min-heap (upper)

#	Problem	Pattern	Core Idea (one line)
111	Sliding Window Median	Two Heaps	+ lazy deletion for elements leaving window
112	IPO / Maximize Capital	Two Heaps	min-heap capital + max-heap profit
113	Merge k Sorted Lists/Arrays	K-Way Merge	heap of one candidate per source
114	Kth Smallest in Sorted Matrix	K-Way Merge / Binary Search	heap or binary search on value range
115	Smallest Range Covering K Lists	K-Way Merge	heap + track running max
116	Find K Pairs with Smallest Sums	K-Way Merge	heap seeded with first row pairs
117	Number of Islands	Graph DFS/BFS	flood fill each unvisited land cell
118	Number of Connected Components	Graph / Union-Find	DFS/BFS or DSU
119	Course Schedule I/II	Topological Sort	Kahn's algorithm, check full ordering
120	Alien Dictionary	Topological Sort	derive edges from adjacent word diffs
121	Minimum Height Trees	Topological-like (leaf trimming)	peel leaves layer by layer
122	Redundant Connection	Union-Find	first edge causing a same-root union
123	Number of Provinces	Union-Find / DFS	connected components via matrix
124	Dijkstra's Shortest Path	Graph / Heap	greedy expand closest unvisited node
125	Bellman-Ford	Graph / DP	relax all edges V-1 times
126	Floyd-Warshall	Graph / DP	all-pairs via intermediate node k

#	Problem	Pattern	Core Idea (one line)
127	Kruskal's MST	Greedy / Union-Find	sort edges, add if no cycle
128	Prim's MST	Greedy / Heap	grow tree via cheapest connecting edge
129	Is Graph Bipartite	Graph Coloring	2-color via BFS/DFS
130	Rotting Oranges	Multi-source BFS	all rotten oranges start in queue
131	01 Matrix	Multi-source BFS	all zeros start in queue
132	Climbing Stairs	1D DP	dp[i]=dp[i-1]+dp[i-2]
133	House Robber	1D DP	dp[i]=max(dp[i-1], dp[i-2]+nums[i])
134	House Robber II	1D DP	run House Robber twice (exclude first/last)
135	House Robber III	Tree DP	return (robbed, notRobbed) pair per node
136	Coin Change	Unbounded Knapsack DP	dp[amt]=min(dp[amt-coin]+1)
137	Coin Change II	Unbounded Knapsack DP	coins outer loop, amounts inner loop
138	Subset Sum	0/1 Knapsack DP	dp[w] = dp[w] OR dp[w-num]
139	Partition Equal Subset Sum	0/1 Knapsack DP	subset sum with target = total/2
140	Target Sum	0/1 Knapsack DP	transform +/- assignment into subset sum
141	Longest Common Subsequence	LCS DP	dp[i][j] match/no-match recurrence
142	Edit Distance	LCS-family DP	min(insert, delete, replace)+1

#	Problem	Pattern	Core Idea (one line)
143	Longest Palindromic Subsequence	LCS DP	LCS(s, reverse(s))
144	Longest Increasing Subsequence	LIS DP / Binary Search	O(n²) DP or O(n log n) patience sorting
145	Russian Doll Envelopes	LIS (2D)	sort width asc / height desc, LIS on height
146	Unique Paths	Grid DP	dp[i][j]=dp[i-1][j]+dp[i][j-1]
147	Minimum Path Sum	Grid DP	dp[i][j]=cost+min(up,left)
148	Dungeon Game	Grid DP (reverse)	compute backwards from bottom-right
149	Palindrome Partitioning II	Interval + 1D DP	precompute isPalindrome, then min cuts
150	Matrix Chain Multiplication	Interval DP	try every split point k
151	Word Break	1D DP	dp[i] = OR over j of dp[j] && dict has s[j:i]
152	Decode Ways	1D DP	dp[i]=dp[i-1]+dp[i-2] with validity checks
153	Interleaving String	2D DP	dp[i][j] from top or left match
154	Maximal Square	Grid DP	min(up,left,diag)+1 on a 1-cell
155	Regular Expression Matching	2D DP	special handling of * and .
156	Perfect Squares	Unbounded Knapsack DP	squares as "coins"
157	Best Time to Buy/Sell Stock	State Machine DP / Greedy	track min price, max profit
158	Best Time to Buy/Sell Stock II	Greedy	sum all positive day deltas
159	Best Time to Buy/Sell Stock III/IV	State Machine DP	dp[i][k][holding]

#	Problem	Pattern	Core Idea (one line)
160	Activity Selection / Non-overlapping Intervals	Greedy	sort by end time
161	Jump Game	Greedy	track farthest reachable index
162	Jump Game II	Greedy (BFS-like)	track currentEnd/farthest, count jumps
163	Gas Station	Greedy	reset start candidate when tank goes negative
164	Task Scheduler	Greedy / Math	formula from max frequency task
165	Huffman Encoding	Greedy / Heap	merge two least-frequent nodes repeatedly
166	Minimum Number of Platforms	Greedy / Two Pointers	sort arrivals/departures, merge scan
167	Fractional Knapsack	Greedy	sort by value/weight ratio
168	Segment Tree Range Sum	Advanced DS	build/update/query in O(log n)
169	Fenwick Tree Range Sum	Advanced DS	i & (-i) bit trick
170	LRU Cache	Advanced DS Design	HashMap + Doubly Linked List
171	LFU Cache	Advanced DS Design	+ frequency buckets, track min freq
172	Design Twitter	Design + K-Way Merge	heap merge of followee tweets
173	Design Hit Counter	Design + Queue	timestamp queue, evict old entries
174	Design Rate Limiter	Design	token bucket / sliding window counter
175	Design Snake Game	Design	deque body + hashset collision check

30. Frequently Asked Questions

Q: Do I really not need to solve 500+ problems? A: Correct — the value of solving problems drops sharply after you've internalized a pattern (Section 19.5 suggests 5-8 per pattern is usually enough). Beyond that, more problems mostly build speed/confidence, not new understanding. Time is better spent on mock interviews and revisiting your mistake log.

Q: Should I memorize code solutions? A: No — memorize the **recurrence/insight**, not the exact code. Interviewers routinely tweak well-known problems slightly (different constraints, an added twist), and memorized code that doesn't map to the *reasoning* falls apart immediately under a tweak. Re-derive from the pattern every time you practice.

Q: What if I get a problem I've genuinely never seen before? A: This is normal, even at the senior level. Fall back to Section 21.4's process: clarify, small example by hand, brute force first, connect to the closest of the 16 patterns, ask for a hint if truly stuck. Interviewers weigh *process* heavily, not just the final answer.

Q: How important is exact syntax correctness? A: Less than logical correctness, but it still matters — sloppy syntax signals lack of fluency. Aim for code that would compile/run with at most minor fixable typos, and verbally acknowledge any syntax you're unsure of ("I believe this is the right method name, let me double check...").

Q: Is it bad to start with brute force? A: No — it's expected and often required. It establishes a correctness baseline, demonstrates you understand the problem, and gives the interviewer a checkpoint before you dive into optimization. Just don't STOP at brute force without attempting to optimize (unless explicitly told the brute force is sufficient).

Q: How do I know if a DP recurrence is correct before coding it? A: Trace it by hand on the smallest non-trivial example (Section 27.5's dry-run technique). If the hand-traced table matches the expected answer, you have strong confidence before investing time in code.

Q: What's the single highest-leverage topic to study if I only have one week? A: Sections 4 (Arrays/Strings incl. Two Pointers & Sliding Window), 5 (Hashing), 10 (Binary Search), and 14.1-14.7 (core DP: 1D DP, Knapsack). These four areas alone cover the plurality of real interview questions across companies.

Q: How do I practice "explaining out loud" if I study alone? A: Literally talk through your reasoning aloud while solving (yes, even alone) — this builds the verbal-articulation muscle independent of having a listener. Recording yourself and reviewing it afterward is even more effective at catching unclear explanations.

---

31. Closing Summary

You now have, in a single reference:

- A first-principles understanding of complexity analysis.
- Every foundational and advanced data structure, explained with working code.
- Every major algorithm family (sorting, searching, recursion, backtracking, greedy, DP, graph algorithms) with brute-force-to-optimized reasoning.
- 16 reusable interview patterns with templates and trigger phrases.
- A ~175-problem quick-reference index (Section 29) plus fully worked explanations for the majority of those problems throughout Sections 4-18.
- Advanced bonus material (graph theory extensions, string algorithms, combinatorics, design questions) for depth beyond the core 16 patterns.
- A realistic week-by-week study plan that explicitly avoids the "grind 500 problems" trap.
- Numeric dry-runs for the algorithms most prone to subtle bugs.
- A language cheat sheet so you don't lose time on syntax during a real interview.
- Interview-day communication templates and a mistake-prevention checklist.

The path forward: follow the 8-week roadmap (Section 19), maintain your mistake log, do timed mocks weekly starting from Week 3 onward, and revisit Sections 17 (pattern cheat sheet) and 29 (problem index) the day before any interview. Depth of understanding, not breadth of memorized solutions, is what will carry you through the interview room.

Good luck — now go build that mistake log and start Week 1.

End of guide.